

Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática

Unidade Curricular de Laboratórios de Informática IV

Ano Letivo de 2025/2026

Sistema de gestão integrada para uma cadeia de pequenas lojas de conveniência

Francisco Soares(106882) Carlos Silva(106792) Nuno Soares(107366)
Diogo Silva(97941) Rodrigo Guimarães(93236)

3 de maio de 2026

LI4

Resumo

O presente relatório documenta as fases de **engenharia de requisitos** (Etapa 1) e de **arquitetura e design** (Etapa 2) do sistema **Belo Mercado Manager**, uma plataforma de gestão integrada encomendada por uma cadeia de retalho de proximidade composta por cinco lojas na zona norte de Portugal. Os capítulos relativos à implementação (Etapa 3) e à verificação e validação (Etapa 4) serão preenchidos nas entregas correspondentes do projeto. A equipa fez uso de técnicas de eliciação e modelação assistidas por *Large Language Models* (LLM) atuando como aceleradores de processo, permitindo a identificação de *stakeholders*, a simulação de entrevistas, a elaboração de *user stories*, a modelação de casos de uso segundo o formato expandido de Ian Sommerville, e o desenho da arquitetura em camadas, da camada de dados e da especificação da API.

Como resultado, foram identificados 7 módulos funcionais, 19 requisitos funcionais, 16 requisitos não-funcionais, 29 *user stories* e 19 casos de uso detalhados, cobrindo os principais fluxos operacionais do sistema, desde a autenticação e gestão de vendas até ao controlo de inventário, gestão de fornecedores e geração de relatórios analíticos. Sobre esta base de requisitos, foi desenhada uma arquitetura em camadas com instância única centralizada (Blazor/.NET, SQL Server, JWT, API REST), documentada por diagramas de classes, sequência e componentes, modelo lógico de dados, especificação de API, dez *Architecture Decision Records* e seis *wireframes* dos ecrãs principais.

Área de Aplicação: Engenharia de *Software*; Sistemas de Informação para Gestão de Retalho.

Palavras-Chave: Gestão de Lojas de Conveniência, Engenharia de Requisitos, Casos de Uso, *User Stories*, LLM, Blazor, .NET, Sistemas de Informação, Retalho.

Índice

1 Introdução **1**

1.1 Apresentação do caso de estudo 2
1.2 Motivação e Objetivos 2
1.3 Estrutura do relatório 2

2 Definição do domínio do sistema **4**

2.1 Contextualização 4
2.2 Apresentação do Caso de Estudo 5
2.3 Motivação e Objetivos 6
2.4 Análise de Contexto e Restrições 8
2.4.1 Contexto Operacional 8
2.4.2 Restrições Técnicas 8
2.4.3 Restrições Legais e Regulamentares 9
2.4.4 Restrições Operacionais 9
2.4.5 Restrições de Projeto 9
2.5 Viabilidade 9
2.6 Recursos Materiais e Humanos 10
2.6.1 Equipa de Trabalho e *Stakeholders* 11
2.7 Plano de Execução 12

3 Conceção e Engenharia de Requisitos Assistida por LLM **13**

3.1 Métodos de Levantamento de Requisitos 13
3.2 Elicitação de Requisitos 13
3.3 Simulação de Entrevistas com *Stakeholders* 14
3.3.1 Entrevista 1 - Administrador da Cadeia (Sr. Manuel Correia, 55 anos) . 15
3.3.2 Entrevista 2 - Gerente de Loja (D. Teresa Vieira, 38 anos) 15
3.3.3 Entrevista 3 - Operador de Caixa (João Martins, 22 anos) 15
3.3.4 Entrevista 4 - Fornecedor (Distribuição Almeida, Lda.) 15
3.3.5 Entrevista 5 - Contabilista (Dr. Carlos Ferreira, 48 anos) 15
3.4 *User Stories* 16
3.5 Requisitos Funcionais e Não Funcionais 19
3.6 Análise de Requisitos — Ambiguidades e Refinamentos 22
3.7 Casos de Uso 24
3.7.1 Descrição Detalhada dos Casos de Uso 27
3.7.1.1 UC01 — Registar Venda (POS) 27
3.7.1.2 UC15 — Autenticar no Sistema 28

3.8 Modelo de Domínio 28
3.9 Diagrama de casos de uso 29

4 Arquitetura e Design do Software utilizando LLM 32
4.1 Visão Geral da Arquitetura 32
4.2 Arquitetura da Camada de Negócios 34
4.2.1 Diagrama de Classes 34
4.2.2 Diagramas de Sequência 37
4.2.2.1 SD-UC01 — Registar Venda (POS) 37
4.2.2.2 SD-UC04 — Abrir e Fechar Caixa 38
4.2.2.3 SD-UC10 — Criar Encomenda a Fornecedor 39
4.2.2.4 SD-UC12 — Consolidar Dados Diários 40
4.2.2.5 SD-UC15 — Autenticar no Sistema 41
4.2.3 Diagrama de Componentes 42
4.3 Arquitetura da Camada de Dados 45
4.3.1 Modelo Lógico 45
4.4 Especificação da API 47
4.5 Decisões Arquiteturais 50
4.6 Proposta de Interface 56
4.6.1 POS — Ecrã de Venda em Curso (UC01) 57
4.6.2 POS — Ecrã de Pagamento (UC01) 58
4.6.3 POS — Fecho de Caixa (UC04) 59
4.6.4 *Backoffice — Dashboard* de KPIs (UC13) 60
4.6.5 *Backoffice* — Gestão de Catálogo (UC05) 61
4.6.6 *Backoffice* — Gestão de Utilizadores (UC16) 62
4.7 Matriz de Rastreabilidade 64

5 Implementação e Desenvolvimento por LLM 68

6 Verificação, Validação e Avaliação da Qualidade do Software Produzido 69

7 Conclusões e Trabalho Futuro 70
7.1 Conclusões 70
7.2 Trabalho Futuro 70

Lista de Siglas e Acrónimos 73

Anexos 75
Anexo A — Transcrição das Entrevistas Simuladas 75
Anexo B — Casos de Uso Detalhados (UC02–UC19) 81
Anexo C — Cronograma e Diagrama de Gantt 94
Anexo D — Diagrama UML Geral de Casos de Uso 94

Lista de Figuras

3.1 Modelo de Domínio 29
3.2 Diagrama de Use Cases 30

4.1 Arquitetura global da solução. 33
4.2 Diagrama de classes do sistema. 36
4.3 SD-UC01 — Registar Venda. 38
4.4 SD-UC04 — Abrir e Fechar Caixa. 39
4.5 SD-UC10 — Criar Encomenda a Fornecedor. 40
4.6 SD-UC12 — Consolidar Dados Diários (topologia centralizada). 41
4.7 SD-UC15 — Autenticar no Sistema. 42
4.8 Diagrama de componentes. 44
4.9 Modelo lógico de dados. 46
4.10 WF-01 — POS: ecrã de venda em curso (UC01). 58
4.11 WF-02 — POS: ecrã de pagamento (UC01). 59
4.12 WF-03 — POS: fecho de caixa (UC04). 60
4.13 WF-04 — *Backoffice*: *dashboard* de KPIs (UC13). 61
4.14 WF-05 — *Backoffice*: gestão de catálogo de produtos (UC05). 62
4.15 WF-06 — *Backoffice*: gestão de utilizadores (UC16). 63

Lista de Tabelas

2.1 Restrições técnicas. 8
2.2 Restrições legais e regulamentares. 9
2.3 Restrições operacionais. 9
2.4 Restrições de projeto. 9
2.5 Equipa de trabalho e responsabilidades. 11
2.6 *Stakeholders* do sistema (resumo). 12
2.7 Timeline de execução do projeto por etapas. 12

3.1 *User stories* — Administrador da Cadeia. 17
3.2 *User stories* — Gerente de Loja. 18
3.3 *User stories* — Operador de Caixa. 18
3.4 *User stories* — Administrador do Sistema (IT). 19
3.5 Requisitos funcionais do sistema (19 RF organizados por módulo). 20
3.6 Requisitos não-funcionais do sistema. 21
3.7 Correspondência entre casos de uso e requisitos funcionais. 25
3.8 Caso de uso UC01 — Registar Venda (POS). 27
3.9 Caso de uso UC15 — Autenticar no Sistema. 28

4.1 Especificação da API REST do sistema Belo Mercado Manager — Parte 1. . . 48
4.2 Especificação da API REST do sistema Belo Mercado Manager — Parte 2. . . 49
4.3 Matriz de rastreabilidade — requisitos funcionais. 65
4.4 Matriz de rastreabilidade — requisitos não-funcionais. 66

7.1 Caso de uso UC02 — Anular Venda e Processar Devolução. 81
7.2 Caso de uso UC03 — Aplicar Promoções e Descontos. 82
7.3 Caso de uso UC04 — Abrir e Fechar Caixa. 83
7.4 Caso de uso UC05 — Gerir Catálogo de Produtos. 84
7.5 Caso de uso UC06 — Consultar e Monitorizar *Stock*. 84
7.6 Caso de uso UC07 — Realizar Inventário Manual. 85
7.7 Caso de uso UC08 — Gerar Sugestões de Reposição. 86
7.8 Caso de uso UC09 — Gerir Fornecedores. 87
7.9 Caso de uso UC10 — Criar Encomenda a Fornecedor. 88
7.10 Caso de uso UC11 — Registar Receção de Mercadoria. 89
7.11 Caso de uso UC12 — Consolidar Dados Diários. 90
7.12 Caso de uso UC13 — Consultar *Dashboard* de KPIs. 91
7.13 Caso de uso UC14 — Gerar e Exportar Relatórios. 91
7.14 Caso de uso UC16 — Gerir Utilizadores e Perfis. 92

7.15 Caso de uso UC17 — Editar Perfil Próprio. 92
7.16 Caso de uso UC18 — Consultar Logs de Auditoria. 93
7.17 Caso de uso UC19 — Gerir Lojas da Cadeia. 93

1 Introdução

O presente relatório foi elaborado no âmbito da unidade curricular de Laboratórios de Informática IV e documenta o trabalho desenvolvido ao longo do projeto prático proposto. Este trabalho enquadra-se numa perspetiva atual da Engenharia de Software, em que o desenvolvimento de sistemas deixou de assentar apenas em processos tradicionais de análise, implementação e teste, passando também a integrar novas ferramentas de apoio à decisão, à produção técnica e à validação de soluções. Entre essas ferramentas, destacam-se os *Large Language Models* (LLMs), cuja utilização tem vindo a ganhar relevância nas várias fases do ciclo de vida do *software*.

No entanto, a utilização de LLMs neste contexto não foi encarada de forma meramente instrumental ou acrítica. Um dos aspetos centrais deste trabalho consistiu precisamente em reconhecer que o valor destas ferramentas depende da qualidade da sua utilização, da capacidade de interpretação dos resultados que produzem e da validação técnica realizada pelos próprios engenheiros de *software*. Nesse sentido, o desenvolvimento do projeto implicou não só o recurso aos LLMs como apoio ao trabalho, mas também uma análise crítica do seu contributo, dos seus limites e dos riscos associados à sua utilização, nomeadamente ao nível da fiabilidade das respostas, da adequação das soluções sugeridas e da necessidade de mecanismos de governação, revisão e responsabilidade técnica.

Interação com o LLM

Prompt utilizado
Com base no enunciado do trabalho prático, escreve uma introdução para o respetivo relatório. A resposta deve enquadrar o trabalho no contexto atual da Engenharia de Software e mencionar o papel que os Large Language Models (LLMs) irão tomar ao longo do projeto.

Análise crítica
A resposta gerada pelo LLM foi em geral alinhada com o contexto académico do relatório. O texto incluía corretamente a referência ao enquadramento da Engenharia de Software e ao papel dos LLMs no desenvolvimento do trabalho. Ainda assim, foram feitas algumas alterações com o objetivo de melhorar a sua adequação ao estilo pretendido para o relatório. Em particular, a resposta foi encurtada e simplificada em alguns pontos, de forma a torná-lo mais direto e natural, evitando respostas excessivamente densas.

1.1 Apresentação do caso de estudo

O caso de estudo deste relatório corresponde à cadeia de retalho de proximidade **Belo Mercado**, composta por cinco lojas na zona norte de Portugal. O sistema proposto, **Belo Mercado Manager**, pretende suportar a operação diária de cada loja e a consolidação central de informação para apoio à decisão.

Numa perspetiva funcional, o sistema cobre os processos nucleares de venda, inventário, fornecedores, encomendas e relatórios, assegurando rastreabilidade e consistência dos dados entre as várias lojas. A estrutura de dados de referência inclui entidades como *Loja*, *Produto*, *Venda*, *Stock*, *Fornecedor*, *Encomenda*, *Cliente* e *Utilizador*, que suportam os principais fluxos operacionais.

1.2 Motivação e Objetivos

A cadeia Belo Mercado opera atualmente com sistemas heterogéneos — algumas lojas com caixas registadoras simples, outras com folhas de cálculo — o que limita a visibilidade global do negócio, dificulta o controlo de inventário e torna penosa a consolidação de dados para reporte financeiro. Esta fragmentação cria fricção operacional crescente à medida que a cadeia se expande.

O sistema **Belo Mercado Manager** visa endereçar estas limitações através da centralização da informação operacional, do suporte ao registo de vendas, da gestão integrada de inventário e fornecedores, e da disponibilização de indicadores fiáveis de gestão. A caracterização detalhada do problema, o conjunto completo de objetivos operacionais e a análise de contexto e restrições são apresentados no Capítulo 2.

1.3 Estrutura do relatório

O presente relatório está organizado em sete capítulos, alinhados com as quatro etapas do projeto:

- **Capítulo 1 – Introdução**: apresenta o enquadramento geral do projeto, o caso de estudo, a motivação e os objetivos, e descreve a estrutura do relatório.
- **Capítulo 2 – Definição do Domínio do Sistema**: caracteriza o contexto de negócio da cadeia Belo Mercado, fundamenta a necessidade do sistema, apresenta os objetivos operacionais, analisa a viabilidade, descreve os *stakeholders* e os recursos envolvidos e apresenta o plano de execução do projeto.
- **Capítulo 3 – Conceção e Engenharia de Requisitos Assistida por LLM** (Etapa 1):

documenta o processo de engenharia de requisitos, incluindo os métodos de levantamento, as entrevistas com *stakeholders*, as *user stories*, os requisitos funcionais e não funcionais, a análise de ambiguidades e os casos de uso.

- **Capítulo 4 – Arquitetura e Design do *Software* utilizando LLM** (Etapa 2): apresenta a visão geral da arquitetura, os modelos de classes, sequência e componentes, o modelo lógico de dados, a especificação da API REST, as decisões arquiteturais (ADRs), a proposta de interface e a matriz de rastreabilidade entre requisitos e artefactos.

- **Capítulo 5 – Implementação e Desenvolvimento por LLM** (Etapa 3): documentará a implementação incremental dos módulos do sistema com apoio de LLM. Conteúdo a desenvolver na entrega correspondente.

- **Capítulo 6 – Verificação, Validação e Avaliação da Qualidade do *Software* Produzido** (Etapa 4): cobrirá a estratégia e os resultados de testes, validação funcional e avaliação de qualidade do produto. Conteúdo a desenvolver na entrega correspondente.

- **Capítulo 7 – Conclusões e Trabalho Futuro**: sintetiza os resultados das etapas concluídas e apresenta os próximos passos do projeto.

2 Definição do domínio do sistema

2.1 Contextualização

Nos últimos anos, o retalho de proximidade em Portugal tem vindo a ganhar relevância, impulsionado por hábitos de consumo orientados para conveniência, rapidez e compras mais frequentes e de menor volume. Esta tendência intensificou-se durante o período pandémico, quando as restrições e a maior permanência em casa aumentaram a procura por lojas de bairro e formatos de proximidade, e manteve-se nos anos seguintes com a valorização de soluções de compra "rápidas" e acessíveis no quotidiano.

É neste enquadramento que surge a cadeia Belo Mercado, que após iniciar atividade em 2019 com uma primeira loja tem acompanhado a evolução do setor e a procura por soluções mais convenientes, expandindo gradualmente a operação, sendo atualmente composta por cinco lojas na zona norte de Portugal. A Belo Mercado foca-se em necessidades imediatas (alimentação, higiene, limpeza e artigos do dia-a-dia), frequentemente em zonas residenciais e de passagem, onde a disponibilidade do produto e a rapidez do atendimento têm impacto direto na experiência do cliente.

Com o aumento do número de lojas, torna-se especialmente relevante garantir mecanismos eficazes de coordenação e controlo entre pontos de venda geograficamente dispersos. Em particular, a operação passa a depender da capacidade de recolher e agregar diariamente a informação gerada em cada loja — vendas, movimentos de *stock* e entradas de mercadoria — para que, ao final de cada dia, seja possível obter uma visão consolidada do desempenho da cadeia e suportar as decisões de gestão de forma consistente.

Interação com o LLM

Prompt utilizado
Com base no documento "CE01-Caso-de-Estudo-Mercearia" que contém algum conteúdo e nomenclatura e formas de construir conteúdo que devemos seguir. Desta forma apresenta o enquadramento do caso de estudo sobre o retalho de proximidade em Portugal pós-pandemia. Apresenta a empresa 'Belo Mercado' e descreve os seus desafios de gestão, focando na necessidade de consolidar dados diários de vendas e stocks para garantir o controlo e a coordenação da operação. De seguida, adiciona uma subsecção que estruture as fragilidades e os problemas atuais do negócio que fundamentam e justificam a adoção do sistema integrado.

Análise crítica
A resposta que o LLM gerou forneceu-nos uma boa base narrativa adequada para enquadrar o setor de retalho de proximidade e a evolução da cadeia Belo Mercado. Ainda assim, apresentava formulações demasiado gerais e algumas inferências que exigiram validação com o caso de estudo.
Na versão final, o texto foi ajustado para ter maior precisão e de forma a parecer um pouco mais realista, mais consistente a nível temporal e com maior foco no problema operacional, que era a necessidade de consolidação diária e coordenação entre as várias lojas.

2.2 Apresentação do Caso de Estudo

O Belo Mercado Manager é um sistema de *software* concebido para apoiar a gestão integrada da cadeia 'Belo Mercado'. O objetivo é assegurar que a informação operacional produzida em cada loja é recolhida, consolidada e disponibilizada de forma fiável ao nível central, permitindo ao administrador acompanhar o desempenho agregado e tomar decisões informadas, nomeadamente no que diz respeito ao controlo e reposição de inventário.

No dia-a-dia, cada loja atua de forma autónoma através de vendas, atendimento ao cliente e controlo do inventário local, com operadores de caixa e gerentes a assegurarem a execução operacional. No entanto, para manter uma visão única do negócio, é determinante que os dados relevantes sejam consolidados de forma consistente, garantindo a integridade da informação e reduzindo as discrepâncias entre lojas.

O desenvolvimento do Belo Mercado Manager deverá seguir boas práticas de Engenharia de *Software* e Engenharia de Requisitos, com particular foco na clareza, consistência e rastreabilidade da informação ao longo do ciclo de desenvolvimento. Assim, o sistema pretende equilibrar autonomia operacional e controlo central, suportando uma visão global contínua que permita melhorar o controlo de inventário e a tomada de decisão em toda a cadeia.

Interação com o LLM

Prompt utilizado
Escreve uma breve descrição do sistema de software, 'Belo Mercado Manager', que servirá para a gestão centralizada da cadeia Belo Mercado. Foca-te em 3 planos principais, objetivo operacional, dinâmica local vs central e rigor técnico.

Análise crítica
O LLM, neste prompt, respondeu de forma coerente com o objetivo da secção, destacando a dinâmica local vs. central e o papel do sistema proposto. No entanto, a proposta inicial fornecida era demasiado genérica em alguns pontos e não explicitava suficientemente o valor técnico da solução para suportar a rastreabilidade e consistência dos dados.
Sendo assim, fizemos uma revisão para reforçar a ligação entre a operação diária, a consolidação central e a tomada de decisão, preservando assim a clareza e objetividade do nosso caso de estudo. O resultado final ficou mais alinhado com o domínio de negócio e com a arquitetura funcional que descrevemos no relatório.

2.3 Motivação e Objetivos

A cadeia Belo Mercado opera atualmente com sistemas de registo heterogéneos: algumas lojas utilizam caixas registadoras simples sem integração digital, enquanto outras recorrem a folhas de cálculo Excel. Esta situação origina um conjunto de problemas operacionais relevantes que comprometem a eficiência e o controlo do negócio.

Em primeiro lugar, os registos manuais são suscetíveis a erros e omissões, levando à perda de informação no fecho diário. Em segundo lugar, a inexistência de um sistema integrado impede a realização de análises cruzadas entre lojas, dificultando a identificação de padrões de vendas e a avaliação global do desempenho da cadeia. Adicionalmente, a ausência de mecanismos automáticos de controlo de stock conduz frequentemente a ruturas, resultando em perdas de vendas e impacto negativo na satisfação dos clientes. Por fim, a falta de informação de faturação centralizada e estruturada dificulta a organização dos dados necessários ao cumprimento de obrigações fiscais e à análise rigorosa da situação financeira do negócio.

Perante estes problemas, torna-se necessária a implementação de um sistema integrado que permita resolver estas fragilidades e suportar de forma eficaz a operação da cadeia. Nesse sentido, o sistema Belo Mercado Manager deverá cumprir os seguintes objetivos principais:

1. Permitir o **registo de vendas** em cada loja, com identificação de produtos, quantidades, preços e respetivo IVA.

2. Simular a emissão de **faturas** em modo protótipo académico, com cálculo de IVA e dados fiscais essenciais, sem certificação fiscal AT, conforme definido na ADR-009.

3. Manter o **catálogo de produtos** atualizado e partilhado entre todas as lojas, com suporte a categorias e subcategorias.

4. Gerir o **inventário** de cada loja de forma autónoma, com alertas automáticos de *stock* mínimo.

5. Registar e gerir **fornecedores**, respetivos catálogos de preços e condições de fornecimento.

6. Permitir a criação e gestão de **encomendas** a fornecedores, com acompanhamento do estado da entrega.

7. Efetuar a **consolidação diária** dos dados de cada loja num sistema central.

8. Disponibilizar **relatórios e *dashboards*** de gestão, com filtros por loja, período e categoria de produto.

9. Implementar **controlo de acessos** por perfil de utilizador (administrador, gerente, operador de caixa).

10. Garantir a **segurança dos dados**, incluindo encriptação, *backups* e conformidade com o RGPD.

Interação com o LLM

Prompt utilizado
Faz uma análise detalhada da operação das lojas Belo Mercado. Descreve a diversidade de produtos e os desafios de gerir diferentes ritmos de reposição. Explica as consequências de não ter um sistema integrado e detalha como o 'Belo Mercado Manager' deve funcionar para apoiar as vendas diárias, a gestão de catálogo comum e a consolidação de dados ao final do dia para o administrador. Por fim, apresenta uma lista enumerada dos 10 objetivos operacionais core do sistema para contornar estas fragilidades.

Análise crítica
A proposta do LLM cobriu bem o problema operacional e sugeriu até bons objetivos pertinentes, mas trouxe uma redundância textual e um tom excessivamente promocional em algumas passagens (algo que notamos não só neste ponto, mas ao longo de todo o projeto). Além disso, alguns objetivos estavam formulados de modo pouco verificável. Na revisão, os objetivos foram consolidados em enunciados claros e orientados a resultado, mantendo assim um melhor encadeamento problema-resposta. O texto final foi simplificado para melhorar a legibilidade, reduzir ao máximo quaisquer repetições que possam existir e garantir a coerência com os requisitos funcionais e não funcionais definidos mais adiante.

2.4 Análise de Contexto e Restrições

A análise de contexto e restrições teve como finalidade compreender o ambiente operacional no qual o sistema será inserido e identificar as condicionantes — técnicas, legais, operacionais e de projeto — que enquadram e limitam as opções de conceção e desenvolvimento.

O contexto operacional foi caracterizado a partir das seguintes fontes: a descrição do caso de estudo, que define o modelo de negócio e a autonomia de cada loja; os resultados das entrevistas simuladas com os *stakeholders* (ver Secção 3.3 e Anexo A), que revelaram os processos atuais, as dificuldades sentidas e as expectativas de cada perfil; e a investigação de legislação aplicável, nomeadamente o RGPD e os requisitos da Autoridade Tributária relativos a faturação eletrónica (Portaria n.º 363/2010).

2.4.1 Contexto Operacional

- Cada loja funciona de forma **autónoma** (tipicamente 07h00–23h00), com consolidação diária.
- O sistema central agrega dados de **5 lojas** localizadas em diferentes cidades da zona norte de Portugal.
- Os operadores de caixa necessitam de uma interface **rápida e intuitiva**, sem exigência de formação técnica especializada.

2.4.2 Restrições Técnicas

ID	Restrição
RT01	O sistema deve ser desenvolvido em .NET (Blazor).
RT02	A base de dados deve ser relacional (SQL Server ou equivalente).
RT03	O sistema deve funcionar em navegador web.
RT04	A consolidação diária deve ser automatizada e resiliente a falhas.

Tabela 2.1: Restrições técnicas.

Nota: a versão inicial deste levantamento incluía uma restrição adicional (RT05) que exigia operação *offline* na loja com sincronização posterior ao sistema central. Após análise de *trade-offs* documentada na ADR-001 (Secção 4.5), esta restrição foi removida com a adoção da topologia de instância única centralizada, passando o sistema a depender da disponibilidade da ligação à Internet em cada loja para a operação POS. Os requisitos de disponibilidade (RNF-04 e RNF-05) endereçam este risco.

2.4.3 Restrições Legais e Regulamentares

ID	Restrição
RL01	Cumprimento do RGPD no tratamento de dados pessoais.
RL02	Reconhecimento dos requisitos da AT aplicáveis à faturação, ficando a certificação fiscal completa fora do âmbito do protótipo académico.
RL03	Preservação de dados financeiros por mínimo de 10 anos.

Tabela 2.2: Restrições legais e regulamentares.

2.4.4 Restrições Operacionais

ID	Restrição
RO01	Registo de venda em menos de 2 segundos.
RO02	O sistema não deve exigir formação técnica especializada.
RO03	Disponibilidade de 99,5% durante o horário de funcionamento.

Tabela 2.3: Restrições operacionais.

2.4.5 Restrições de Projeto

ID	Restrição
RP01	Equipa de 5 elementos.
RP02	Duração total de 5 meses (fevereiro a junho de 2026).
RP03	Metodologia sequencial e incremental, com 4 etapas definidas.
RP04	Desenvolvimento académico no âmbito de LI4 — Universidade do Minho.

Tabela 2.4: Restrições de projeto.

2.5 Viabilidade

Este projeto apresenta uma forte viabilidade assentando nos benefícios económicos e operacionais diretos que a implementação do sistema Belo Mercado Manager trará para a cadeia. Com a adoção de um sistema integrado centralizado, prevê-se que o negócio valide o investimento através das seguintes metas projetadas:

- **Aumento em cerca de 20% nas vendas**: Ao garantir alertas automáticos de níveis mínimos de *stock* e otimizar o processo de reposição nas 5 lojas, a cadeia evita ruturas

de produtos com elevada procura, convertendo antigas falhas de inventário em vendas efetivas.

- **Redução drástica da perda de itens**: A introdução de um sistema rigoroso de controlo e validação (como o agendamento de inventários globais e parciais e a justificação obrigatória de quebras) prevê uma diminuição significativa no desperdício de bens (especialmente perecíveis e de validade curta).

- **Otimização operacional dos recursos humanos**: A eliminação de procedimentos manuais assentes em papel e a consolidação automatizada de dados no final do dia permitirão realocar o tempo dos funcionários da frente de loja para um atendimento ao cliente mais ágil, melhorando a satisfação do consumidor.

- **Maior poder de negociação e controlo de dados**: Com o cruzamento em tempo real dos dados de vendas por loja e a gestão global da cadeia, a Administração terá melhores indicadores para negociar com fornecedores e aplicar estratégias de descontos/promoções.

Interação com o LLM

Prompt utilizado
Atua como um Gestor de Projeto. Escreve a secção de 'Viabilidade e Benefícios Esperados' para o sistema Belo Mercado Manager. O texto deve convencer os *stakeholders* de que o investimento vale a pena, focando-se em: metas quantitativas, eficiência operacional, vantagem estratégica e escolha tecnológica. O tom deve ser persuasivo, demonstrando que o projeto tem um retorno tangível e é tecnicamente viável.

Análise crítica
O LLM produziu uma base útil para discutir a viabilidade e os benefícios esperados, com boa identificação de ganhos operacionais e estratégicos. Contudo, alguns valores apresentados surgiram como projeções otimistas sem explicitação de premissas (e.g. o aumento inicial de vendas em cerca de 75 %)
O nosso grupo manteve os indicadores como metas projetadas e ajustou a linguagem para evitar afirmações não comprovadas nesta fase. O texto final ficou, então, com uma maior prudência técnica e um melhor enquadramento de risco.

2.6 Recursos Materiais e Humanos

Para o desenvolvimento do presente projeto foi necessário alocar um conjunto de recursos humanos adequados à natureza do trabalho. Em particular, o projeto conta com 5 elementos da equipa de desenvolvimento, responsáveis pelas várias atividades associadas à conceção, análise, especificação e desenvolvimento do sistema. A execução técnica é conduzida com supervisão de arquitetura e acompanhamento metodológico contínuo ao longo das diferentes etapas do projeto. Considera-se ainda que a validação do sistema exige contacto com os

potenciais intervenientes do domínio, nomeadamente utilizadores e responsáveis pela gestão da cadeia de lojas, de forma a apoiar o levantamento e o refinamento dos requisitos.

No que diz respeito aos recursos materiais, foi necessário dispor de *hardware* adequado, nomeadamente os computadores pessoais dos elementos da equipa. Ao nível de *software*, recorreu-se a ferramentas de apoio técnico e documental como o Visual Studio Code, Overleaf (LaTeX), Visual Paradigm e Git/GitHub, essenciais para a programação, modelação, escrita do relatório e controlo de versões. Por fim, foi ainda utilizado o apoio de LLMs, enquanto recurso complementar ao processo de engenharia de requisitos, produção de documentação e estruturação do trabalho.

Interação com o LLM

Prompt utilizado
Escreve uma secção de recursos para o projeto Belo Mercado Manager. Explica a composição da equipa técnica e as responsabilidades no ciclo completo, desde análise até implementação. Refere as ferramentas de trabalho (VS Code, Overleaf para LaTeX, Visual Paradigm e Git/GitHub) e inclui o uso de LLMs como apoio à estruturação documental e à engenharia de requisitos.

Análise crítica
A resposta do LLM apresentou corretamente a separação entre recursos humanos e materiais e incluiu corretamente as ferramentas que efetivamente seriam usadas no projeto. Ainda assim, alguns trechos estavam demasiado descritivos e incluíam pressupostos que exigiam refinamento para manter o rigor documental.
Na revisão final, foram mantidos apenas os elementos consistentes com o contexto real do nosso grupo, clarificando assim as responsabilidades e utilidade prática dos recursos. O resultado final ficou então mais conciso, verificável e adequado.

2.6.1 Equipa de Trabalho e *Stakeholders*

Nome	**Número**	**Responsabilidades Principais**
Francisco Soares	106882	Coordenação, Eng. Requisitos, Arquitetura
Carlos Silva	106792	Base de Dados, *Backend*
Nuno Soares	107366	*Frontend* (Blazor), UI/UX
Diogo Silva	97941	API, Integração, Testes
Rodrigo Guimarães	93236	Documentação, Relatórios, QA

Tabela 2.5: Equipa de trabalho e responsabilidades.

Os *stakeholders* do sistema foram identificados e classificados em primários (utilizadores diretos do sistema) e secundários (partes afetadas indiretamente).

Stakeholder	**Descrição**	**Interesse no Sistema**
Administrador da Cadeia	Responsável pela gestão global da cadeia.	Relatórios consolidados, *dashboards*, gestão de fornecedores.
Gerente de Loja	Responsável pela operação diária de uma loja.	Controlo de inventário, relatórios locais, gestão de reposição.
Operador de Caixa	Funcionário que realiza o registo de vendas.	Interface POS simples, registo de vendas, emissão de faturas.
Fornecedor	Responsável por repor o *stock* de produtos.	Receção de encomendas, comunicação de preços.
Cliente	Pessoa que adquire produtos nas lojas.	Experiência de compra eficiente, emissão de fatura.
Contabilista	Profissional de contabilidade e fiscalidade.	Dados de faturação, relatórios financeiros.
Administrador IT	Responsável técnico pela manutenção do sistema.	Gestão de utilizadores, *backups*, monitorização.

Tabela 2.6: *Stakeholders* do sistema (resumo).

2.7 Plano de Execução

O plano de execução foi definido por etapas sequenciais com marcos de validação, mantendo a documentação como atividade contínua desde o início do projeto. A síntese temporal encontra-se na Tabela 2.7, e o diagrama de Gantt detalhado encontra-se no Anexo C.

Etapa	**Período**	**Atividades Principais**
E1	09/02 – 02/03	Conceção, levantamento inicial, elicitação e estruturação documental.
E2	02/03 – 23/03	Arquitetura e *design* da solução.
E3	23/03 – 13/04	Implementação incremental dos módulos prioritários.
E4	13/04 – 11/05	Verificação, validação e consolidação técnica/documental.
Encerramento	26/05 – 27/05	Revisão final, empacotamento e entrega.

Tabela 2.7: Timeline de execução do projeto por etapas.

3 Conceção e Engenharia de Requisitos Assistida por LLM

3.1 Métodos de Levantamento de Requisitos

O levantamento de requisitos recorreu a uma combinação de técnicas complementares, com o objetivo de recolher informação suficientemente abrangente e consistente para especificar o sistema:

1. **Simulação de entrevistas com *stakeholders*** — Utilização de LLM para simular entrevistas com cada tipo de *stakeholder*, cobrindo os seus processos atuais, dificuldades e expectativas (ver Secção 3.3 e Anexo A).
2. **Análise de domínio** — Estudo do caso de estudo CE01 (Mercearia) como referência metodológica e de escala para o contexto de negócio.
3. ***Brainstorming* assistido por LLM** — Geração e refinamento iterativo de requisitos a partir dos cenários identificados.
4. **Análise de sistemas semelhantes** — Estudo de sistemas POS e ERP do setor de retalho de proximidade.
5. **Refinamento iterativo** — Revisão e eliminação de ambiguidades com apoio de LLM (ver Secção 3.6).

3.2 Elicitação de Requisitos

A elicitação de requisitos teve como objetivo recolher informação junto dos *stakeholders* e a partir do contexto do domínio, de forma a identificar necessidades, problemas atuais e expectativas relativamente ao sistema.

As principais fontes de informação consideradas foram:

- *stakeholders* com participação direta no processo (utilizadores operacionais e gestão);
- documentação e regras existentes relevantes para o domínio (procedimentos, formulá-

rios, registos e políticas internas, quando aplicável);

- descrição do processo atual e exemplos reais de situações típicas e excecionais.

Para recolha de informação foram usadas entrevistas semi-estruturadas e discussão orientada por exemplos. As entrevistas incluíram perguntas fechadas para confirmar passos do processo e dados necessários (ex.: que informação é registada, quando e por quem) e perguntas abertas para identificar problemas frequentes, necessidades não satisfeitas e melhorias desejadas. Sempre que útil, foram utilizados cenários de alto nível para apoiar os *stakeholders* a explicitar detalhes que normalmente ficam implícitos, incluindo exceções e variações do processo.

Os resultados da elicitação foram registados como uma lista de necessidades, objetivos e restrições, mantendo a referência à origem (*stakeholder*/documento/cenário). Esta lista constituiu o ponto de partida para a transformação em *user stories*, clarificação de ambiguidades e posterior derivação de requisitos funcionais e não funcionais.

Interação com o LLM

Prompt utilizado
Com base no contexto do domínio e nos *stakeholders* identificados, apoia a elicitação de requisitos de um sistema, com o objetivo de recolher informação relevante para identificar necessidades, problemas atuais, restrições e expectativas relativamente ao sistema a desenvolver. Estrutura a recolha de informação através de entrevistas semi-estruturadas e discussão orientada por exemplos, incluindo perguntas fechadas e abertas.

Análise crítica
O LLM contribuiu com uma estrutura metodológica adequada para a elicitação, incluindo fontes de informação e técnicas de recolha. Ainda assim, a resposta inicial era genérica em rastreabilidade e precisava de maior ligação às decisões documentais do projeto.
A versão final foi refinada para explicitar a combinação de perguntas abertas/fechadas, cenários de apoio e registo da origem de cada necessidade. A revisão humana garantiu coerência com as entrevistas sintetizadas no corpo e com o detalhe disponibilizado em anexo.

3.3 Simulação de Entrevistas com *Stakeholders*

A simulação de entrevistas com *stakeholders* constituiu uma das técnicas de elicitação mais relevantes deste projeto, tirando partido direto das capacidades dos LLMs para gerar diálogos realistas com perfis ficcionais mas representativos dos utilizadores reais do sistema. Foram realizadas **5 entrevistas simuladas** com recurso a um LLM. As transcrições completas encontram-se no Anexo A.

3.3.1 Entrevista 1 - Administrador da Cadeia (Sr. Manuel Correia, 55 anos)

Revelou necessidades de visibilidade centralizada, negociação com fornecedores ao nível da cadeia, alertas de *stock* e relatórios comparativos entre lojas. Destacou a importância de perfis de acesso diferenciados e a necessidade de um *dashboard* diário com o desempenho agregado.

3.3.2 Entrevista 2 - Gerente de Loja (D. Teresa Vieira, 38 anos)

Evidenciou problemas com registos manuais em *Excel*, falta de alertas automáticos de *stock* e necessidade de encomendas integradas. Valoriza a consolidação automática ao fecho de caixa e uma interface simples para operadores com pouca formação técnica.

3.3.3 Entrevista 3 - Operador de Caixa (João Martins, 22 anos)

Destacou a importância da rapidez no registo de vendas, a necessidade de pesquisa alternativa de produtos quando o leitor falha, emissão automática de fatura com NIF e aplicação automática de promoções.

3.3.4 Entrevista 4 - Fornecedor (Distribuição Almeida, Lda.)

Identificou ineficiências no processo de encomendas descentralizado e a necessidade de padronização e centralização das encomendas. Sugeriu a confirmação de encomendas no sistema com registo de datas de entrega e resolução de discrepâncias.

3.3.5 Entrevista 5 - Contabilista (Dr. Carlos Ferreira, 48 anos)

Sublinhou o processo penoso de compilação de dados financeiros mensais e a necessidade de exportação automática de dados de faturação por loja, com discriminação de IVA por taxa. Confirmou os requisitos legais de retenção de dados por 10 anos e a necessidade de certificação AT para o *software* de faturação.

Interação com o LLM

Prompt utilizado

Simula uma entrevista de elicitação de requisitos com os *stakeholders* definidos. Assume o perfil que te for fornecido, incluindo nome, idade, formação, experiência, função, responsabilidades e contexto profissional, e responde sempre de forma coerente com esse perfil. As respostas devem ser suficientemente concretas e detalhadas para permitir a identificação de requisitos funcionais e não funcionais. Mantém sempre o papel do *stakeholder* durante toda a entrevista e responde em formato de diálogo com o analista.

Análise crítica

O LLM conseguiu manter a coerência de personagem e produzir entrevistas com conteúdo útil para extração de necessidades. Contudo, a versão inicial apresentava alguns trechos redundantes e diferenças de granularidade entre perfis.

Na revisão, os resultados foram sintetizados por stakeholder no corpo do relatório e mantidas as transcrições completas em anexo, em linha com a estratégia de concisão documental. Esta abordagem melhorou legibilidade sem perder evidência de origem.

3.4 *User Stories*

As *user stories* seguem o formato *"Como [ator], quero [ação], para que [benefício]."* e estão associadas a requisitos funcionais para garantir a rastreabilidade. O resultado foi **29 *user stories*** distribuídas por 4 atores: 10 para o Administrador da Cadeia (US-A01 a US-A10), 8 para o Gerente de Loja (US-G01 a US-G08), 8 para o Operador de Caixa (US-O01 a US-O08) e 3 para o Administrador IT (US-IT01 a US-IT03).

ID	*User Story*	RF	Pr.
US-A01	Como administrador, quero visualizar um *dashboard* com as vendas de todas as lojas, para acompanhar o desempenho global.	RF13	Alta
US-A02	Como administrador, quero comparar as vendas entre lojas num dado período, para identificar lojas com desempenho inferior.	RF13	Média
US-A03	Como administrador, quero receber alertas de ruturas de *stock* em qualquer loja, para coordenar a reposição.	RF06	Alta
US-A04	Como administrador, quero gerar relatórios de vendas por categoria de produto, para tomar decisões sobre o sortido.	RF14	Alta
US-A05	Como administrador, quero gerir o catálogo centralizado de produtos, para manter informação consistente.	RF05	Alta
US-A06	Como administrador, quero gerir os fornecedores da cadeia, para negociar condições centralizadas.	RF09	Alta
US-A07	Como administrador, quero exportar relatórios em PDF e CSV, para partilhar com o contabilista.	RF14	Média
US-A08	Como administrador, quero consultar o *log* de ações críticas, para auditar operações suspeitas.	RF18	Média
US-A09	Como administrador, quero gerir os utilizadores e os seus perfis de acesso, para controlar quem acede a quê.	RF16	Alta
US-A10	Como administrador, quero ver o estado das consolidações diárias, para garantir dados atualizados.	RF12	Média

Tabela 3.1: *User stories* — Administrador da Cadeia.

ID	*User Story*	RF	Pr.
US-G01	Como gerente, quero ver o *stock* atual de todos os produtos da minha loja, para planear a reposição.	RF06	Alta
US-G02	Como gerente, quero receber alertas quando um produto atinge o *stock* mínimo, para encomendar a tempo.	RF06	Alta
US-G03	Como gerente, quero criar encomendas a fornecedores diretamente no sistema, para não usar email ou telefone.	RF10	Alta
US-G04	Como gerente, quero registar a receção de mercadoria e atualizar o *stock* automaticamente.	RF11	Alta
US-G05	Como gerente, quero ver um resumo diário de vendas da minha loja, para avaliar o desempenho do dia.	RF14	Alta
US-G06	Como gerente, quero realizar contagens de inventário e registar discrepâncias, para identificar perdas.	RF07	Média
US-G07	Como gerente, quero acompanhar o estado das encomendas a fornecedores, para saber quando esperar entregas.	RF10	Média
US-G08	Como gerente, quero receber sugestões automáticas de encomenda, para reposição mais eficiente.	RF08	Média

Tabela 3.2: *User stories* — Gerente de Loja.

ID	*User Story*	RF	Pr.
US-O01	Como operador, quero registar uma venda passando produtos pelo leitor de código de barras.	RF01	Alta
US-O02	Como operador, quero pesquisar um produto pelo nome ou código quando o leitor não funcionar.	RF01	Alta
US-O03	Como operador, quero que o sistema calcule o total e o IVA automaticamente.	RF01	Alta
US-O04	Como operador, quero emitir fatura com o NIF do cliente quando solicitado.	RF01	Alta
US-O05	Como operador, quero que as promoções sejam aplicadas automaticamente.	RF03	Média
US-O06	Como operador, quero realizar a abertura e o fecho de caixa no sistema.	RF04	Alta
US-O07	Como operador, quero anular uma venda quando necessário, para corrigir erros.	RF02	Média
US-O08	Como operador, quero processar devoluções de clientes.	RF02	Média

Tabela 3.3: *User stories* — Operador de Caixa.

ID	*User Story*	RF	Pr.
US-IT01	Como administrador IT, quero gerir os utilizadores e as suas permissões, para manter o sistema seguro.	RF16	Alta
US-IT02	Como administrador IT, quero monitorizar o estado das consolidações diárias, para intervir em caso de falha.	RF12	Média
US-IT03	Como administrador IT, quero consultar os *logs* de auditoria, para investigar incidentes de segurança.	RF18	Média

Tabela 3.4: *User stories* — Administrador do Sistema (IT).

Interação com o LLM

Prompt utilizado

Atua como especialista em Engenharia de Requisitos e, com base nos perfis dos *stakeholders* identificados, gera *user stories* para cada um. Em cada *story*, os atores devem expressar, de forma simples, as funcionalidades e expectativas que têm perante o *software*, indicando também o valor que essa funcionalidade lhes traz no seu dia-a-dia.

Análise crítica

A geração inicial do LLM foi útil para acelerar a produção de stories e cobrir os vários perfis de utilizador. No entanto, surgiram sobreposições e formulações demasiado próximas, o que dificultava priorização e validação.

Foi realizado refinamento manual com consolidação de duplicados, normalização linguística e mapeamento para requisitos funcionais. O resultado final ficou mais consistente, testável e orientado para a rastreabilidade bidirecional.

3.5 Requisitos Funcionais e Não Funcionais

A derivação de requisitos funcionais e não funcionais seguiu a distinção proposta por Sommerville [1] (cap. 4) e a estrutura recomendada pela norma **IEEE 830/29148** [2, 3]: os **requisitos funcionais** descrevem as funções que o sistema deve executar, enquanto os **requisitos não funcionais** definem restrições sobre a forma como essas funções devem ser prestadas (desempenho, segurança, usabilidade, entre outros).

Os 19 requisitos funcionais foram consolidados a partir de uma especificação inicial mais granular (52 RF), organizados em 7 módulos funcionais. Os requisitos não funcionais foram derivados de categorias normalizadas, com métricas quantificáveis sempre que possível.

ID	Requisito	Prioridade
Módulo 1 — Gestão de Vendas		
RF01	**Registar Venda (POS)** — Registo de vendas com pesquisa de produtos, cálculo de IVA, múltiplos métodos de pagamento e emissão de fatura simplificada (modo protótipo académico — ver ADR-009).	Alta
RF02	**Anular Venda e Processar Devolução** — Anulação de vendas com motivo, devoluções com atualização de *stock* e notas de crédito.	Média
RF03	**Aplicar Promoções e Descontos** — Definição e aplicação automática de promoções no POS.	Média
RF04	**Abrir e Fechar Caixa** — Registo de abertura/fecho, contagem de numerário, preparação para consolidação.	Alta
Módulo 2 — Gestão de Produtos e Catálogo		
RF05	**Gerir Catálogo de Produtos** — Catálogo centralizado com EAN-13, categorias, preços, IVA e histórico.	Alta
Módulo 3 — Gestão de Inventário/*Stocks*		
RF06	**Consultar e Monitorizar *Stock*** — *Stock* por produto por loja, alertas de *stock* mínimo.	Alta
RF07	**Realizar Inventário Manual** — Contagens físicas, discrepâncias, alertas de validade.	Média
RF08	**Gerar Sugestões de Reposição** — Sugestões automáticas com base em vendas e *stock* mínimo.	Média
Módulo 4 — Gestão de Fornecedores e Compras		
RF09	**Gerir Fornecedores** — CRUD de fornecedores, associação a produtos, histórico.	Alta
RF10	**Criar Encomenda a Fornecedor** — Encomendas com rascunhos e acompanhamento de estados.	Alta
RF11	**Registar Receção de Mercadoria** — Receção, discrepâncias, atualização de *stock*.	Alta
Módulo 5 — Consolidação e Relatórios		
RF12	**Consolidar Dados Diários** — Agregação automática diária dos dados operacionais (vendas, movimentos de *stock*, KPIs) em tabelas de leitura denormalizadas, com histórico e retentativa em falhas.	Alta
RF13	**Consultar *Dashboard* de KPIs** — KPIs e comparação entre lojas.	Alta
RF14	**Gerar e Exportar Relatórios** — Relatórios filtráveis, exportação em PDF/CSV.	Média
Módulo 6 — Gestão de Utilizadores e Acessos		
RF15	**Autenticar no Sistema** — *Login*, bcrypt, bloqueio após 5 tentativas, sessão baseada em JWT (*access token* de 15 min e *refresh token* de 7 dias com rotação — ver ADR-004), perfis diferenciados.	Alta
RF16	**Gerir Utilizadores e Perfis** — CRUD de utilizadores, perfis, associação a lojas.	Alta
RF17	**Editar Perfil Próprio** — Edição de dados pessoais e alteração de *password* (mínimo 8 caracteres, maiúscula, minúscula, dígito e símbolo).	Média
RF18	**Consultar Logs de Auditoria** — *Logs* de ações críticas, imutáveis, consulta restrita a administrador.	Média
Módulo 7 — Gestão de Lojas		
RF19	**Gerir Lojas da Cadeia** — CRUD de lojas, associação de funcionários, configurações.	Alta

Tabela 3.5: Requisitos funcionais do sistema (19 RF organizados por módulo).

ID	Categoria	Requisito	Pr.
RNF-01	Desempenho	Registo de venda em ≤ 2 s.	Alta
RNF-02	Desempenho	Consolidação diária em < 5 min.	Alta
RNF-03	Desempenho	Geração de relatórios em < 10 s (até 1 ano de dados).	Média
RNF-04	Disponibilidade	Disponibilidade do sistema $\geq 99{,}5\%$ durante o horário de funcionamento das lojas (07h00–23h00), medida mensalmente.	Alta
RNF-05	Disponibilidade	Disponibilidade total do sistema $\geq 99\%$ em janela 24/7 (incluindo horário não comercial e janelas de manutenção planeadas), medida mensalmente.	Alta
RNF-06	Fiabilidade	Integridade transacional (ACID) em todas as operações críticas.	Alta
RNF-07	Fiabilidade	Retentativa automática de consolidação (3 tentativas a 30, 60 e 90 min).	Média
RNF-08	Segurança	*Passwords* armazenadas com bcrypt e *salt*.	Alta
RNF-09	Segurança	Comunicações encriptadas (HTTPS/TLS).	Alta
RNF-10	Segurança	Conformidade com o RGPD no tratamento e retenção de dados pessoais.	Alta
RNF-11	Usabilidade	Interface POS operável por utilizador sem formação técnica em menos de 1 h.	Alta
RNF-12	Usabilidade	Interface responsiva para *desktop* e *tablet*.	Média
RNF-13	Escalabilidade	Suporte a até 20 lojas sem degradação de desempenho.	Média
RNF-14	Manutenibilidade	Código conforme padrões .NET com comentários XML e cobertura de testes $\geq 70\%$.	Média
RNF-15	Portabilidade	Compatível com Chrome, Firefox e Edge (versões recentes).	Alta
RNF-16	*Backup*	*Backups* automáticos diários com retenção mínima de 10 anos.	Alta

Tabela 3.6: Requisitos não-funcionais do sistema.

Interação com o LLM

Prompt utilizado
Atua como Especialista em Engenharia de Requisitos. Extrai requisitos funcionais e não funcionais a partir dos resultados das entrevistas e das *user stories* geradas. Organiza os requisitos funcionais por módulo funcional e classifica cada requisito por prioridade. Para os requisitos não funcionais, usa categorias normalizadas (desempenho, disponibilidade, fiabilidade, segurança, usabilidade, escalabilidade, manutenibilidade, portabilidade e backup) e gera métricas quantificáveis sempre que possível.

Análise crítica
O LLM foi útil para acelerar a derivação inicial dos requisitos funcionais e não funcionais e para propor uma organização modular coerente com o domínio. Ainda assim, a versão preliminar incluía alguns requisitos sobrepostos e outros formulados de forma demasiado genérica para validação objetiva.
Na revisão, os requisitos foram consolidados, desambiguados e convertidos em enunciados mais mensuráveis, com prioridade e rastreabilidade explícitas. O resultado final ficou mais consistente com os cenários de uso e mais adequado para servir de base à implementação e aos testes.

3.6 Análise de Requisitos — Ambiguidades e Refinamentos

O refinamento de requisitos ambíguos é uma atividade crítica da engenharia de requisitos, tendo por objetivo eliminar imprecisões, contradições e omissões que possam comprometer as fases subsequentes de desenvolvimento. Sommerville [1] (cap. 4, secção 4.6) enfatiza que os requisitos devem ser verificados quanto a completude, consistência e ausência de ambiguidades. Um requisito é considerado ambíguo quando não é mensurável, verificável ou quando admite múltiplas interpretações.

Foram identificados os seguintes requisitos ambíguos e aplicados os respetivos refinamentos:

Ambiguidade 1: "Consolidação ao fim do dia"

- *Requisito original*: "A informação deverá ser consolidada ao fim do dia."
- *Problema*: O que define "fim do dia"? Depende do utilizador, da loja ou de um horário fixo?
- *Refinamento*: A consolidação é acionada pelo fecho de caixa (UC04) ou por processo agendado às 23h30, com retentativa automática a 30, 60 e 90 minutos em caso de falha (RNF-07).

Ambiguidade 2: "Operação autónoma"

- *Requisito original*: "Cada loja atua de forma autónoma."
- *Problema*: O sistema deve funcionar sem ligação à Internet? Por quanto tempo?
- *Refinamento (Etapa 1)*: Cada loja operaria com instância local independente (RT05), com sincronização posterior ao sistema central.
- *Refinamento revisto (Etapa 2)*: Após análise de *trade-offs* (ADR-001), a equipa adotou uma topologia de **instância única centralizada**. "Autonomia" passa a referir-se à autonomia **operacional** de cada loja (gestão local de *stock*, encomendas, abertura/fecho de caixa) e não à autonomia técnica de *deployment*. A operação POS depende da disponibilidade da ligação à Internet, mitigada pelos requisitos de disponibilidade RNF-04 e RNF-05.

Ambiguidade 3: "Relatórios de gestão"

- *Requisito original*: "Geração de relatórios por loja, período e categoria."
- *Problema*: Quais os tipos específicos de relatório? Em que formatos?
- *Refinamento*: 4 tipos — vendas, *stock*, compras/fornecedores e *dashboard* de KPIs. Exportação em PDF e CSV (RF14, RNF-03).

Ambiguidade 4: "O sistema deve ser seguro"

- *Requisito original*: "O sistema deve ser seguro."
- *Problema*: "Seguro" não é mensurável nem verificável.
- *Refinamento*: O sistema só pode ser acedido por utilizadores autenticados (RF15); a *password* deve ter no mínimo 8 caracteres, incluindo maiúscula, minúscula, dígito e símbolo (RF17); após 5 tentativas falhadas, a conta é bloqueada durante 15 minutos (RF15/RNF-08).

Interação com o LLM

Prompt utilizado
Faz uma revisão aos requisitos definidos até ao momento, identificando ambiguidades na sua formulação. Para cada ambiguidade encontrada, formula questões de clarificação e apresenta possíveis propostas de refinamento que permitam eliminá-la. No final, indica que cada refinamento proposto deverá ser revisto e aprovado pela equipa de trabalho.

Análise crítica
O LLM ajudou a identificar ambiguidades relevantes e a sugerir perguntas de clarificação com rapidez. Ainda assim, parte dos refinamentos propostos estava formulada de modo abstrato e exigiu concretização em critérios operacionais verificáveis.
A equipa converteu enunciados vagos em formulações mensuráveis e alinhadas ao contexto do sistema (ex.: consolidação temporal, operação autónoma e tipologia de relatórios). O resultado final reforça a qualidade dos requisitos e reduz o risco de interpretação divergente.

3.7 Casos de Uso

A modelação de casos de uso teve como objetivo descrever, de forma detalhada e estruturada, as interações entre os atores e o sistema para cada funcionalidade identificada nos requisitos funcionais. Os casos de uso seguem o **formato expandido** recomendado por Sommerville [1] (cap. 4, secção 4.5) e Cockburn [4], incluindo fluxo principal, fluxos alternativos, fluxos de exceção, pré e pós-condições, regras de negócio e validações de dados.

Foram modelados **19 casos de uso** (UC01–UC19), com correspondência 1:1 aos 19 requisitos funcionais, conforme a Tabela 3.7.

UC	Nome	RF
UC01	Registar Venda (POS)	RF01
UC02	Anular Venda e Processar Devolução	RF02
UC03	Aplicar Promoções e Descontos	RF03
UC04	Abrir e Fechar Caixa	RF04
UC05	Gerir Catálogo de Produtos	RF05
UC06	Consultar e Monitorizar *Stock*	RF06
UC07	Realizar Inventário Manual	RF07
UC08	Gerar Sugestões de Reposição	RF08
UC09	Gerir Fornecedores	RF09
UC10	Criar Encomenda a Fornecedor	RF10
UC11	Registar Receção de Mercadoria	RF11
UC12	Consolidar Dados Diários	RF12
UC13	Consultar *Dashboard* de KPIs	RF13
UC14	Gerar e Exportar Relatórios	RF14
UC15	Autenticar no Sistema	RF15
UC16	Gerir Utilizadores e Perfis	RF16
UC17	Editar Perfil Próprio	RF17
UC18	Consultar Logs de Auditoria	RF18
UC19	Gerir Lojas da Cadeia	RF19

Tabela 3.7: Correspondência entre casos de uso e requisitos funcionais.

Os diagramas UML de casos de uso foram criados em Visual Paradigm. O diagrama UML geral encontra-se no Anexo D; os casos de uso detalhados permanecem organizados no Anexo B.

São apresentados em detalhe os casos de uso UC01 e UC15, representativos dos módulos de maior impacto operacional (POS e autenticação). Os restantes casos de uso (UC02–UC19) encontram-se detalhados no Anexo B.

Interação com o LLM

Prompt utilizado

Para cada requisito funcional (RF01–RF19), gera o caso de uso correspondente (UC01–UC19) em formato expandido, com nível de detalhe suficiente para servir de base à implementação. Inclui fluxo principal, fluxos alternativos e de exceção, pré e pós-condições, regras de negócio e referências aos requisitos. Gera também código PlantUML para os diagramas de casos de uso organizados por módulo funcional.

Análise crítica

O LLM facilitou a criação inicial dos casos de uso em formato expandido e ajudou a manter uma estrutura uniforme entre UCs. No entanto, a primeira versão incluía alguns fluxos excessivamente verbosos e certas regras de negócio ainda pouco alinhadas com as prioridades reais do sistema.

Na revisão, os casos de uso foram ajustados para reforçar a clareza, a consistência e o mapeamento direto com os requisitos funcionais. Também foi feito controlo de qualidade sobre pré/pós-condições, exceções e rastreabilidade, garantindo uma base mais sólida para a implementação e a validação.

3.7.1 Descrição Detalhada dos Casos de Uso

3.7.1.1 UC01 — Registar Venda (POS)

ID / Nome	UC01 — Registar Venda (POS)
Ator Principal	Operador de Caixa
Descrição	Registar a venda de um ou mais produtos com cálculo automático de totais, IVA e emissão de fatura/recibo.
Pré-condições	1) Operador autenticado. 2) Caixa do dia aberta (UC04). 3) Produtos ativos no catálogo.
Pós-condições	1) Venda registada com estado "Concluída". 2) *Stock* decrementado. 3) Fatura/recibo emitido (modo protótipo académico — ver ADR-009). 4) Alertas de *stock* mínimo gerados, se aplicável.
Fluxo Principal	1. Clicar "Nova Venda". 2. Passar produto no leitor (EAN-13). 3. Sistema valida código e *stock*. 4. Produto adicionado com preço, IVA e qtd=1. 5. Totais recalculados. 6. Repetir para mais produtos. 7. "Finalizar Venda" → ecrã de pagamento. 8. Selecionar método (Numerário/Cartão/MB). 9. Se numerário: introduzir valor, sistema calcula troco. 10. Confirmar → venda persistida, *stock* atualizado, fatura emitida.
Fluxos Alt.	**3a.** Leitor falha → pesquisa textual (nome/código). **5a.** Fatura com NIF → validação mod 11. **11a.** Remover produto da lista. **14a.** Cliente desiste → venda cancelada (sem afetar *stock*).
Exceções	E1: Produto não encontrado. E2: *Stock* insuficiente. E3: NIF inválido. E4: Valor insuficiente (numerário). E5: Falha BD (*rollback* ACID).
Regras de Negócio	RN01: Mínimo 1 produto. RN02: IVA por linha (6/13/23%). RN03: Nº fatura sequencial por loja/ano. RN04: Faturação em modo **protótipo académico** (ver ADR-009); a conformidade fiscal certificada da AT (Portaria 363/2010, ATCUD, *hash chain*, SAF-T) está fora do âmbito do projeto.
RF Associados	RF01, RF02, RF03, RF06, RNF-01, RNF-06

Tabela 3.8: Caso de uso UC01 — Registar Venda (POS).

3.7.1.2 UC15 — Autenticar no Sistema

ID / Nome	UC15 — Autenticar no Sistema
Ator Principal	Utilizador (qualquer perfil)
Descrição	*Login* com email/*password*, validação bcrypt, sessão JWT, redirecionamento por perfil.
Pré-condições	1) Sistema acessível via navegador. 2) Conta ativa com credenciais. 3) Sem sessão ativa.
Pós-condições	1) Sessão autenticada (JWT). 2) *Dashboard* do perfil carregado. 3) *Log* de auditoria (userId, IP, resultado).
Fluxo Principal	1. Página de *login*. 2. Preencher email e *password*. 3. "Entrar". 4. Validar formato. 5. Consultar BD (`Utilizadores`). 6. Comparar *hash* (bcrypt). 7. Verificar estado "Ativo". 8. Verificar bloqueio (< 5 tentativas). 9. Sucesso $\rightarrow$ sessão JWT $\rightarrow$ *dashboard* por perfil.
Fluxos Alt.	**5a.** Formato inválido $\rightarrow$ mensagem *inline*. **6a/7a.** Credenciais inválidas (mensagem genérica). **8a.** Conta inativa. **9a.** Conta bloqueada (15 min).
Exceções	E1: Falha BD $\rightarrow$ "Serviço indisponível". E2: Navegador fechado $\rightarrow$ sem sessão.
Regras de Negócio	RN01: 5 tentativas $\rightarrow$ bloqueio 15 min. RN02: Sem enumeração de utilizadores. RN03: *Access token* JWT expira após 15 min; *refresh token* válido durante 7 dias com rotação a cada utilização (ADR-004). RN04: bcrypt com *salt*.
RF Associados	RF15, RF18, RNF-08, RNF-09, RNF-10

Tabela 3.9: Caso de uso UC15 — Autenticar no Sistema.

Os restantes casos de uso (UC02–UC14 e UC16–UC19) encontram-se detalhados no Anexo B.

3.8 Modelo de Domínio

Tendo por base os requisitos previamente recolhidos e o contexto de negócio caracterizado nos capítulos anteriores, foi desenvolvido o modelo de domínio que representa, de forma estática e independente da solução a desenvolver, as principais entidades e relações que caracterizam a cadeia Belo Mercado.

O modelo organiza-se em torno da Loja, que agrega os funcionários (especializados em Operador de Caixa e Gerente de Loja), o inventário e a atividade operacional. O catálogo é partilhado por toda a cadeia através das entidades Produto e Categoria, sendo a presença de cada produto numa loja representada por Stock. Cada produto tem associada uma Validade e pode ser alvo de Promoção aplicáveis num dado período.

A operação de venda concretiza-se em Caixa (sessão de trabalho diária), Venda e respetivas

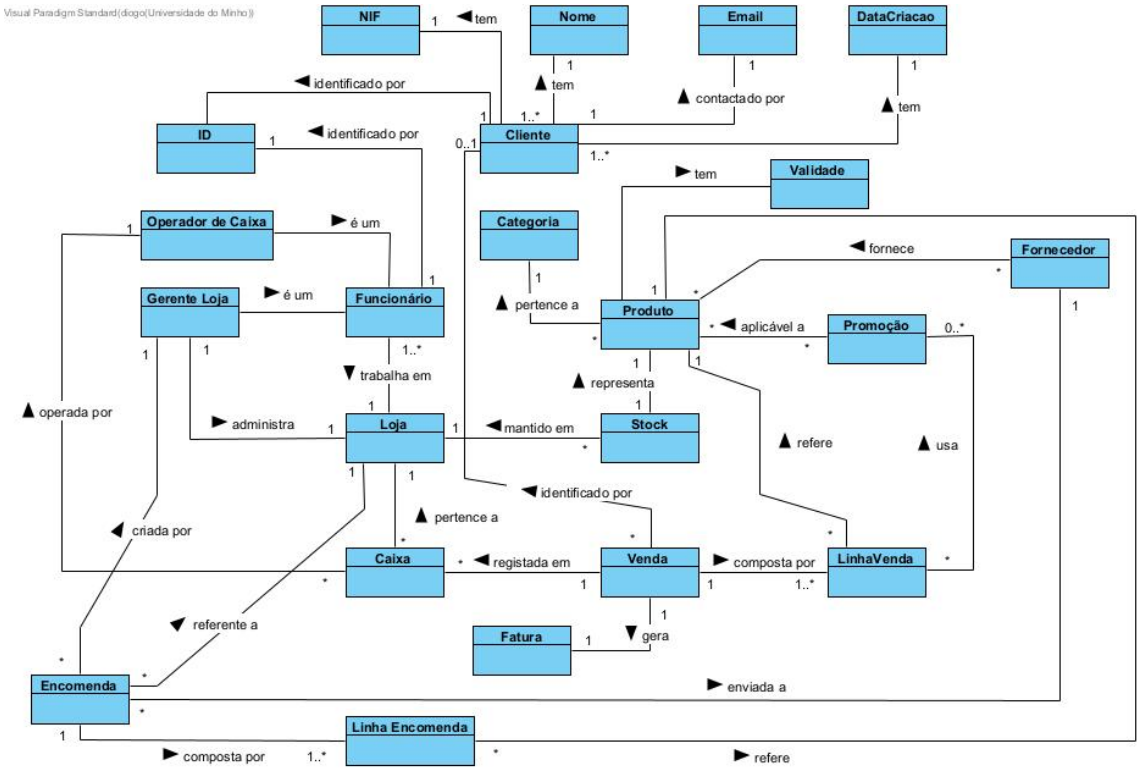


Figura 3.1: Modelo de Domínio

LinhaVenda, culminando na emissão de uma Fatura. Quando o consumidor solicita identificação fiscal, a Venda fica opcionalmente associada a um Cliente, entidade que armazena os dados mínimos necessários para emissão de fatura com NIF (cardinalidade 0..1). Note-se que o Cliente, embora identificado como *stakeholder*, não é ator do sistema — não interage diretamente com a aplicação, sendo apenas registado como entidade de dados quando solicita fatura nominativa. O abastecimento é suportado por Encomenda feita por uma loja a um Fornecedor, composta por linhas que referem os produtos do catálogo.

Em suma, o modelo captura os três eixos do negócio(organização, venda e abastecimento) fornecendo uma base conceptual coerente para as etapas seguintes.

3.9 Diagrama de casos de uso

Os atores identificados no diagrama Operador de Caixa, Gerente de Loja, Administrador da Cadeia e Administrador IT especializam o ator genérico Utilizador e interagem com casos de uso que, em essência, se encontram distribuídos por cinco categorias funcionais: operações de venda, gestão de inventário e loja, administração global e configurações, segurança e administração técnica, e tarefas automatizadas.

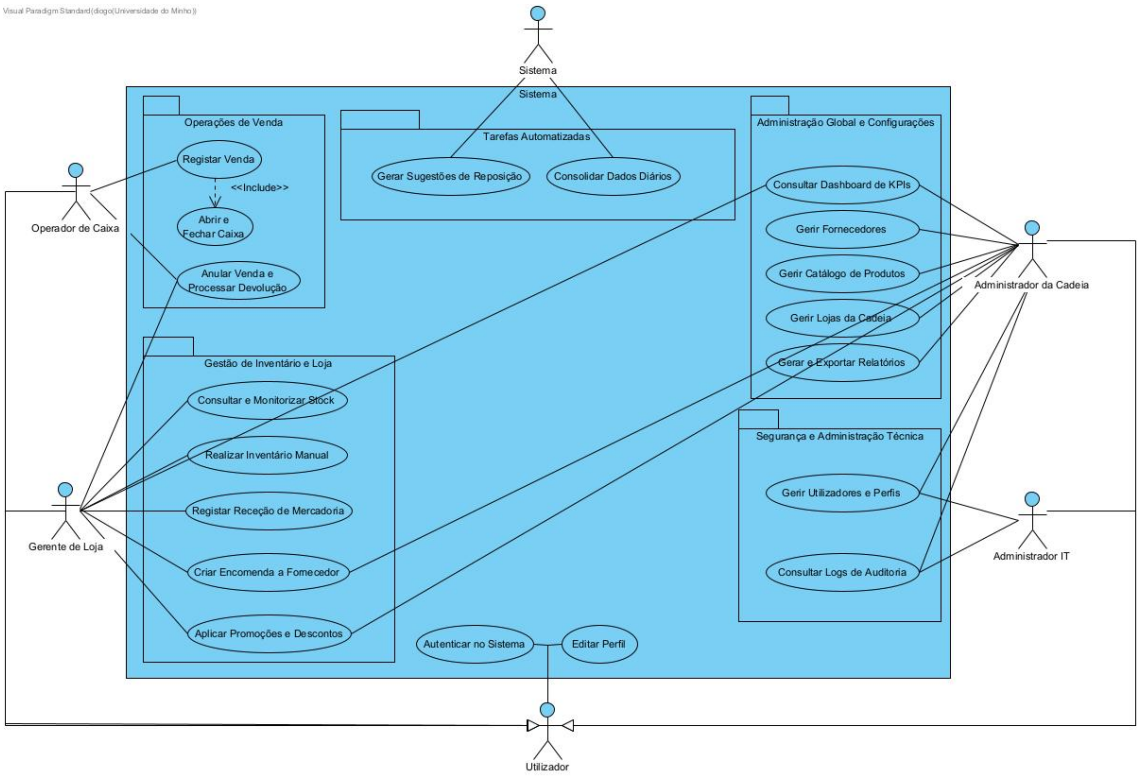


Figura 3.2: Diagrama de Use Cases

Nas operações de venda, da responsabilidade do Operador de Caixa, encontram-se os casos de uso Registar Venda, que constitui a atividade central do ponto de venda e inclui obrigatoriamente a operação Abrir e Fechar Caixa, e ainda Anular Venda e Processar Devolução, destinado à correção de vendas previamente registadas e ao processamento de devoluções.

Na gestão de inventário e loja, da responsabilidade do Gerente de Loja, estão incluídos os casos de uso Consultar e Monitorizar Stock, Realizar Inventário Manual, Registar Receção de Mercadoria, Criar Encomenda a Fornecedor e Aplicar Promoções e Descontos, cobrindo todo o ciclo operacional de uma loja, desde o controlo do inventário até à reposição junto dos fornecedores. Na administração global e configurações, da responsabilidade do Administrador da Cadeia, foram identificados Consultar Dashboard de KPIs, Gerir Fornecedores, Gerir Catálogo de Produtos, Gerir Lojas da Cadeia e Gerar e Exportar Relatórios, que suportam a visão consolidada do negócio e a configuração transversal da cadeia.

Na categoria de segurança e administração técnica, partilhada pelo Administrador da Cadeia e pelo Administrador IT, encontram-se os casos de uso Gerir Utilizadores e Perfis e Consultar Logs de Auditoria, destinados ao controlo de acessos e à rastreabilidade das ações realizadas no sistema.

Por fim, as tarefas automatizadas, executadas pelo próprio sistema sem intervenção direta de um ator humano, compreendem Gerar Sugestões de Reposição e Consolidar Dados Diários,

correspondendo a processos agendados que asseguram, respetivamente, o reabastecimento proativo das lojas e a agregação diária de informação operacional no sistema central. Transversalmente a todas as categorias, qualquer utilizador do sistema necessita de Autenticar no Sistema antes de aceder às funcionalidades, podendo também Editar Perfil para manter os seus dados atualizados.

4 Arquitetura e Design do Software utilizando LLM

4.1 Visão Geral da Arquitetura

A arquitetura do sistema **Belo Mercado Manager** foi concebida para responder a um conjunto de necessidades operacionais e de gestão claramente identificadas nas etapas anteriores: assegurar uma operação diária fluida em cada loja (registo de vendas, gestão de *stock*, encomendas a fornecedores), garantir uma visão consolidada e centralizada do desempenho da cadeia para o Administrador, e cumprir os requisitos de proteção de dados pessoais (RGPD). A conformidade fiscal certificada da Autoridade Tributária está fora do âmbito do projeto, conforme documentado na ADR-009.

O sistema adota uma arquitetura **em camadas (*layered architecture*) com uma instância única centralizada**, na qual todas as lojas se ligam à mesma aplicação *web* através da Internet. Este modelo simplifica a implantação e a manutenção evolutiva (uma única *base-line* de código, uma única base de dados, uma única *pipeline* de *deploy*), evita problemas de divergência de dados entre instâncias e elimina a complexidade associada à sincronização entre nós distribuídos. A justificação detalhada desta escolha encontra-se na *Architecture Decision Record* ADR-001 (Secção 4.5).

A arquitetura está organizada em três camadas lógicas:

- **Camada de Apresentação (UI)**: composta por duas aplicações *web* servidas a partir do mesmo servidor central — `POSWebUI` (interface de ponto de venda, otimizada para velocidade e simplicidade de operação) e `BackofficeWebUI` (portal de gestão para Gerentes de Loja, Administrador da Cadeia e Administrador IT). Ambas são desenvolvidas em **Blazor**, sobre o ecossistema *.NET*.
- **Camada de Aplicação (*Business Logic*)**: contém os serviços que implementam as regras de negócio, organizados por domínio funcional — `AuthAccessService`, `SalesService`, `InventoryService`, `ProcurementService`, `CatalogService`, `ReportingService`, `StoreManagementService` e `AuditService`. Inclui ainda dois componentes de execução agendada: `ConsolidationScheduler` (responsável pela geração de agregados diários e materialização de KPIs) e `ReplenishmentScheduler` (responsável por gerar sugestões automáticas de reposição).

- **Camada de Persistência**: composta pelo componente `RelationalPersistence`, que abstrai o acesso à base de dados relacional, e pela própria base de dados `BeloMercadoDB`. A escolha de uma base de dados relacional decorre da natureza fortemente transacional e estruturada do domínio (vendas, faturação, *stock*), bem como dos requisitos de integridade ACID (RNF-06).

A arquitetura segue o princípio da **separação de responsabilidades**: cada componente tem uma fronteira funcional bem definida e comunica com os restantes através de interfaces explícitas (descritas no diagrama de componentes da Secção 4.2.3). Esta organização favorece a manutenibilidade (RNF-14), a testabilidade e a evolução incremental do sistema — novos casos de uso podem ser adicionados a um serviço existente sem impactar os restantes.

O `AuditService` ocupa um papel **transversal**: é invocado por todos os outros serviços para registar eventos relevantes do ponto de vista de segurança, auditoria e conformidade (RF18, RNF-08, RNF-10). Esta abordagem centraliza a política de auditoria num único componente, evitando a dispersão de lógica de *logging* pelo código de domínio.

O acesso ao sistema é controlado pelo `AuthAccessService`, que implementa autenticação baseada em *JSON Web Tokens* (JWT) com *refresh tokens*, conforme detalhado no diagrama de sequência SD-UC15 (Secção 4.2.2). As permissões diferenciadas por perfil (Administrador da Cadeia, Gerente de Loja, Operador de Caixa, Administrador IT) são aplicadas ao nível de cada serviço, com base nas *claims* contidas no *token*.

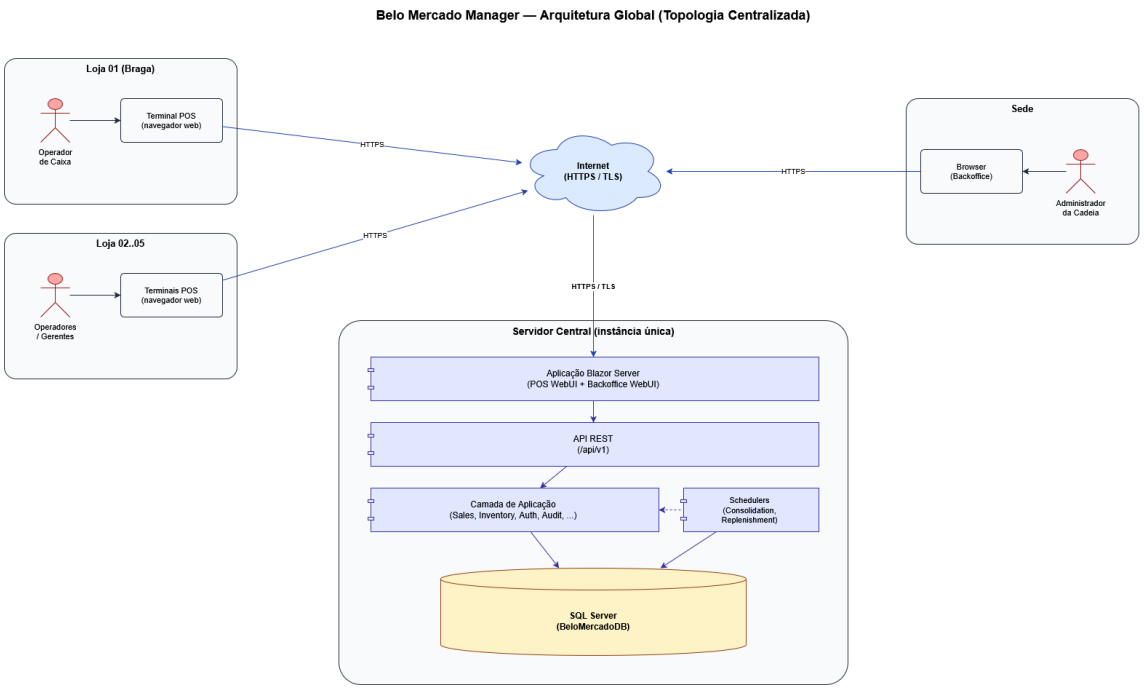


Figura 4.1: Arquitetura global da solução.

Interação com o LLM

Prompt utilizado
Atua como Arquiteto de *Software* sénior. Com base nos requisitos funcionais (RF01–RF19), nos requisitos não funcionais (RNF-01 a RNF-16) e nos casos de uso já definidos para o sistema Belo Mercado Manager, propõe uma arquitetura global adequada ao contexto: cadeia de 5 lojas de retalho de proximidade, *stack* fixada em Blazor/.NET com base de dados relacional, projeto académico de 5 alunos durante 3 semanas. Avalia as alternativas (instância centralizada única vs. instância por loja com sincronização) e justifica a escolha. Identifica os componentes principais e o estilo arquitetural recomendado.

Análise crítica
A resposta inicial do LLM apresentou uma proposta excessivamente sofisticada, recomendando uma arquitetura híbrida com instâncias autónomas por loja, padrão de *transactional outbox* e modelo de *Clean Architecture* com elementos de DDD. Embora tecnicamente sólida, esta abordagem foi considerada desproporcionada para o âmbito académico e para o prazo de 3 semanas disponível para a Etapa 2. A equipa optou por uma topologia mais simples (instância única centralizada) e uma arquitetura em camadas clássica, justificada na ADR-001. O LLM mostrou-se eficaz a clarificar *trade-offs* entre as alternativas e a articular a justificação da escolha, mas tendeu sistematicamente a sobrestimar a complexidade necessária — foi essencial impor explicitamente as restrições de prazo e de equipa em cada *prompt* para obter respostas adequadas ao contexto.

4.2 Arquitetura da Camada de Negócios

Esta secção descreve em detalhe a estrutura interna da camada de negócios, recorrendo a diagramas UML produzidos com o auxílio de LLM e finalizados em *Visual Paradigm*: o diagrama de classes do domínio, os diagramas de sequência dos casos de uso prioritários, e o diagrama de componentes que sintetiza a organização funcional do sistema.

4.2.1 Diagrama de Classes

O diagrama de classes (Figura 4.2) modela as entidades centrais do domínio do sistema, organizadas em **8 *packages*** que refletem os principais subdomínios identificados na Etapa 1: Vendas, Catálogo, Inventário, Compras, Organização, Segurança, Operação Diária e Relatórios.

Os principais elementos do modelo são:

- ***Package* Vendas**: contém a classe `Venda` (registo principal de uma transação no POS), `LinhaVenda` (cada item da venda), `Fatura` (documento fiscal emitido) e `Cliente`, entidade autónoma que armazena os dados mínimos necessários para emissão de fatura

nominativa — NIF, nome (opcional), email (opcional) e data de criação. `Cliente` é independente de `Utilizador` (são entidades com naturezas distintas: `Cliente` representa o consumidor externo identificado por NIF; `Utilizador` representa o funcionário que opera o sistema), associando-se opcionalmente a `Venda` e a `Fatura` apenas quando o consumidor solicita identificação fiscal.

- ***Package* Catálogo**: contém `Produto` (referência centralizada com EAN-13), `Categoria` (com suporte a hierarquia) e dois *value objects* — `Preco` e `Quantidade` — que encapsulam regras de validação e formato.
- ***Package* Inventário**: contém `Stock`, que representa o nível de existências de um produto numa loja específica, com limites mínimo e máximo configuráveis.
- ***Package* Compras**: contém `Encomenda`, `LinhaEncomenda` e `Fornecedor`, suportando o fluxo de aprovisionamento (UC09–UC11).
- ***Package* Organização**: contém `Cadeia` (entidade agregadora), `Loja`, `Funcionario` e `GerenteDeLoja` (especialização de `Funcionario` via generalização UML).
- ***Package* Segurança**: contém `Utilizador`, `Perfil` e `LogAuditoria`, suportando autenticação, autorização e rastreabilidade.
- ***Package* Operação Diária**: contém `CaixaDiaria` (registo de abertura/fecho de caixa) e `ConsolidacaoDiaria` (registo de cada operação de consolidação executada pelo *scheduler*).
- ***Package* Relatórios**: contém `Relatorio`, com suporte a múltiplos tipos (Vendas, *Stock*, Compras, Financeiro, KPI) e exportação em PDF/CSV.

O modelo recorre a ***enums*** para representar estados e tipos discretos: `EstadoVenda`, `EstadoEncomenda`, `EstadoCaixa`, `EstadoConsolidacao`, `MetodoPagamento` e `TipoRelatorio`. Esta abordagem garante consistência de valores e simplifica validações.

As principais relações são de **composição** (e.g., `Venda` compõe-se de `LinhaVenda`; `Encomenda` compõe-se de `LinhaEncomenda`) e de **associação** com cardinalidades explícitas. A cardinalidade `Loja 1 -- 0..* Stock` reflete o facto de cada produto ter um nível de *stock* próprio em cada loja, permitindo gestão local autónoma do inventário ainda que o catálogo seja partilhado entre lojas (RF05).

Note-se que o atributo `hashIntegridade` originalmente previsto na classe `ConsolidacaoDiaria` foi removido em virtude da topologia adotada: como não existe transferência de dados entre nós distribuídos, a integridade da consolidação é garantida pelas propriedades ACID das transações da base de dados, tornando o *hash* de integridade redundante. Esta decisão está documentada na ADR-006.

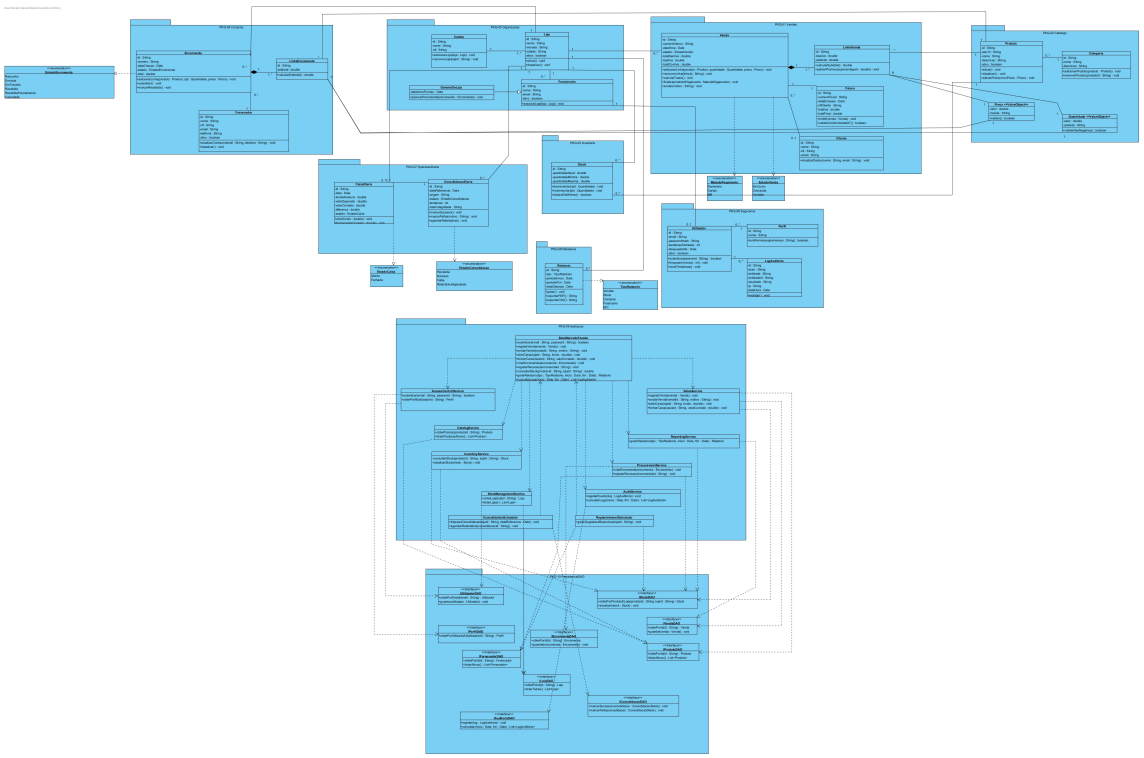


Figura 4.2: Diagrama de classes do sistema.

Interação com o LLM

Prompt utilizado
Atua como Arquiteto de *Software*. A partir do modelo de domínio, dos casos de uso (UC01–UC19) e dos requisitos funcionais já definidos para o Belo Mercado Manager, gera um diagrama de classes detalhado em PlantUML. Organiza as classes em *packages* por subdomínio funcional. Identifica *value objects*, *enums* e relações com cardinalidades explícitas. Inclui apenas atributos e métodos que sejam relevantes para o domínio (evita acessores triviais).

Análise crítica
O LLM gerou um diagrama de classes inicial bastante completo, com identificação correta de *value objects* (`Preco`, `Quantidade`) e *enums* apropriados. A organização em *packages* foi sugerida espontaneamente e mostrou-se adequada. Foram, no entanto, necessários ajustes manuais: algumas classes vinham com excesso de atributos técnicos (e.g., *timestamps* de criação/modificação em todas as entidades) que poluíam a leitura do diagrama; foi introduzida a classe `Cliente` apenas após segunda iteração, quando se notou que a fatura com NIF pressupunha esta entidade; a generalização `GerenteDeLoja` → `Funcionario` foi sugerida pelo LLM e aceite por refletir o papel especializado deste perfil. A decisão de remover o atributo `hashIntegridade` de `ConsolidacaoDiaria` foi tomada pela equipa após reavaliação da topologia (passagem de instância híbrida para centralizada) e o LLM confirmou que era seguro fazê-lo, dado que a integridade passa a ser garantida pelas transações ACID.

4.2.2 Diagramas de Sequência

Foram modelados diagramas de sequência para os casos de uso considerados **prioritários** pela sua centralidade operacional ou pela complexidade dos fluxos envolvidos: UC01 (Registar Venda), UC04 (Abrir e Fechar Caixa), UC10 (Criar Encomenda a Fornecedor), UC12 (Consolidar Dados Diários) e UC15 (Autenticar no Sistema). Os restantes casos de uso seguem padrões de interação suficientemente próximos destes para que a sua descrição em sequência fosse considerada redundante para efeitos de documentação arquitetural.

4.2.2.1 SD-UC01 — Registar Venda (POS)

O diagrama SD-UC01 (Figura 4.3) descreve o fluxo nuclear do sistema: o registo de uma venda no posto de caixa. O Operador de Caixa inicia uma sessão de venda através da `POSWebUI`, que invoca o `SalesService` para criar a sessão. O `SalesService` valida que existe uma caixa do dia aberta (consultando a camada de persistência), sem a qual nenhuma venda pode ser registada (RN01 do UC01).

Para cada item adicionado à venda, o `SalesService` coordena três operações: obtém o preço e regras aplicáveis junto do `CatalogService` (incluindo aplicação automática de promoções,

RF03); reserva o *stock* junto do `InventoryService`; e atualiza os totais da venda. O fluxo prevê um cenário de exceção — *stock* insuficiente — que devolve um erro de negócio à interface, permitindo ao operador ajustar a quantidade.

Quando o operador finaliza a venda, o `SalesService` persiste a venda, as suas linhas e a fatura numa única transação atómica (garantindo RNF-06). Se a persistência falhar, é feito *rollback* e o operador é notificado; se for bem-sucedida, é registado um evento no `AuditService` (`VENDA_FINALIZADA`) e a interface confirma a conclusão da venda com o número de fatura emitido.

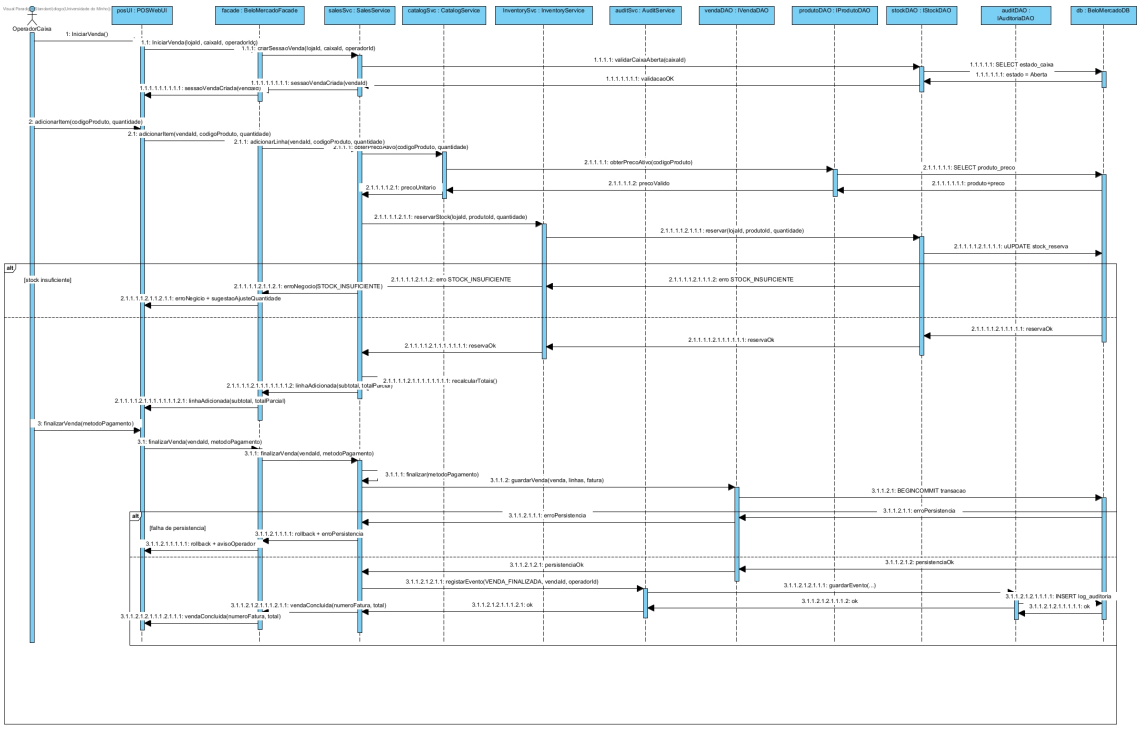


Figura 4.3: SD-UC01 — Registar Venda.

4.2.2.2 SD-UC04 — Abrir e Fechar Caixa

O diagrama SD-UC04 (Figura 4.4) descreve dois fluxos relacionados: a abertura de caixa no início do turno, e o fecho de caixa no final do dia. Na abertura, o operador introduz o fundo inicial; o `SalesService` valida que não existe outra caixa aberta para a loja no mesmo dia (RN01 do UC04, "1 caixa por loja por dia") e cria o registo `CaixaDiaria`.

No fecho, o sistema calcula o valor esperado em caixa (com base nas vendas registadas) e compara-o com o valor contado pelo operador, calculando a diferença. O fecho de caixa é o evento que **aciona a consolidação diária** (UC12), através de uma chamada ao `ConsolidationScheduler`. Esta integração entre fluxos garante que os agregados diários ficam disponíveis no *dashboard*

do Administrador o mais cedo possível, sem necessidade de esperar pelo *trigger* agendado das 23h30.

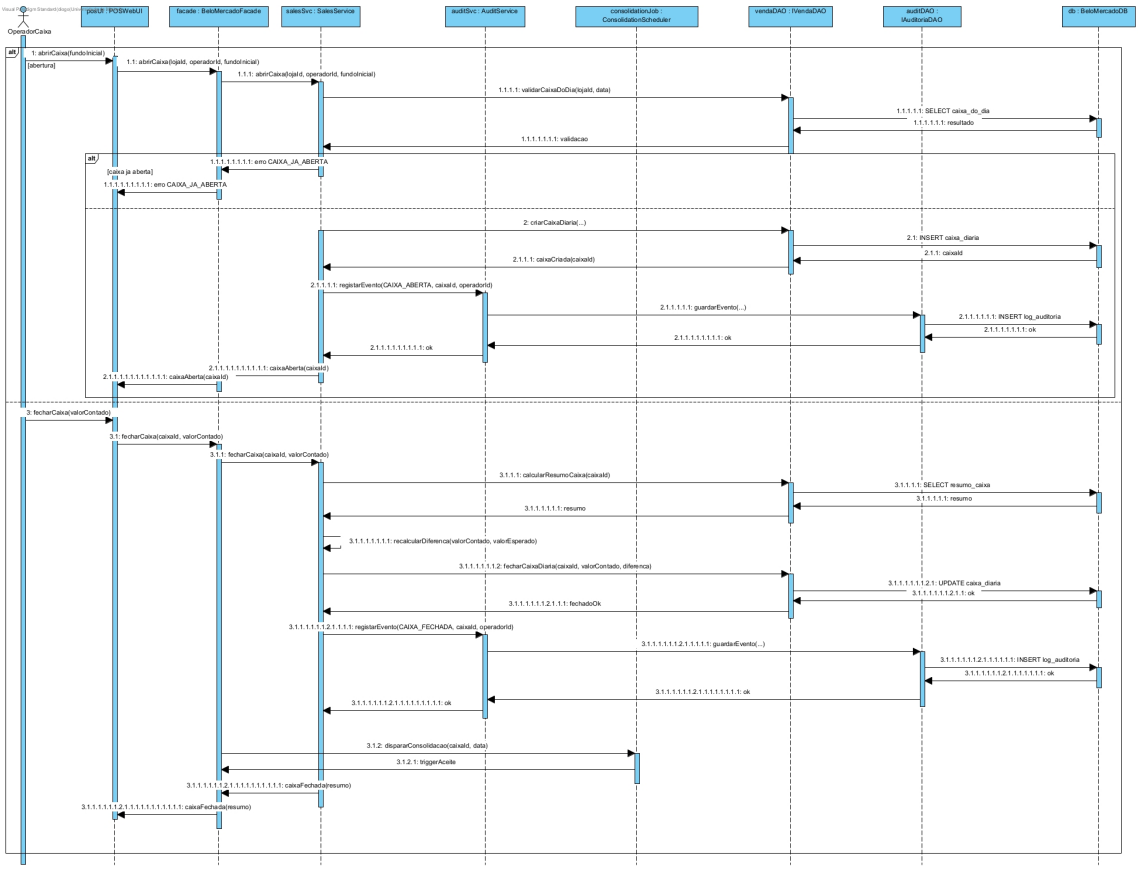


Figura 4.4: SD-UC04 — Abrir e Fechar Caixa.

4.2.2.3 SD-UC10 — Criar Encomenda a Fornecedor

O diagrama SD-UC10 (Figura 4.5) descreve o fluxo de criação de uma encomenda a fornecedor pelo Gerente de Loja. O fluxo divide-se em três fases: criação do rascunho (com validação de fornecedor ativo), adição de linhas (com consulta do estado de *stock* ao `InventoryService` para apoiar a decisão), e submissão final da encomenda. Cada transição de estado é registada no `AuditService`.

A interação entre `ProcurementService` e `InventoryService` ilustra um padrão comum no sistema: serviços de domínio diferentes colaboram através de interfaces explícitas, sem partilha direta de dados. Esta abordagem mantém a coesão de cada serviço e facilita a sua evolução independente.

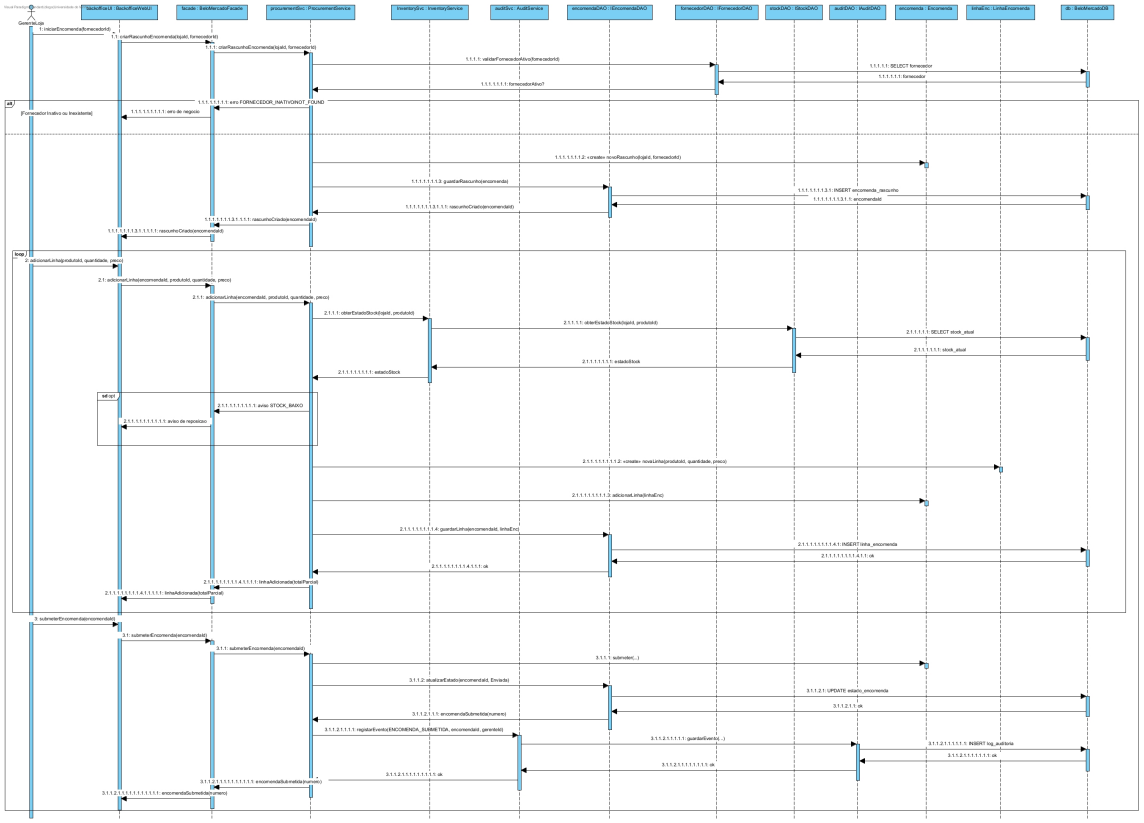


Figura 4.5: SD-UC10 — Criar Encomenda a Fornecedor.

4.2.2.4 SD-UC12 — Consolidar Dados Diários

O diagrama SD-UC12 (Figura 4.6) foi reformulado em virtude da adoção da topologia centralizada (ADR-001). Na topologia inicialmente considerada (instância por loja com sincronização para o central), a consolidação envolvia transferência de dados via API entre nós distintos, com chave de idempotência e *hash* de integridade do *payload*. Na topologia atual, todos os dados já estão na base de dados central desde o momento da sua criação, pelo que a consolidação deixa de ser uma operação de transferência e passa a ser uma operação de **agregação e materialização** de dados para fins analíticos e operacionais.

O `ConsolidationScheduler` dispara a consolidação por dois *triggers* possíveis: imediatamente após o fecho de caixa de uma loja (*trigger* `FechoCaixa`, propagado a partir do SD-UC04), ou pelo *cron* agendado das 23h30 (*trigger* `Agendado`). Para cada loja com fecho de caixa concluído, o `ConsolidationService` colabora com `SalesService`, `InventoryService` e `ReportingService` para calcular agregados de vendas, movimentos de *stock* e KPIs do dia. Os resultados são persistidos em tabelas de leitura denormalizadas (consultadas diretamente pelo *dashboard*) e o evento de sucesso é registado no `AuditService`.

O cenário de exceção — falha no cálculo de agregados (e.g., *timeout* de *query*, *deadlock*) — aciona uma estratégia de retentativa em 30, 60 e 90 minutos (RNF-07), mantendo a robustez do processo sem necessidade de intervenção manual.

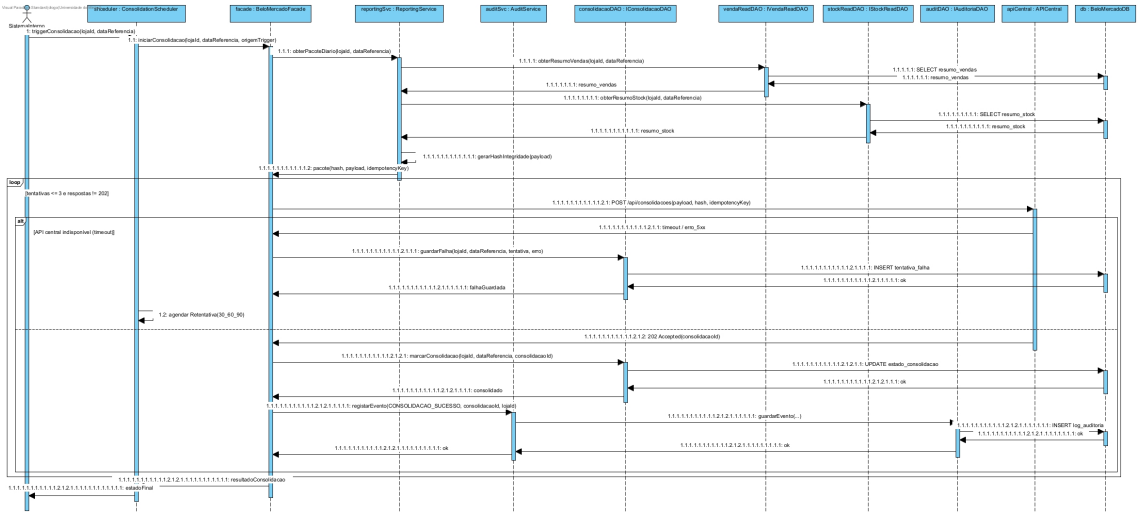


Figura 4.6: SD-UC12 — Consolidar Dados Diários (topologia centralizada).

4.2.2.5 SD-UC15 — Autenticar no Sistema

O diagrama SD-UC15 (Figura 4.7) descreve o fluxo de autenticação. O utilizador submete credenciais à `WebUI`, que invoca o `AuthAccessService`. Este obtém o utilizador da base de dados, verifica o *hash* da *password* e valida o estado da conta (ativa, não bloqueada).

O fluxo prevê três cenários: credenciais inválidas (incrementa o contador de tentativas falhadas, bloqueando a conta após 5 tentativas conforme RN01 do UC15, retornando 401); conta bloqueada (retorna 423); *login* válido (atualiza *último login*, regista evento de auditoria e devolve um par JWT + *refresh token* à interface, juntamente com o perfil do utilizador para fins de redirecionamento). O JWT inclui as *claims* necessárias para autorização nos restantes serviços (`sub`, `role`, `lojaId`, `exp`).

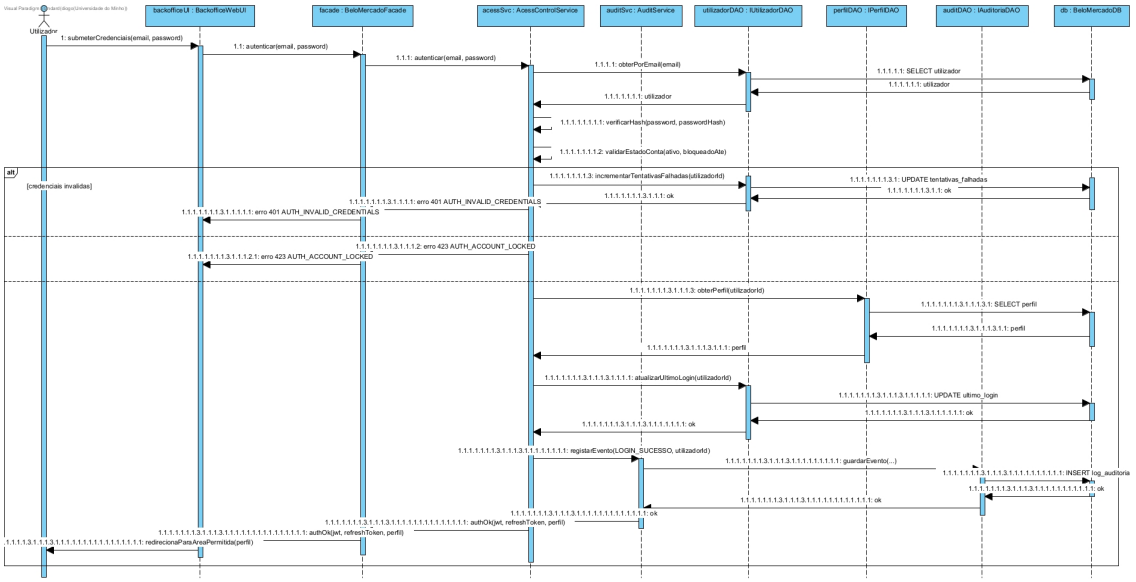


Figura 4.7: SD-UC15 — Autenticar no Sistema.

Interação com o LLM

Prompt utilizado

Atua como Arquiteto de *Software*. Para cada um dos casos de uso prioritários (UC01, UC04, UC10, UC12, UC15), gera um diagrama de sequência em PlantUML que mostre as interações entre o ator, a UI, os serviços de aplicação envolvidos, a camada de persistência e o `AuditService`. Inclui fluxos alternativos e de exceção relevantes (`alt/else`). Mantém consistência de nomes de serviços com o diagrama de componentes.

Análise crítica

O LLM mostrou-se particularmente eficaz na geração de diagramas de sequência: as interações propostas eram quase sempre coerentes com o diagrama de componentes e com as regras de negócio dos casos de uso. Foram necessárias correções pontuais — por exemplo, o fluxo inicial do SD-UC01 não validava a caixa aberta antes de criar a sessão de venda, o que foi acrescentado por insistência da equipa; e o SD-UC15 não previa inicialmente o cenário de conta bloqueada como caso distinto. A reformulação do SD-UC12 (de modelo distribuído para modelo centralizado) foi feita iterativamente: o LLM produziu uma primeira proposta que ainda continha vestígios da topologia anterior (chave de idempotência, *hash* de *payload*), e foi necessário um segundo *prompt* explicitando a topologia A para obter um diagrama coerente.

4.2.3 Diagrama de Componentes

O diagrama de componentes (Figura 4.8) sintetiza a organização funcional do sistema, identificando os componentes principais, as suas dependências e as interfaces que expõem. Os

componentes estão agrupados em três *packages* correspondentes às camadas arquiteturais: **UI** (2 componentes), **Application** (10 componentes) e **Persistence** (2 componentes).

Cada componente da camada de Application expõe uma ou mais interfaces lógicas (e.g., `ISalesApi`, `ICaixaApi`, `IStockApi`) que são consumidas pela camada de UI. Esta separação por interface — em vez de uma API monolítica — documenta com precisão as fronteiras de responsabilidade de cada serviço e facilita a substituição ou reimplementação de componentes individuais.

A camada de Application acede à base de dados exclusivamente através do componente `RelationalPersistence`, que abstrai os detalhes do ORM (*Entity Framework Core*) e expõe interfaces orientadas a domínio (`IUserRepository`, `IProductRepository`, `IStockRepository`, etc.). Esta abordagem isola o código de domínio das tecnologias de persistência e melhora a testabilidade.

O `AuditService` é representado com setas **transversais** — todos os outros componentes da camada de Application invocam-no para registar eventos. Esta dependência transversal é deliberada: garante que a auditoria não pode ser ignorada acidentalmente em nenhum fluxo de negócio.

O mapeamento entre casos de uso e componentes primários (incluído na nota inferior do diagrama) facilita a rastreabilidade entre requisitos e arquitetura, e é detalhado na matriz de rastreabilidade (Secção 4.7).

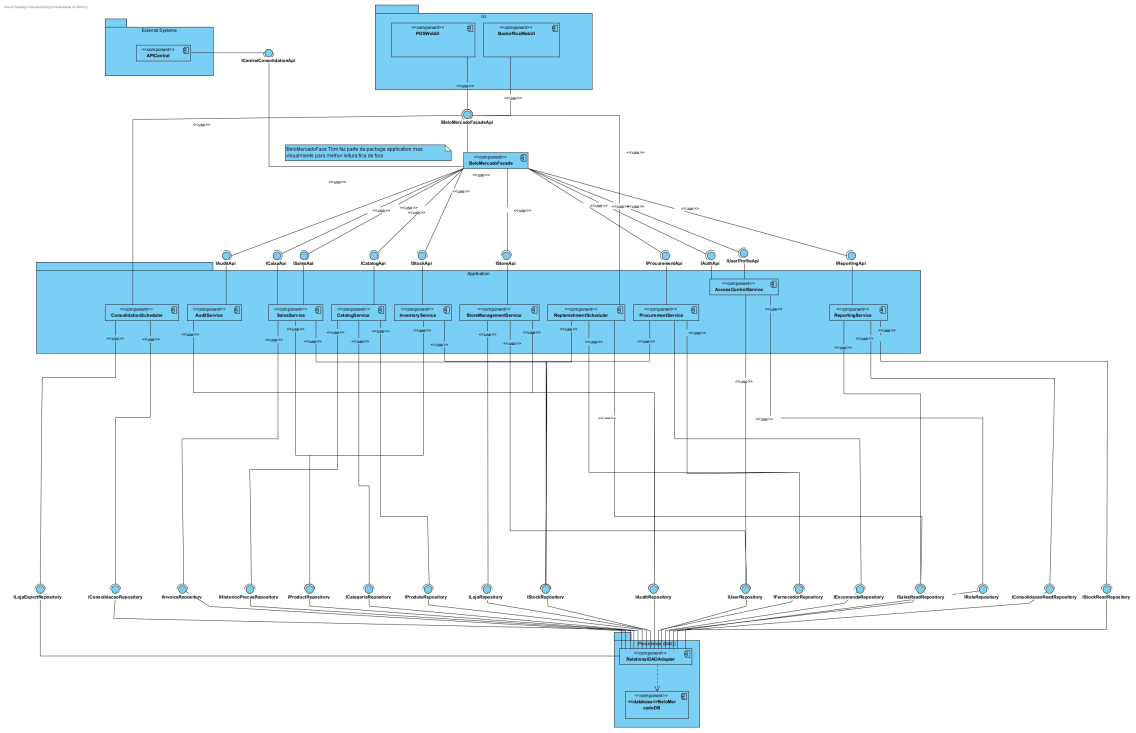


Figura 4.8: Diagrama de componentes.

Interação com o LLM

Prompt utilizado

Com base no diagrama de classes e nos casos de uso, gera um diagrama de componentes em PlantUML para o sistema Belo Mercado Manager. Organiza os componentes em camadas (UI, Application, Persistence). Para cada componente da camada de Application, identifica a(s) interface(s) que expõe. Indica o componente primário responsável por cada caso de uso. Modela o `AuditService` como dependência transversal a todos os outros serviços.

Análise crítica

A proposta inicial do LLM agrupava todas as funcionalidades em três ou quatro componentes muito genéricos (`UserService`, `BusinessService`, `ReportService`), o que perdia a granularidade necessária para refletir os 19 casos de uso. Foi necessário um *prompt* explícito de refinamento: "divide os componentes da camada de Application por domínio funcional, com cardinalidade aproximada de um componente por subdomínio do diagrama de classes". Após este refinamento, a proposta convergiu para os 10 componentes apresentados, com mapeamento UC→componente coerente. A separação dos *schedulers* (`ConsolidationScheduler`, `ReplenishmentScheduler`) como componentes próprios foi sugerida pela equipa para distinguir claramente os fluxos disparados por agendamento dos fluxos síncronos disparados por utilizador.

4.3 Arquitetura da Camada de Dados

4.3.1 Modelo Lógico

O modelo lógico de dados deriva do diagrama de classes (Secção 4.2.1) através de mapeamento objeto-relacional, com adaptações específicas decorrentes das decisões arquiteturais tomadas. Cada classe não-abstrata é representada por uma tabela; relações de composição traduzem-se em chaves estrangeiras com integridade referencial; *value objects* (`Preco`, `Quantidade`) foram **normalizados em tabelas próprias** (`Preco`) ou embebidos como colunas (quantidades em `LinhaVenda`); *enums* são representados como colunas `VARCHAR` com restrição `CHECK`.

O modelo lógico introduz **cinco tabelas adicionais** face à modelação inicial do diagrama de classes, decorrentes de requisitos identificados durante a especificação da arquitetura:

- `Promocao`, `PromocaoLoja` e `LinhaVendaPromocao` — suportam o requisito RF03 (UC03 *Aplicar Promoções e Descontos*), ausentes do diagrama de classes inicial;
- `KPIDiario` e `RankingProdutosDiario` — tabelas de leitura denormalizadas materializadas pelo `ConsolidationScheduler` (UC12), conforme reformulação descrita na ADR-006.

A tabela `Fatura` mantém uma estrutura simplificada (número fiscal sequencial por loja/ano, referência opcional ao cliente, totais e referência à venda), por se tratar de uma implementação académica que não exige conformidade fiscal certificada (ver ADR-009). A entidade `Cliente` é representada como tabela própria (`cliente_id`, `nif`, `nome`, `email`, `data_criacao`) e relaciona-se com `Venda` através de chave estrangeira opcional `cliente_id` em `Venda`, refletindo a cardinalidade `0..* -- 0..1` (a maioria das vendas é a consumidor final não identificado e não exige registo do cliente). A tabela `Cliente` é totalmente independente da tabela `Utilizador` — são entidades com naturezas e ciclos de vida distintos, e qualquer relação direta entre ambas seria semanticamente incorreta (um `Utilizador` é um funcionário interno com credenciais de acesso ao sistema, enquanto um `Cliente` é apenas um registo de identificação fiscal de um consumidor externo).

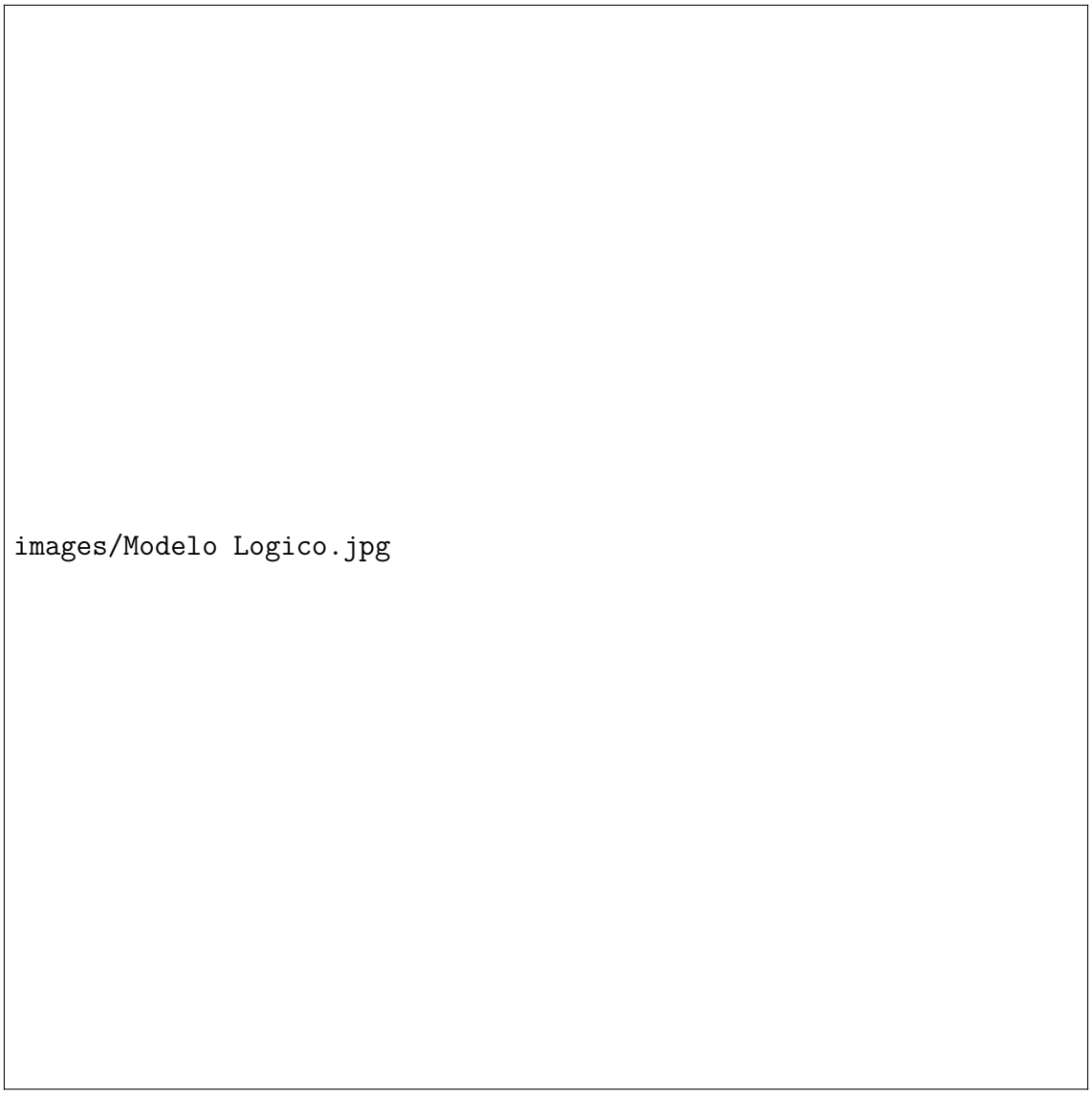


Figura 4.9: Modelo lógico de dados.

O esquema completo conta com 26 tabelas, organizadas pelos mesmos subdomínios do diagrama de classes. As principais convenções adotadas são:

- Chaves primárias do tipo `UUID`, geradas pela aplicação ou pela base de dados (`uuid_generate_v4()`), evitando dependência de sequências e simplificando integração;
- Restrições de integridade referencial em todas as chaves estrangeiras, com `ON DELETE CASCADE` apenas onde existe relação de composição forte (`LinhaVenda` → `Venda`, `LinhaEncomenda` → `Encomenda`);
- Restrições `CHECK` para validação de domínios (estados, taxas de IVA, valores não-

negativos);

- Índices secundários em colunas frequentemente filtradas (`loja_id+data_hora` em `Venda`, `loja_id+data_referencia` em `KPIDiario`, etc.);
- Restrições `UNIQUE` para invariantes de domínio (uma caixa por loja por dia, um produto por loja em `Stock`, série/número de fatura único).

O DDL completo para SQL Server, incluindo todas as restrições, índices e *check constraints*, está disponível no repositório Git do projeto, no ficheiro `database/ddl_belomercado.sql`, sendo omitido deste relatório por questões de extensão.

Interação com o LLM

Prompt utilizado
Atua como Engenheiro de Dados. A partir do diagrama de classes do Belo Mercado Manager e das decisões arquiteturais tomadas (ADR-001 a ADR-010), produz o modelo lógico relacional, identificando: (1) tabelas em falta no diagrama de classes (e.g., promoções, agregados de leitura); (2) restrições de integridade (`UNIQUE`, `CHECK`, `ON DELETE`); (3) índices secundários que suportem as *queries* mais frequentes dos casos de uso. Gera o resultado em formato DBML e DDL para SQL Server.

Análise crítica
O LLM revelou-se eficaz a detectar incoerências entre o diagrama de classes e os requisitos — nomeadamente a ausência total de modelação de promoções no diagrama, apesar do RF03 e do UC03 as exigirem; e a duplicação de `email` entre `Funcionario` e `Utilizador`. A primeira proposta do LLM tendeu, no entanto, a sobre-modelar o domínio (e.g., propôs uma tabela `Endereco` normalizada apenas para suportar moradas de fornecedores e clientes, e propôs ainda uma estrutura completa de séries de faturação com encadeamento criptográfico de *hash* para conformidade com a Portaria 363/2010, o que foi rejeitado por se entender que a conformidade fiscal certificada está fora do âmbito de um projeto académico — ver ADR-009). A decisão de simplificar `VendaCliente` para chave estrangeira opcional foi sugerida pela equipa após verificar que a cardinalidade real era `0..1` e não `0..*`.

4.4 Especificação da API

A API do sistema segue o estilo arquitetural **REST**, com formato `application/json` e autenticação baseada em *Bearer JWT* (com exceção do *endpoint* de *login*). A escolha de REST face a alternativas como GraphQL é justificada na ADR-008.

Convenções gerais aplicáveis a todos os *endpoints*:

- ***Base path***: `/api`.

- **Autenticação**: cabeçalho `Authorization: Bearer <jwt>` obrigatório, exceto em `POST /api/auth/login`.
- **Versionamento**: prefixo de versão na URL (ex.: `/api/v1/...`); a versão atual é a v1.
- **Identificação de loja**: cabeçalho `X-Store-Id` obrigatório em *endpoints* operacionais que dependem de contexto de loja, validado contra a *claim* `lojaId` do JWT.
- **Rastreabilidade**: cabeçalho `Correlation-Id` opcional, propagado em respostas e *logs* para facilitar diagnóstico.
- **Erros**: respostas de erro seguem o formato RFC 7807 (`application/problem+json`) com código de erro de domínio (e.g., `STOCK_INSUFICIENTE`, `AUTH_INVALID_CREDENTIALS`) para além do código HTTP.

Os *endpoints* estão organizados por agregado de domínio. A Tabela **??** apresenta a especificação completa, com mapeamento explícito a casos de uso.

Método	*Path*	UC	Autorização	Erros principais
Autenticação e Perfil				
POST	/api/auth/login	UC15	Pública	401, 423
POST	/api/auth/refresh	UC15	Refresh token	401
POST	/api/auth/logout	UC15	Bearer	401
GET	/api/me	UC17	Bearer	401
PUT	/api/me	UC17	Bearer	422
PUT	/api/me/password	UC17	Bearer	401, 422
Vendas e Caixa				
POST	/api/vendas	UC01	Operador, Gerente	409, 422
GET	/api/vendas/{id}	UC01	Operador, Gerente	404
POST	/api/vendas/{id}/anular	UC02	Operador, Gerente	404, 409, 422
POST	/api/vendas/{id}/devolucao	UC02	Operador, Gerente	404, 422
POST	/api/caixas/abrir	UC04	Operador	409, 422
POST	/api/caixas/{id}/fechar	UC04	Operador	404, 409, 422
GET	/api/caixas/{id}	UC04	Operador, Gerente	404
Promoções				
GET	/api/promocoes	UC03	Bearer	—
POST	/api/promocoes	UC03	Admin	422
PUT	/api/promocoes/{id}	UC03	Admin	404, 422
DELETE	/api/promocoes/{id}	UC03	Admin	404, 409
Catálogo de Produtos				
GET	/api/produtos	UC05	Bearer	—
GET	/api/produtos/{id}	UC05	Bearer	404
POST	/api/produtos	UC05	Admin	409, 422
PUT	/api/produtos/{id}	UC05	Admin	404, 422
DELETE	/api/produtos/{id}	UC05	Admin	404, 409
GET	/api/categorias	UC05	Bearer	—
POST	/api/categorias	UC05	Admin	422
Inventário				
GET	/api/stock	UC06	Gerente, Admin	—
GET	/api/stock/alertas	UC06	Gerente, Admin	—
POST	/api/inventarios	UC07	Gerente	409, 422
GET	/api/sugestoes-reposicao	UC08	Gerente	—

Tabela 4.1: Especificação da API REST do sistema Belo Mercado Manager — Parte 1.

Método	*Path*	UC	Autorização	Erros principais
Fornecedores e Encomendas				
GET	`/api/fornecedores`	UC09	Gerente, Admin	—
POST	`/api/fornecedores`	UC09	Admin	409, 422
PUT	`/api/fornecedores/{id}`	UC09	Admin	404, 422
POST	`/api/encomendas`	UC10	Gerente	422
GET	`/api/encomendas`	UC10	Gerente, Admin	—
GET	`/api/encomendas/{id}`	UC10	Gerente, Admin	404
POST	`/api/encomendas/{id}/submeter`	UC10	Gerente	404, 409
POST	`/api/encomendas/{id}/rececionar`	UC11	Gerente	404, 409, 422
Relatórios e *Dashboards*				
GET	`/api/dashboards/kpi`	UC13	Gerente, Admin	—
GET	`/api/relatorios`	UC14	Gerente, Admin	—
POST	`/api/relatorios/exportar`	UC14	Gerente, Admin	422
Administração e Auditoria				
GET	`/api/utilizadores`	UC16	Admin, IT	—
POST	`/api/utilizadores`	UC16	Admin, IT	409, 422
PUT	`/api/utilizadores/{id}`	UC16	Admin, IT	404, 422
DELETE	`/api/utilizadores/{id}`	UC16	Admin, IT	404
GET	`/api/auditoria/logs`	UC18	Admin, IT	—
GET	`/api/lojas`	UC19	Admin	—
POST	`/api/lojas`	UC19	Admin	409, 422
PUT	`/api/lojas/{id}`	UC19	Admin	404, 422

Tabela 4.2: Especificação da API REST do sistema Belo Mercado Manager — Parte 2.

Notas adicionais sobre *endpoints* críticos:

`POST /api/vendas`: requer cabeçalho `Idempotency-Key` (UUID) para garantir que retentativas do cliente em caso de falha de rede não geram vendas duplicadas. O servidor mantém um cache de chaves processadas durante 7 dias.

`POST /api/encomendas/{id}/submeter`: encomendas com valor superior a 5000 € exigem aprovação adicional do Administrador (devolve 403 com código `APPROVAL_REQUIRED` se chamado por Gerente).

`POST /api/vendas/{id}/anular`: anulações sobre vendas com mais de 24 horas exigem perfil Gerente (devolve 403 com código `APPROVAL_REQUIRED` se chamado por Operador), conforme RN01 do UC02.

Operação interna (não exposta como API pública): a consolidação diária (UC12) é executada exclusivamente pelo `ConsolidationScheduler` interno e não dispõe de *endpoint* REST público. Pode, contudo, ser disparada manualmente por um administrador IT através de uma operação de gestão (`POST /api/admin/consolidacoes/disparar`, sujeita a permissão IT).

Interação com o LLM

Prompt utilizado
Atua como Arquiteto de *Software*. A partir dos 19 casos de uso (UC01–UC19) e do diagrama de componentes, deriva uma especificação completa de API REST para o sistema Belo Mercado Manager. Para cada *endpoint*, indica método HTTP, *path*, caso de uso associado, perfis autorizados e códigos de erro principais. Aplica convenções REST (verbos HTTP semânticos, recursos pluralizados, sub-recursos para ações específicas).

Análise crítica
A proposta do LLM cobriu adequadamente os 19 casos de uso, com convenções REST bem aplicadas (uso correto de `POST` para ações não-idempotentes, `PUT` para atualizações, *nested paths* para sub-recursos como `/vendas/{id}/anular`). Tendência observada e corrigida: o LLM propunha sistematicamente *endpoints* adicionais que não correspondiam a nenhum caso de uso (e.g., `GET /api/produtos/{id}/historico-precos`, `POST /api/notificacoes`), o que foi removido em revisão para manter a especificação alinhada com os requisitos efetivamente levantados. A decisão de **não expor publicamente** o *endpoint* de consolidação foi tomada pela equipa, contrariando a sugestão inicial do LLM, por ser uma operação interna que não deve ser invocável por clientes externos.

4.5 Decisões Arquiteturais

Esta secção documenta as principais decisões arquiteturais do sistema sob a forma de *Architecture Decision Records* (ADRs), seguindo o formato **Nygard** (Contexto, Decisão, Consequências). Cada ADR captura uma decisão significativa, as alternativas consideradas e os *trade-offs* aceites, servindo de referência para futuras evoluções do sistema.

ADR-001 — Topologia de *Deployment*: Instância Única Centralizada

Estado: Aceite.

Contexto: O sistema serve uma cadeia de 5 lojas geograficamente dispersas que precisam de operar diariamente e cujos dados consolidados devem estar disponíveis ao Administrador. Foram consideradas três topologias: (a) instância única centralizada, com lojas a ligar via Internet; (b) instância por loja com servidor central de consolidação; (c) modelo híbrido com sincronização bidirecional.

Decisão: Adoção de **instância única centralizada** (topologia A). Toda a aplicação *web* é servida por um único *deployment*; as lojas acedem ao sistema através da Internet com autenticação JWT.

Consequências: Vantagens — modelo de *deployment* simples (uma única *base-line*, uma única *pipeline* CI/CD); inexistência de problemas de divergência de dados entre instâncias; *dashboards* sempre atualizados em tempo real (a consolidação UC12 deixa de ser de transferência e passa a ser de agregação); manutenção e evolução evolutiva simplificadas. Desvantagens aceites — o sistema fica dependente da disponibilidade da ligação à Internet em cada loja (o requisito RT05 de operação *offline* foi reavaliado e considerado fora de âmbito para a Etapa 2; o sistema confia na disponibilidade da ligação para a operação POS); a disponibilidade do sistema central torna-se mais crítica (mitigada pelo RNF-04 e RNF-05).

ADR-002 — Estilo Arquitetural: Arquitetura em Camadas

Estado: Aceite.

Contexto: Foi necessário escolher um estilo arquitetural que organizasse o código de forma a favorecer manutenibilidade, testabilidade e clareza. Foram avaliadas três opções: (a) arquitetura em camadas clássica (*N-tier*); (b) *Clean Architecture* com inversão de dependências e DDD tático; (c) arquitetura hexagonal (*ports and adapters*).

Decisão: Adoção de **arquitetura em camadas** com três camadas (UI, Application, Persistence) e separação clara de responsabilidades. A camada de Application está organizada por domínio funcional em serviços (`SalesService`, `InventoryService`, etc.) que encapsulam regras de negócio.

Consequências: Vantagens — estilo amplamente conhecido pela equipa, com *learning curve* reduzida; baixa cerimónia de implementação; alinhado com a documentação oficial de Microsoft para aplicações ASP.NET Core de complexidade moderada. Desvantagens aceites — não impõe inversão de dependências entre Application e Persistence (a camada de Application depende diretamente do componente `RelationalPersistence`), o que é mitigado pela utilização de interfaces de repositório; menor adequação a evoluções futuras para microsserviços (não previsto no âmbito).

ADR-003 — *Stack* Tecnológica: Blazor + .NET + SQL Server

Estado: Aceite (decisão prévia, mantida).

Contexto: A *stack* tecnológica é uma restrição técnica do projeto (RT01–RT03), com *.NET*/Blazor para a aplicação *web* e base de dados relacional. A escolha específica do servidor de base de dados foi avaliada nesta etapa.

Decisão: Utilização de **Blazor Server** para a UI (POS e *Backoffice*), **ASP.NET Core** para a camada de Application, **Entity Framework Core** como ORM e **Microsoft SQL Server** (com edição Express em ambientes de desenvolvimento e edição Standard ou superior em produção) como sistema de gestão de base de dados.

Consequências: Vantagens — ecossistema integrado da Microsoft (forte alinhamento entre .NET, EF Core, SQL Server e ferramentas como SQL Server Management Studio e Azure Data Studio); experiência de desenvolvimento integrada (C# em *front-end* e *back-end*); SQL Server bem documentado e com edição gratuita (Express) adequada à fase académica e a deployments de pequena dimensão; suporte nativo a *Always Encrypted*, *Transparent Data Encryption* e auditoria, úteis para os requisitos de RGPD (RNF-10). Desvantagens aceites — Blazor Server depende de uma ligação SignalR persistente, o que reforça a importância da disponibilidade da ligação à Internet (já endereçada pela ADR-001); SQL Server Express tem limites de 10 GB por base de dados, 1 GB de RAM e 1 *CPU socket*, o que exigirá migração para edição Standard se a cadeia crescer significativamente; SQL Server tem custos de licenciamento em produção quando os limites do Express forem ultrapassados.

ADR-004 — Autenticação e Autorização: JWT com *Refresh Token*

Estado: Aceite.

Contexto: O sistema requer autenticação de utilizadores com perfis diferenciados (Operador de Caixa, Gerente de Loja, Administrador da Cadeia, Administrador IT) e gestão de sessões compatível com APIs REST.

Decisão: Adoção de autenticação baseada em **JWT (JSON Web Tokens)** com *access token* de curta duração (15 minutos) e *refresh token* de duração média (7 dias) com rotação a cada utilização. O JWT inclui as *claims* `sub`, `role`, `lojaId` e `exp`. A autorização é verificada em cada *endpoint* com base no perfil e, quando aplicável, no `lojaId` (impedindo que um Gerente de Loja A aceda a dados de Loja B).

Consequências: Vantagens — modelo *stateless* compatível com escalabilidade horizontal; revogação suportada via lista negra de *refresh tokens*; compatível com clientes diversos (*web*, móvel se necessário). Desvantagens aceites — complexidade adicional de gestão de *refresh tokens* (rotação, deteção de reutilização); *tokens* comprometidos só perdem validade no fim da sua duração (mitigado pela curta duração do *access token*).

ADR-005 — Estilo de API: REST com OpenAPI

Estado: Aceite.

Contexto: Foi necessário definir o estilo da API que serve a UI (POS e *Backoffice*) e potencialmente clientes futuros. As opções avaliadas foram REST e GraphQL.

Decisão: Adoção de **REST** com convenções HTTP padrão, formato JSON, especificação OpenAPI 3 gerada a partir do código (Swashbuckle/Scalar) e versionamento por prefixo de URL (`/api/v1`).

Consequências: Vantagens — estilo amplamente conhecido pela equipa e pelos consumidores potenciais; *caching* HTTP nativo; ferramentas maduras (Swagger UI, Postman); idempotência expressa em verbos HTTP. Desvantagens aceites — *over-fetching* possível em alguns endpoints de *dashboard* (mitigado por *endpoints* específicos para casos de uso de leitura intensiva, como `/api/dashboards/kpi`).

ADR-006 — Reformulação da Consolidação Diária (UC12)

Estado: Aceite (substitui versão anterior do SD-UC12).

Contexto: Na conceção inicial, a consolidação diária pressupunha uma topologia distribuída com transferência de dados entre instâncias por loja e um servidor central, recorrendo a chave de idempotência e *hash* de integridade do *payload*. A adoção da topologia centralizada (ADR-001) tornou redundantes estes mecanismos.

Decisão: Reformulação do UC12 para deixar de ser uma operação de **transferência** e passar a ser uma operação de **agregação e materialização**. O `ConsolidationScheduler` colabora com os serviços de domínio (`SalesService`, `InventoryService`, `ReportingService`) para calcular agregados diários (KPIs, *rankings* de produtos, alertas de *stock*) e persisti-los em tabelas de leitura denormalizadas. Os atributos `hashIntegridade` e `origem` da classe `ConsolidacaoDiaria` foram removidos por terem perdido significado.

Consequências: Vantagens — simplificação significativa da implementação (sem necessidade de API de consolidação, sem gestão de retentativas de rede); *dashboards* podem ser atualizados em tempo quase real; melhor aproveitamento do investimento em índices da base de dados (as *queries* de agregação ocorrem na mesma BD onde os dados estão). Desvantagens aceites — a operação de consolidação consome recursos da BD central e pode impactar a operação POS se executada em horário de funcionamento (mitigado pela execução noturna às 23h30 e/ou imediatamente após o fecho de cada caixa).

ADR-007 — `AuditService` como Componente Transversal

Estado: Aceite.

Contexto: Os requisitos RF18 (consultar *logs* de auditoria), RNF-08 (segurança) e RNF-10 (RGPD) exigem que ações relevantes do sistema sejam registadas com rastreabilidade total (utilizador, ação, timestamp, IP, resultado). Foram avaliadas duas abordagens: (a) implementar *logging* de auditoria diretamente em cada serviço; (b) centralizar num componente dedicado.

Decisão: Criação do `AuditService` como componente **transversal**, invocado por todos os serviços de Application no momento de eventos relevantes (login, criação/anulação de venda, alteração de utilizador, etc.). O `AuditService` encapsula a política de retenção, formatação

e armazenamento de *logs*.

Consequências: Vantagens — política de auditoria centralizada e consistente; alterações à política aplicam-se a todos os serviços sem reescrita de código; facilita conformidade com RGPD e auditorias externas. Desvantagens aceites — introduz uma dependência adicional em todos os serviços (mitigada por se tratar de uma chamada *in-process* de baixa latência); risco de o `AuditService` se tornar um ponto de falha (mitigado por persistência assíncrona com *buffer* local em memória).

ADR-008 — ORM: Entity Framework Core

Estado: Aceite.

Contexto: O acesso à base de dados pode ser feito através de SQL puro (com Dapper ou ADO.NET) ou através de um ORM completo. O sistema tem cerca de 20 entidades de domínio com relações ricas, e a equipa privilegia velocidade de desenvolvimento sobre micro-otimização de *queries*.

Decisão: Utilização de **Entity Framework Core** como ORM principal, com migrações geridas via *Code-First*. Para *queries* de leitura intensiva (KPIs, relatórios), está prevista a utilização opcional de Dapper para contornar limitações de *performance* do EF Core em *queries* agregadoras complexas.

Consequências: Vantagens — alta produtividade em CRUD; migrações automatizadas e versionadas; LINQ permite *queries* expressivas e *type-safe*; mapeamento direto do diagrama de classes para entidades EF Core. Desvantagens aceites — *queries* complexas geradas pelo EF Core podem ser ineficientes (mitigado pelo uso seletivo de Dapper); *learning curve* inicial para utilizadores menos experientes (mitigada por documentação interna e revisões de código).

ADR-009 — Conformidade Fiscal: Implementação Simplificada

Estado: Aceite.

Contexto: A faturação em Portugal está sujeita à Portaria 363/2010 e legislação subsequente (incluindo Portaria 195/2020 sobre ATCUD), que exige *software* certificado pela Autoridade Tributária para emissão de faturas a consumidor final. Os mecanismos técnicos previstos pela legislação — numeração sequencial obrigatória, encadeamento criptográfico (*hash chain*) RSA entre faturas, geração de ATCUD, geração de *QR code* conforme especificação da AT, e exportação SAF-T(PT) — são complexos e a sua implementação correta exigiria várias semanas-pessoa. A certificação efetiva está, em qualquer caso, fora do âmbito de um projeto académico.

Decisão: Adoção de uma **implementação simplificada de faturação**, adequada à natureza

académica do projeto. A tabela `Fatura` contém apenas os campos essenciais: `numero_fiscal` (sequencial por loja e ano), `data_emissao`, `cliente_id` (*foreign key* opcional para a entidade `Cliente` quando o consumidor solicita fatura com NIF), `total_iva`, `total_final`. A numeração sequencial é gerida por código aplicacional simples, sem tabela dedicada de séries. Todas as faturas exibem a menção "**PROTÓTIPO ACADÉMICO — SEM CONFORMIDADE FISCAL AT**" no rodapé. Ficam explicitamente fora do âmbito: o encadeamento criptográfico, ATCUD, *QR code*, exportação SAF-T(PT) e qualquer integração com o portal e-Fatura da AT.

Consequências: Vantagens — esforço de implementação reduzido em cerca de 4–5 dias-pessoa, libertando esses recursos para outras frentes da Etapa 3; estrutura de dados claramente identificada como protótipo, evitando o risco de uso indevido em produção; código focado nas competências centrais de Engenharia de *Software* (POS, inventário, consolidação) em detrimento de complexidade fiscal específica de Portugal. Desvantagens aceites — o sistema não pode ser usado em ambiente real; a evolução para sistema certificado exigiria uma reformulação substancial da tabela `Fatura` e a introdução de um módulo criptográfico dedicado, eventualmente com uma tabela `FaturaSerie` a ser adicionada nessa altura.

ADR-010 — Resiliência da Consolidação: Retentativas Automáticas

Estado: Aceite.

Contexto: Embora a consolidação diária na topologia centralizada já não envolva transferência de rede, ainda pode falhar por causas locais à BD: *timeouts*, *deadlocks*, recursos saturados pela operação corrente. O RNF-07 exige resiliência da consolidação.

Decisão: Implementação de uma estratégia de retentativa automática com *backoff* fixo a **30, 60 e 90 minutos** após uma falha. Após 3 tentativas falhadas, o estado da consolidação é marcado como `Falha` e gera-se um alerta crítico para o Administrador IT (UC18 / RF18).

Consequências: Vantagens — robustez face a falhas transitórias sem necessidade de intervenção; o operador não é interrompido; o estado é totalmente rastreável (cada tentativa fica registada na entidade `ConsolidacaoDiaria`). Desvantagens aceites — atraso máximo de 3 horas até alerta em caso de falha persistente (considerado aceitável dado que os dados originais estão sempre seguros na BD operacional).

Interação com o LLM

Prompt utilizado
Atua como Arquiteto de *Software* sénior. Com base na arquitetura definida (instância única centralizada, camadas, Blazor + .NET + SQL Server, JWT, REST), identifica as 8 a 10 decisões arquiteturais mais significativas e documenta-as no formato *Architecture Decision Record* (Nygard): Estado, Contexto, Decisão, Consequências (vantagens e desvantagens aceites). Cada ADR deve ter entre 150 e 250 palavras.

Análise crítica
O LLM produziu uma primeira versão dos ADRs com qualidade desigual: ADRs estruturalmente corretos mas alguns demasiado genéricos (e.g., "Estado: Aceite. Decisão: Usar boas práticas"). Foi necessário um *prompt* de refinamento exigindo **trade-offs explícitos** e **alternativas avaliadas** em cada ADR. A versão final convergiu após 3 iterações, com a equipa a acrescentar manualmente as ADRs mais específicas do contexto (ADR-006 sobre reformulação da consolidação, ADR-009 sobre escolha de uma implementação simplificada de faturação face à legislação portuguesa), que o LLM dificilmente teria proposto sem conhecimento explícito da reformulação ocorrida e do compromisso pragmático adotado pela equipa. A força do LLM nesta tarefa foi a estruturação consistente do formato Nygard; a sua fraqueza foi a tendência para apresentar decisões como livres de *trade-offs*, o que foi corrigido em revisão.

4.6 Proposta de Interface

A proposta de interface foi desenhada como um conjunto de *wireframes* de baixa fidelidade (estilo *blueprint* monocromático), com o objetivo de documentar a estrutura, os fluxos e as decisões de UX dos ecrãs principais do sistema, sem prematuramente comprometer escolhas de design visual. Esta abordagem segue a recomendação de boas práticas de UX (Balsamiq, Nielsen Norman Group) que distingue claramente *wireframes* (estrutura) de *mockups* (estilo) e *prototypes* (interação).

Foi selecionado um **conjunto representativo de seis ecrãs**, três da aplicação POS e três da aplicação *Backoffice*, cobrindo os fluxos de maior impacto operacional (registo de venda, pagamento, fecho de caixa, *dashboard* de gestão, gestão de catálogo e gestão de utilizadores). Os restantes ecrãs serão derivados destes padrões durante a Etapa 3 de implementação.

Os princípios de UX adotados foram:

- **POS otimizado para velocidade** — alvos de toque com dimensão mínima de 60×60 px adequados a uso por operadores em ritmo elevado; foco automático na caixa de pesquisa para receber *input* do leitor de códigos de barras; cores semânticas constantes em todo o sistema (*primary* = confirmar, *danger* = cancelar); operação completa de venda em ≤ 4 *taps* (RNF-01, RNF-11);

- ***Backoffice* otimizado para clareza** — navegação lateral persistente; filtros sempre visíveis no topo das listagens; *master-detail* para edição de entidades; KPIs com indicação de variação face a período anterior; transparência sobre frescura dos dados (e.g., timestamp da última consolidação);
- **Acessibilidade** — contraste WCAG AA (4,5:1 mínimo) entre texto e fundo; informação de estado nunca dependente exclusivamente de cor (sempre acompanhada de texto ou ícone);
- **Tornar regras de negócio visíveis** — regras críticas (e.g., obrigatoriedade de justificação para diferenças de caixa $> 10\,€$, política de bloqueio após 5 tentativas falhadas) são apresentadas inline na UI, ajudando o utilizador a antecipar o comportamento do sistema.

4.6.1 POS — Ecrã de Venda em Curso (UC01)

O ecrã principal do POS (Figura 4.10) divide-se em duas zonas: à esquerda, a lista de itens do carrinho com a caixa de pesquisa que mantém o foco automaticamente para receber *input* do leitor de códigos de barras; à direita, o resumo dos totais (com IVA discriminado por taxa, conforme exigência fiscal) e os botões de ação. As promoções aplicadas aparecem como *badge inline* na linha de venda, fornecendo *feedback* imediato ao operador.

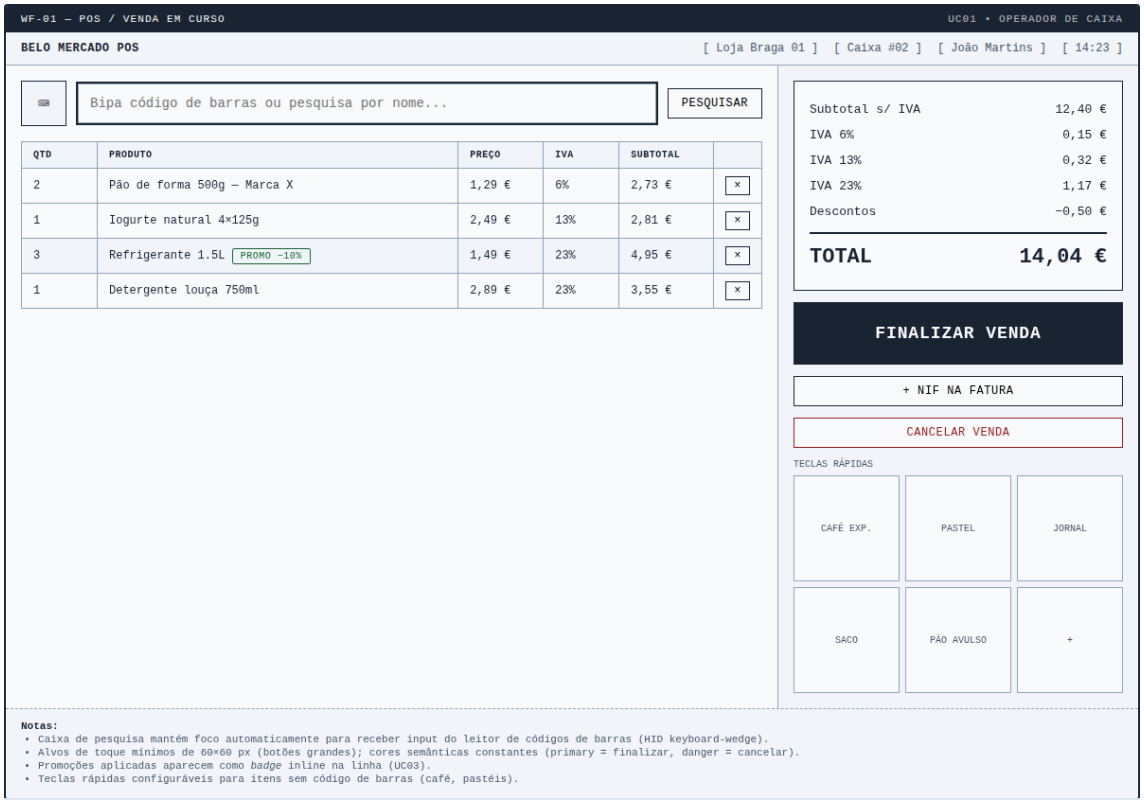


Figura 4.10: WF-01 — POS: ecrã de venda em curso (UC01).

4.6.2 POS — Ecrã de Pagamento (UC01)

O ecrã de pagamento (Figura 4.11) apresenta um *layout* 50/50 onde a metade esquerda contém o resumo da venda (não-editável) e a metade direita contém a seleção de método de pagamento. O cálculo de troco é automático ao introduzir o valor recebido (numerário). O botão de confirmação ocupa a largura total da zona direita e está sempre visível, evitando *clicks* acidentais nos métodos de pagamento.

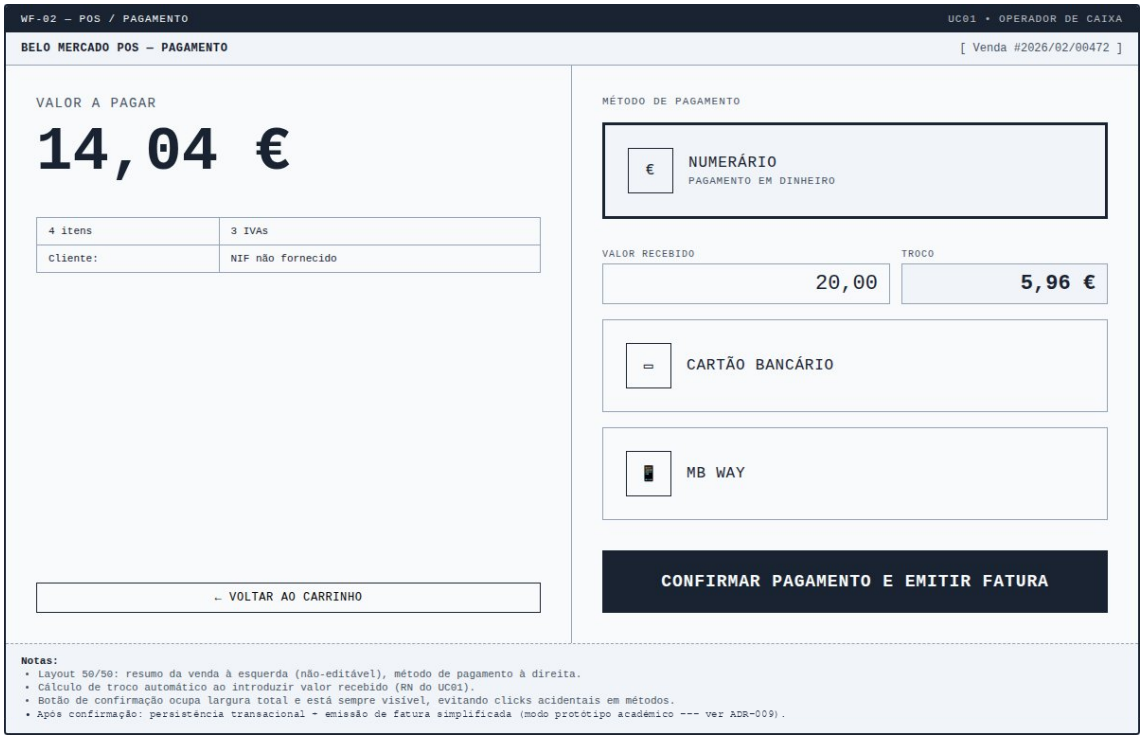


Figura 4.11: WF-02 — POS: ecrã de pagamento (UC01).

4.6.3 POS — Fecho de Caixa (UC04)

O ecrã de fecho de caixa (Figura 4.12) apresenta o resumo do dia à esquerda (gerado automaticamente a partir das vendas registadas) e a contagem física à direita (introduzida pelo operador). A diferença entre valor esperado e valor contado é calculada em tempo real e destacada conforme o sinal. Para diferenças superiores a 10 €, o sistema exige a introdução de uma justificação (RN do UC04). Um aviso explícito informa o operador de que o fecho aciona a consolidação diária (UC12) e bloqueia o POS para novas vendas.

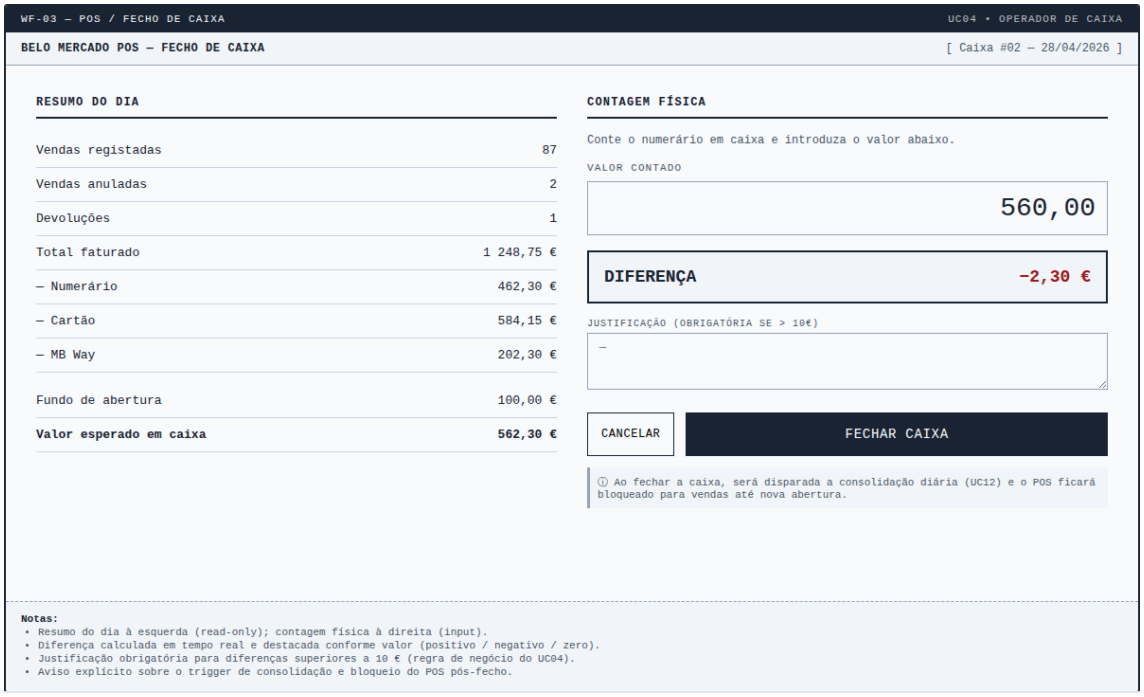


Figura 4.12: WF-03 — POS: fecho de caixa (UC04).

4.6.4 *Backoffice* — *Dashboard* de KPIs (UC13)

O *dashboard* (Figura 4.13) segue a convenção de "5–10 KPIs no topo, gráficos em baixo, comparativo entre lojas em rodapé". Os 4 KPIs de topo (total de vendas, transações, *ticket* médio, alertas de *stock*) apresentam variação percentual face ao período anterior. O gráfico de vendas por dia e o *top* 5 produtos partilham uma linha; o comparativo entre lojas ocupa toda a largura abaixo, com *badges* de alerta de *stock* a usar cores semânticas (verde / amarelo / vermelho). O *timestamp* da última consolidação está visível no topo, garantindo transparência sobre a frescura dos dados.

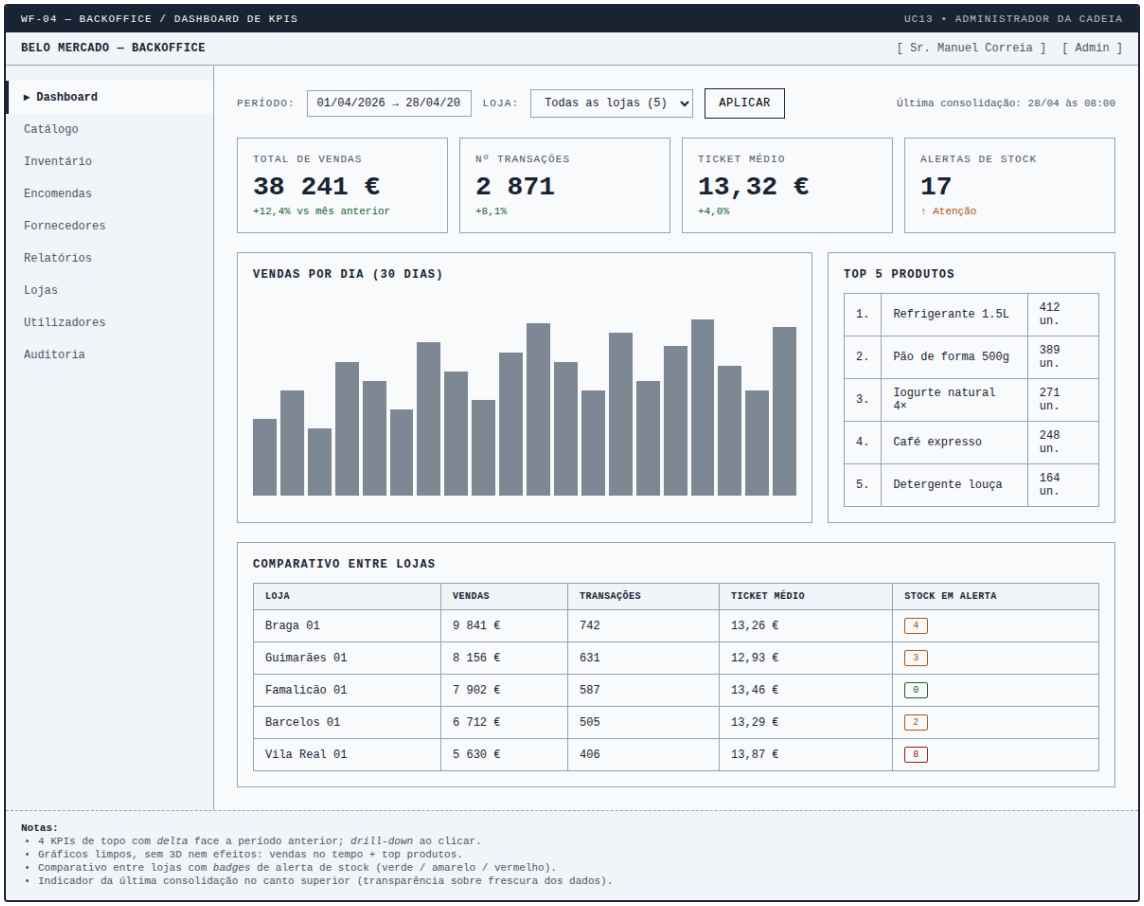


Figura 4.13: WF-04 — *Backoffice*: *dashboard* de KPIs (UC13).

4.6.5 *Backoffice* — Gestão de Catálogo (UC05)

A gestão de catálogo (Figura 4.14) adopta um *layout master-detail* de três colunas: navegação lateral, lista filtrável de produtos e painel de edição lateral. O histórico de preços é apresentado diretamente na ficha do produto, gerado automaticamente a cada alteração. A desativação de um produto é implementada como *soft delete* (RN04 do UC05): o produto fica inativo mas mantém-se acessível para histórico e auditoria.

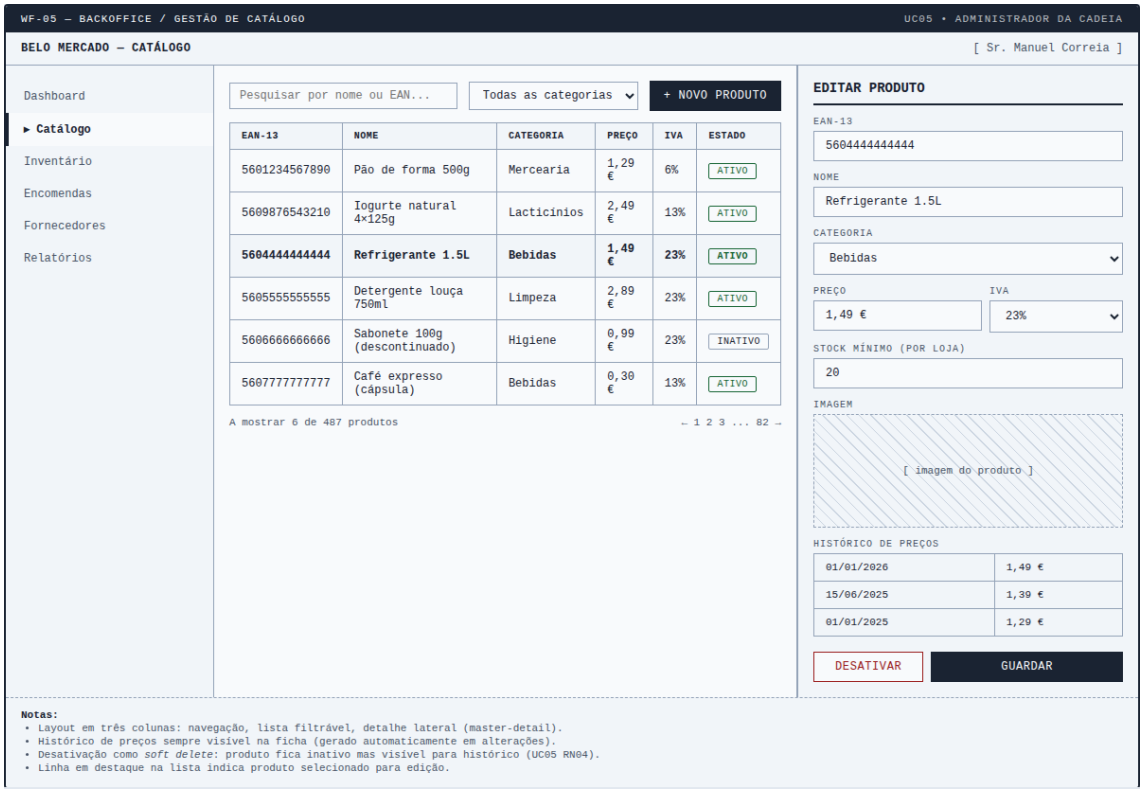


Figura 4.14: WF-05 — *Backoffice*: gestão de catálogo de produtos (UC05).

4.6.6 *Backoffice* — Gestão de Utilizadores (UC16)

A gestão de utilizadores (Figura 4.15) apresenta a lista filtrável por três dimensões críticas para auditoria (perfil, loja, estado). O estado de "Bloqueado"é tratado como uma condição reversível distinta de "Inativo", e as ações disponíveis em cada linha adaptam-se ao estado (um utilizador bloqueado mostra "Desbloquear"em vez de "Editar"). A política de segurança é exposta diretamente no ecrã, tornando explícitas para o administrador as regras que o sistema aplica automaticamente.

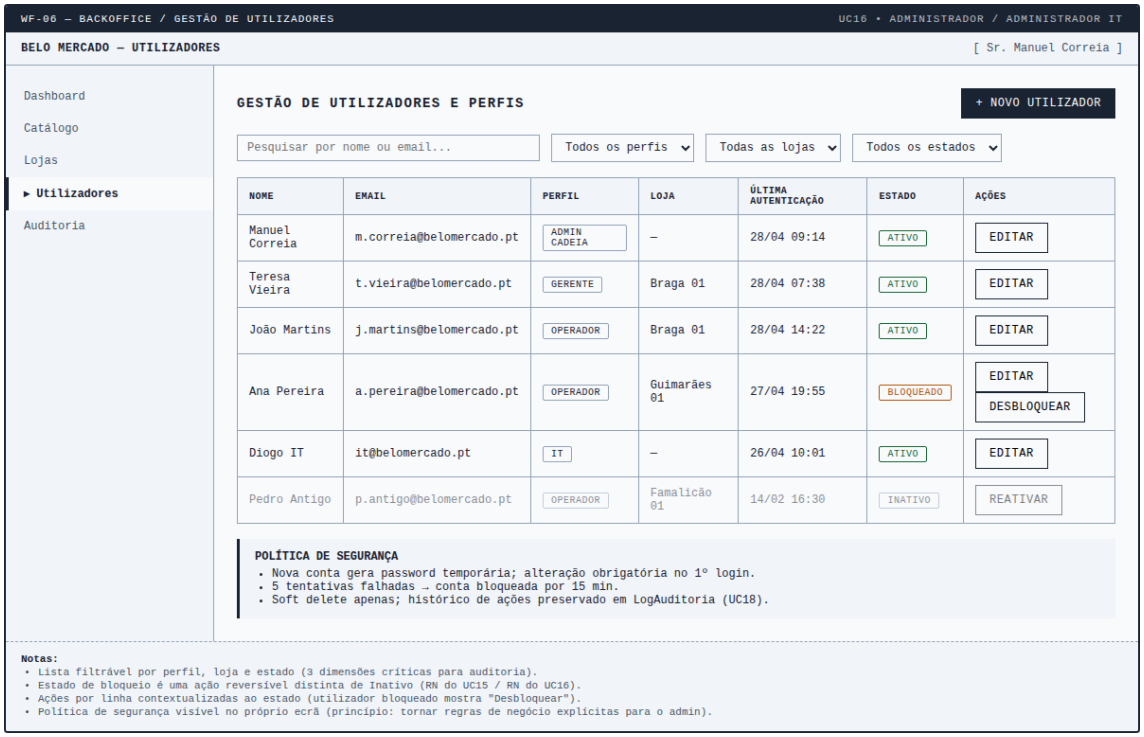


Figura 4.15: WF-06 — *Backoffice*: gestão de utilizadores (UC16).

Interação com o LLM

Prompt utilizado
Atua como *UX Designer*. A partir dos casos de uso UC01, UC04, UC05, UC13, UC16 e dos requisitos não-funcionais relevantes (RNF-01, RNF-11, RNF-12), define a estrutura e o conteúdo dos ecrãs principais do sistema Belo Mercado Manager. Para cada ecrã, identifica: zonas funcionais, elementos críticos, fluxo principal, regras de negócio que devem ser visíveis na UI. Justifica decisões de UX em função do perfil do utilizador (Operador de Caixa = velocidade; Administrador = visão consolidada).

Análise crítica
O LLM produziu propostas estruturalmente sensatas e bem fundamentadas em literatura de UX (referências espontâneas a Nielsen Norman, regras de Fitts, princípios de Tog). Foram, no entanto, necessárias várias correções: a primeira proposta do ecrã de venda apresentava o botão "Finalizar Venda"centrado e do mesmo tamanho que outros botões secundários, o que contraria a hierarquia visual desejada (foi reposicionado e sobredimensionado); o ecrã de fecho de caixa não previa inicialmente o aviso explícito sobre o trigger de consolidação, que foi acrescentado por sugestão da equipa por considerar que utilizadores menos experientes poderiam fechar a caixa sem perceber o impacto. A decisão deliberada de produzir *wireframes* **monocromáticos de baixa fidelidade** foi tomada pela equipa contra a sugestão inicial do LLM (que propunha mockups coloridos com paletas predefinidas), por entender que num documento de planeamento académico o foco deve estar na estrutura e não no estilo visual.

4.7 Matriz de Rastreabilidade

A matriz de rastreabilidade documenta a ligação entre os requisitos identificados na Etapa 1 (RFs e RNFs) e os artefactos arquiteturais produzidos na Etapa 2 (casos de uso, componentes, classes do domínio e *endpoints* de API). Esta rastreabilidade é fundamental para garantir cobertura — nenhum requisito ficou sem suporte arquitetural — e para apoiar futuras alterações, permitindo identificar rapidamente o impacto de uma mudança de requisito nos artefactos a rever.

A Tabela 4.3 apresenta a rastreabilidade dos 19 requisitos funcionais e a Tabela 4.4 a dos 16 requisitos não-funcionais.

RF	Requisito	UC	Componentes	Classes	API
RF01	Registar Venda (POS)	UC01	POSWebUI, SalesService, CatalogService, InventoryService, RelationalPersistence	Venda, LinhaVenda, Fatura, Cliente, Stock, Produto	POST /api/vendas
RF02	Anular Venda e Devoluções	UC02	POSWebUI, SalesService, InventoryService, AuditService	Venda, LinhaVenda, Stock	POST /api/vendas/{id}/anular, POST /api/vendas/{id}/devolucao
RF03	Aplicar Promoções	UC03	BackofficeWebUI, SalesService, CatalogService	Promocao, PromocaoLoja, LinhaVendaPromocao	GET/POST/PUT/DELETE /api/promocoes
RF04	Abrir/Fechar Caixa	UC04	POSWebUI, SalesService, ConsolidationScheduler, AuditService	CaixaDiaria, Venda	POST /api/caixas/abrir, POST /api/caixas/{id}/fechar
RF05	Catálogo de Produtos	UC05	BackofficeWebUI, CatalogService, RelationalPersistence	Produto, Categoria, Preco	GET/POST/PUT/DELETE /api/produtos, /api/categorias
RF06	Monitorizar Stock	UC06	BackofficeWebUI, InventoryService, RelationalPersistence	Stock, Produto, Loja	GET /api/stock, GET /api/stock/alertas
RF07	Inventário Manual	UC07	BackofficeWebUI, InventoryService, AuditService	Stock	POST /api/inventarios
RF08	Sugestões de Reposição	UC08	ReplenishmentScheduler, InventoryService, ProcurementService	Stock, Produto, Fornecedor	GET /api/sugestoes-reposicao
RF09	Gerir Fornecedores	UC09	BackofficeWebUI, ProcurementService, AuditService	Fornecedor	GET/POST/PUT /api/fornecedores
RF10	Encomendas a Fornecedor	UC10	BackofficeWebUI, ProcurementService, InventoryService, AuditService	Encomenda, LinhaEncomenda, Fornecedor	POST /api/encomendas, POST /api/encomendas/{id}/submeter
RF11	Receção de Mercadoria	UC11	BackofficeWebUI, ProcurementService, InventoryService	Encomenda, LinhaEncomenda, Stock	POST /api/encomendas/{id}/rececionar
RF12	Consolidar Dados Diários	UC12	ConsolidationScheduler, SalesService, InventoryService, ReportingService	ConsolidacaoDiaria, KPIDiario, RankingProdutosDiario	(job interno; sem endpoint público)
RF13	*Dashboard* de KPIs	UC13	BackofficeWebUI, ReportingService	KPIDiario, RankingProdutosDiario	GET /api/dashboards/kpi
RF14	Gerar e Exportar Relatórios	UC14	BackofficeWebUI, ReportingService	Relatorio, KPIDiario	GET /api/relatorios, POST /api/relatorios/exportar
RF15	Autenticar no Sistema	UC15	WebUI, AuthAccessService, AuditService	Utilizador, Perfil, LogAuditoria	POST /api/auth/login, /refresh, /logout
RF16	Gerir Utilizadores e Perfis	UC16	BackofficeWebUI, AuthAccessService, AuditService	Utilizador, Perfil, Funcionario, GerenteDeLoja	GET/POST/PUT/DELETE /api/utilizadores
RF17	Editar Perfil Próprio	UC17	WebUI, AuthAccessService	Utilizador	GET/PUT /api/me, PUT /api/me/password
RF18	Logs de Auditoria	UC18	BackofficeWebUI, AuditService	LogAuditoria	GET /api/auditoria/logs
RF19	Gerir Lojas da Cadeia	UC19	BackofficeWebUI, StoreManagementService, AuditService	Cadeia, Loja, Funcionario	GET/POST/PUT /api/lojas

Tabela 4.3: Matriz de rastreabilidade — requisitos funcionais.

RNF	Requisito	Realização Arquitetural
RNF-01	Registo de venda em $\leq 2\,\text{s}$	Otimização do fluxo POSWebUI $\leftrightarrow$ SalesService $\leftrightarrow$ BD; índices em `Stock(loja_id, produto_id)` e `Produto.ean13`; renderização Blazor em LAN.
RNF-02	Consolidação $< 5\,\text{min}$	ConsolidationScheduler com *queries* agregadoras otimizadas; tabelas `KPIDiario` / `RankingProdutosDiario` com índices únicos por loja+data.
RNF-03	Relatórios $< 10\,\text{s}$	Pré-cálculo via `KPIDiario`; ReportingService consulta tabelas materializadas em vez de agregar do raw.
RNF-04	Disponibilidade $\geq 99{,}5\%$ em horário de loja (07h00–23h00)	Topologia centralizada (ADR-001); SLA da infraestrutura central; monitorização contínua durante horário comercial.
RNF-05	Disponibilidade $\geq 99\%$ em janela 24/7	Mesma realização que o RNF-04, com janelas de manutenção planeadas fora do horário de loja; previsão de *backups* (RNF-16).
RNF-06	Integridade transacional ACID	Persistência transacional em `persistirVendaEFatura` (SD-UC01); EF Core 8 + SQL Server transactions; *rollback* automático em falha.
RNF-07	Resiliência da consolidação	ADR-010: retentativas a 30/60/90 min; estado em `ConsolidacaoDiaria`; alerta crítico após 3 falhas.
RNF-08	*Passwords* com bcrypt	AuthAccessService: `verificarHash` (SD-UC15); `Utilizador.password_hash` VARCHAR(255).
RNF-09	Comunicações HTTPS/TLS	Configuração ASP.NET Core; HSTS; TLS 1.2+ obrigatório no *deployment*.
RNF-10	Conformidade RGPD	AuditService transversal (ADR-007); `LogAuditoria` com retenção configurável; `Cliente.nif` com tratamento como dado pessoal.
RNF-11	POS operável $< 1\,\text{h}$ formação	Wireframes com alvos $\geq 60\,\text{px}$; foco automático para *scanner*; cores semânticas constantes; teclas rápidas para itens sem código de barras.
RNF-12	Interface responsiva (*desktop*/*tablet*)	Blazor Server com *breakpoints* CSS; *layouts* de grelha adaptativos.
RNF-13	Suporte a 20 lojas	Topologia centralizada com *database connection pooling*; índices em `loja_id` em todas as tabelas operacionais.
RNF-14	Padrões .NET, comentários XML	Convenção de equipa; *linting* via .NET analyzers.
RNF-15	Compatibilidade Chrome/Firefox/Edge	Blazor Server testado em *evergreens*; sem dependências de APIs experimentais.
RNF-16	*Backups* automáticos diários	Política de *backups* no SGBD (SQL Server *Backup* agendado via SQL Server Agent ou *Maintenance Plans*).

Tabela 4.4: Matriz de rastreabilidade — requisitos não-funcionais.

Interação com o LLM

Prompt utilizado

Atua como Engenheiro de *Software* sénior. A partir dos requisitos funcionais (RF01–RF19), dos requisitos não-funcionais (RNF-01 a RNF-16), dos casos de uso, dos componentes definidos no diagrama de componentes, das classes do diagrama de classes e dos *endpoints* REST especificados, produz uma matriz de rastreabilidade bidirecional. Para cada RF, identifica explicitamente os componentes, classes e *endpoints* que o realizam. Para cada RNF, descreve em texto a forma como é endereçado pela arquitetura (não basta listar componentes, é preciso explicar a realização).

Análise crítica

A matriz produzida pelo LLM mostrou-se útil como ponto de partida, mas revelou três tipos de problemas comuns em saídas geradas por LLMs nesta tarefa: **(1) sobreposições incorretas**, com componentes a serem listados como realizadores de requisitos a que não pertencem; **(2) omissões**, com requisitos não-funcionais como o RNF-09 (HTTPS/TLS) que não tinham realização explícita identificada; **(3) descrições circulares**, em que a "realização arquitetural"se limitava a repetir o requisito por outras palavras. A revisão humana foi essencial em todas as linhas. A força do LLM foi assegurar cobertura sistemática (nenhum RF/RNF foi omitido); a fraqueza foi a profundidade analítica, que exigiu validação caso a caso.

5 Implementação e Desenvolvimento por LLM

6 Verificação, Validação e Avaliação da Qualidade do Software Produzido

7 Conclusões e Trabalho Futuro

7.1 Conclusões

Esta etapa permitiu consolidar uma base de requisitos consistente para o sistema **Belo Mercado Manager**, alinhada com o contexto do caso de estudo e com os objetivos da unidade curricular. Foram identificados e estruturados os principais elementos da engenharia de requisitos: domínio do sistema, *stakeholders*, *user stories*, requisitos funcionais e não funcionais, análise de ambiguidades, especificação SRS e modelação de casos de uso, com rastreabilidade entre artefactos.

Em termos de resultados, a equipa definiu 19 requisitos funcionais, 16 requisitos não funcionais, 29 *user stories* e 19 casos de uso, mantendo correspondência entre necessidades de negócio e especificação técnica. As entrevistas simuladas e respetivos resumos contribuíram para clarificar problemas operacionais reais e para fundamentar decisões de modelação e priorização.

O uso de LLM teve impacto positivo na produtividade e na estruturação inicial dos conteúdos, sobretudo na geração de propostas, reformulação textual e cobertura de cenários. Ainda assim, a experiência confirmou a necessidade de revisão crítica contínua por parte da equipa, uma vez que parte das respostas exigiu validação, ajuste de escopo e reformulação para garantir o rigor, a mensurabilidade e a coerência documental.

Como principal limitação desta etapa, destaca-se o facto de se tratar de uma fase predominantemente de análise e especificação, sem validação empírica através de uma implementação completa. Ainda assim, o grau de confiança nos requisitos definidos é elevado para suportar a etapa seguinte, na medida em que os artefactos produzidos apresentam consistência interna, alinhamento normativo (IEEE 830/29148) e uma ligação explícita ao contexto de negócio.

7.2 Trabalho Futuro

Na etapa seguinte, o foco será colocado na arquitetura e *design* do sistema, transformando os requisitos definidos em decisões técnicas de implementação. Esta fase incluirá a definição da arquitetura lógica e física da solução, modelação da base de dados, desenho dos principais componentes da aplicação e especificação das interfaces entre módulos.

Serão também aprofundados os aspetos de segurança, controlo de acessos, estratégia de con-

solidação diária e requisitos de desempenho, garantindo que as opções de *design* suportam os RNF já estabelecidos. Em paralelo, serão produzidos artefactos de apoio à implementação, como diagramas de arquitetura, modelos de dados detalhados e protótipos das vistas principais.

No plano de execução global, manter-se-á a abordagem incremental definida no cronograma (Anexo C), com validações intermédias por marco e atualização contínua da documentação técnica. A etapa de implementação e posterior verificação/validação irá utilizar esta base como referência, assegurando continuidade entre requisitos, arquitetura e produto final.

Bibliografia

[1] Sommerville, I. (2016). *Software Engineering*. 10.ª ed. Harlow: Pearson Education.

[2] IEEE (1998). *IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications*. New York: Institute of Electrical and Electronics Engineers.

[3] ISO/IEC/IEEE (2018). *ISO/IEC/IEEE 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering*. Geneva: ISO.

[4] Cockburn, A. (2001). *Writing Effective Use Cases*. Boston: Addison-Wesley.

[5] Regulamento (UE) 2016/679 do Parlamento Europeu e do Conselho, de 27 de abril de 2016, relativo à proteção das pessoas singulares no que diz respeito ao tratamento de dados pessoais (RGPD). *Jornal Oficial da União Europeia*, L 119, 1–88.

[6] Microsoft (2024). *ASP.NET Core Blazor*. Disponível em: `https://learn.microsoft.com/en-us/aspnet/core/blazor/` [Acedido em: fevereiro de 2026].

Lista de Siglas e Acrónimos

ACID *Atomicity, Consistency, Isolation, Durability*

ADR *Architecture Decision Record*

API *Application Programming Interface*

BD Base de Dados

CSV *Comma-Separated Values*

EAN *European Article Number*

ETL *Extract, Transform, Load*

HTTPS *HyperText Transfer Protocol Secure*

IVA Imposto sobre o Valor Acrescentado

JWT *JSON Web Token*

KPI *Key Performance Indicator*

LEI Licenciatura em Engenharia Informática

LI4 Laboratórios de Informática IV

LLM *Large Language Model*

NIF Número de Identificação Fiscal

PDF *Portable Document Format*

POS *Point of Sale*

RGPD Regulamento Geral sobre a Proteção de Dados

SRS *Software Requirements Specification*

TLS *Transport Layer Security*

UC Caso de Uso (*Use Case*)

UI *User Interface*

UML *Unified Modeling Language*

UX *User Experience*

Anexos

Anexo A — Transcrição das Entrevistas Simuladas

As entrevistas simuladas com os *stakeholders* do sistema Belo Mercado Manager foram conduzidas com recurso a *Large Language Models* (LLM), atuando como apoio ao processo de Engenharia de Requisitos. Neste anexo são apresentadas as transcrições integrais das cinco entrevistas realizadas.

Entrevista 1 — Administrador da Cadeia (Sr. Manuel Correia)

Data: 15 de fevereiro de 2026
Local: Escritório central da Belo Mercado, Braga
Entrevistador: Analista da equipa de desenvolvimento
Entrevistado: Sr. Manuel Correia — Proprietário e administrador da cadeia Belo Mercado
Perfil: 55 anos, formação em Gestão de Empresas, fundou a Belo Mercado há 8 anos

Analista: Bom dia, Sr. Manuel. Muito obrigado pela sua disponibilidade. Como sabe, estamos a desenvolver um novo sistema de gestão para a sua cadeia de lojas. Gostaríamos de começar por compreender melhor a organização atual da Belo Mercado. Pode descrever-nos como está estruturada a cadeia?

Sr. Manuel: Bom dia. Claro. A Belo Mercado tem neste momento cinco lojas, todas na zona norte. Comecei com uma loja em Braga há oito anos e fui expandindo. Cada loja tem um gerente responsável e dois a três operadores de caixa, dependendo do movimento. Eu supervisiono tudo a partir deste escritório em Braga, mas na prática passo muito tempo em deslocações entre lojas.

Analista: E como funciona a gestão no dia a dia? Como é que acompanha o desempenho de cada loja?

Sr. Manuel: Confesso que esse é o meu maior problema. Cada gerente gere a sua loja de forma relativamente autónoma. No final de cada mês, peço-lhes que me enviem um resumo das vendas. Alguns enviam em Excel, outros escrevem num email, e por vezes tenho de ir à loja pessoalmente para tirar informação da caixa registadora. Não consigo ter uma visão global do negócio de forma simples e atempada.

Analista: Consegue identificar os maiores desafios ou frustrações com esta situação?

Sr. Manuel: Vários. Primeiro, não sei em tempo útil como está cada loja. Se uma loja está a vender mal um mês inteiro, só descubro no final desse mês. Segundo, a gestão de stock é caótica — já aconteceu termos um produto esgotado numa loja e excesso de stock dessa mesma referência noutra, sem que eu soubesse. Terceiro, com os fornecedores, cada gerente negoceia individualmente, e eu perco poder negocial porque não consigo agregar as encomendas.

Analista: E em relação à faturação e aspetos financeiros?

Sr. Manuel: A contabilidade é feita por um contabilista externo. Ele precisa dos dados de faturação de todas as lojas, e compilar isso mensalmente é um pesadelo. Há meses em que os dados chegam incompletos. Preciso que o sistema gere faturas de forma automática e que eu consiga exportar os dados financeiros facilmente.

Analista: Que tipo de informação gostaria de ter sempre disponível?

Sr. Manuel: Quero um painel — um dashboard — onde abro o computador de manhã e vejo: quanto vendeu cada loja ontem, quais os produtos mais vendidos, quais os que estão parados, qual a margem de lucro por loja e por categoria. Quero alertas quando o stock de qualquer produto em qualquer loja descer abaixo de um nível mínimo. Quero comparar lojas entre si. E quero relatórios que possa exportar para enviar ao contabilista.

Analista: E no que respeita a segurança e controlo de acessos?

Sr. Manuel: Sim, isso é importante. Eu devo ver tudo — todas as lojas, todos os dados financeiros. Cada gerente deve ver apenas a informação da sua loja. Os operadores de caixa devem só poder registar vendas e nada mais. E o contabilista provavelmente precisa de acesso a relatórios financeiros, mas não aos dados operacionais.

Analista: Por fim, há alguma funcionalidade específica que considere fundamental e que não tenhamos mencionado?

Sr. Manuel: Sim. Quero que o sistema me permita perceber quais os produtos que devo descontinuar — aqueles que vendem pouco mas ocupam espaço — e quais os que devo apostar mais. Também gostaria de, no futuro, poder dar um programa de fidelização a clientes, mas isso pode ser para uma fase posterior.

Entrevista 2 — Gerente de Loja (D. Teresa Vieira)

Data: 16 de fevereiro de 2026
Local: Loja Belo Mercado Braga
Entrevistador: Analista da equipa de desenvolvimento
Entrevistada: D. Teresa Vieira — Gerente da loja Belo Mercado Braga

Perfil: 38 anos, 12 anos de experiência em retalho, responsável pela loja há 4 anos

Analista: Bom dia, D. Teresa. Obrigado por nos receber. Pode começar por nos descrever o seu dia típico na loja?

D. Teresa: Bom dia. Claro. Chego à loja por volta das 7h30, meia hora antes da abertura. A primeira coisa que faço é uma ronda pelas prateleiras — vejo o que está em falta, o que precisa de ser reposto do armazém e o que está a chegar ao prazo de validade. Depois abro a caixa, verifico o fundo de caixa e dou início ao dia.

Analista: E ao longo do dia?

D. Teresa: Ao longo do dia divido-me entre o atendimento (quando há mais movimento), a gestão de encomendas a fornecedores e a organização da loja. Quando chega uma encomenda de um fornecedor, confiro o que veio, verifico se bate com o que pedi e arrumo. Ao final do dia, fecho a caixa, faço a contagem do dinheiro em caixa, e preparo um resumo para enviar ao Sr. Manuel.

Analista: Como gere atualmente o stock?

D. Teresa: Tenho uma folha de Excel onde vou atualizando, mas confesso que nem sempre está em dia. Quando chega uma encomenda, anoto. Quando noto prateleiras vazias, vou ao Excel ver o que supostamente tenho em armazém. Mas o Excel não se atualiza sozinho com as vendas, por isso há sempre desfasamento. Algumas vezes mandei encomendar coisas que afinal já tinha, e outras vezes fiquei sem stock sem me aperceber a tempo.

Analista: E como funciona o processo de encomenda aos fornecedores?

D. Teresa: Normalmente faço por email ou por telefone. Tenho os contactos dos fornecedores habituais. Vejo o que está a faltar, preparo a lista e envio a encomenda. O problema é que nem sempre me lembro de tudo, e às vezes esqueço-me de encomendar algo que já está em falta há dias. Se o sistema me sugerisse automaticamente o que encomendar, seria uma grande ajuda.

Analista: E em relação ao fecho de caixa e à consolidação de dados?

D. Teresa: O fecho de caixa é feito manualmente. Conto o dinheiro, comparo com o que a caixa registadora diz que deveria estar lá, e anoto a diferença. Depois envio um resumo ao Sr. Manuel, normalmente por WhatsApp ou email. Se isso pudesse ser automático — fecho a caixa no sistema e ele envia tudo — seria perfeito.

Analista: Que funcionalidades gostaria de ter num novo sistema?

D. Teresa: Primeiro, quero saber em tempo real o que tenho em stock. Segundo, quero alertas automáticos — "atenção, o leite está abaixo do mínimo". Terceiro, quero poder fazer encomendas ao fornecedor diretamente no sistema. Quarto, quero ver um resumo diário de vendas sem ter de fazer contas à mão. E quinto, que o envio de dados para o central seja

automático, para eu não ter esse trabalho extra.

Analista: Há mais alguma preocupação?

D. Teresa: Sim. A interface tem de ser simples. Os meus operadores de caixa são miúdos jovens, estudantes, que estão ali umas horas por dia. Não posso andar a dar-lhes formação complicada. O sistema de venda tem de ser tão simples quanto possível: bipa o produto, aparece o preço, no fim clica "pagar" e sai a fatura.

Entrevista 3 — Operador de Caixa (João Martins)

Data: 16 de fevereiro de 2026
Local: Loja Belo Mercado Guimarães
Entrevistador: Analista da equipa de desenvolvimento
Entrevistado: João Martins — Operador de caixa
Perfil: 22 anos, estudante de gestão, trabalha a tempo parcial (turnos de 4 horas)

Analista: Olá, João. Pode descrever-me como funciona o processo de venda atualmente?

João: Olá. É relativamente simples. O cliente chega com os produtos à caixa. Eu passo os artigos pelo leitor de código de barras, um a um. A caixa registadora vai somando. Quando acabo, digo o total ao cliente, ele paga — normalmente em dinheiro ou multibanco — e eu dou-lhe o troco e o talão.

Analista: E quando algo não corre bem?

João: O que acontece mais frequentemente é o leitor não conseguir ler o código de barras. Pode ser porque a etiqueta está danificada ou porque é um produto novo que ainda não está no sistema. Quando isso acontece, tenho de procurar o preço manualmente — às vezes na etiqueta da prateleira, às vezes pergunto à D. Teresa. Isso atrasa tudo e os clientes ficam impacientes, especialmente se está fila.

Analista: E quando um cliente pede fatura com NIF?

João: Isso é uma dor de cabeça. Com o sistema atual, tenho de escrever à mão o NIF num formulário e depois a D. Teresa trata disso. Demora e é propenso a erros. Seria muito melhor se eu pudesse simplesmente digitar o NIF no sistema e a fatura sair automaticamente.

Analista: E em relação a promoções e descontos?

João: Quando há promoções, tenho de me lembrar quais são os produtos em promoção e aplicar o desconto manualmente. Às vezes esqueço-me e o cliente reclama. Se o sistema aplicasse automaticamente, seria muito mais fácil.

Analista: Como funciona a abertura e o fecho de caixa?

João: Logo do início do turno, conto o dinheiro que está na caixa (o fundo de caixa, que normalmente é 100€) e anoto num papel. No final do turno, conto novamente e deixo o papel para a gerente conferir. Às vezes há diferenças de um ou dois euros — normalmente por erros no troco.

Analista: O que gostaria que melhorasse num novo sistema?

João: Rapidez, acima de tudo. Não quero perder tempo a procurar produtos quando o leitor falha — quero poder pesquisar pelo nome. Quero que o NIF seja inserido no sistema e a fatura saia sozinha. Quero que os descontos sejam automáticos. E quero que o sistema seja fácil — não quero ter de decorar 50 botões ou menus.

Entrevista 4 — Fornecedor (Distribuição Almeida, Lda.)

Data: 17 de fevereiro de 2026
Local: Entrevista por videochamada
Entrevistador: Analista da equipa de desenvolvimento
Entrevistado: Sr. António Almeida — Diretor Comercial, Distribuição Almeida, Lda.
Perfil: Fornecedor de produtos de mercearia e bebidas, fornece à Belo Mercado há 5 anos

Analista: Sr. António, pode descrever-nos como funciona atualmente o processo de encomendas com a Belo Mercado?

Sr. António: Recebemos encomendas de cada loja individualmente, normalmente por email ou telefone. Cada gerente manda a sua lista quando precisa. Isso cria-nos alguns problemas: primeiro, recebemos encomendas pequenas espalhadas ao longo da semana, quando poderiam ser agrupadas numa encomenda maior. Segundo, nem sempre as encomendas chegam por escrito — às vezes é uma chamada telefónica e há risco de mal-entendidos na quantidade ou no produto.

Analista: E como funciona a entrega e a confirmação?

Sr. António: Fazemos a entrega na loja e o gerente confere a mercadoria. Normalmente assina a guia de entrega. Mas depois, se houver algum problema — um produto que veio a mais, a menos ou danificado — a comunicação é informal e por vezes demorada. Não temos um sistema centralizado para gerir isso.

Analista: O que o ajudaria a trabalhar melhor com a Belo Mercado?

Sr. António: Se houvesse um sistema onde pudéssemos receber as encomendas de forma padronizada — uma lista formal com códigos de produto, quantidades e preços acordados. Idealmente, eu poderia confirmar a encomenda no sistema, registar a data de entrega prevista e, quando entregar, o gerente confirma a receção. Assim fica tudo registado e não há mal-entendidos.

Analista: E quanto a preços e condições?

Sr. António: Os preços e condições são negociados anualmente com o Sr. Manuel, mas depois precisam de estar acessíveis no sistema para que as encomendas saiam com os valores corretos. Se eu atualizar um preço, gostaria que o sistema refletisse essa alteração. E se houver descontos de volume — por exemplo, mais de 100 unidades com 5% de desconto — isso deveria ser aplicado automaticamente.

Entrevista 5 — Contabilista (Dr. Carlos Ferreira)

Data: 18 de fevereiro de 2026
Local: Escritório do contabilista
Entrevistador: Analista da equipa de desenvolvimento
Entrevistado: Dr. Carlos Ferreira — Contabilista responsável pela contabilidade da Belo Mercado
Perfil: 48 anos, 20 anos de experiência em contabilidade de PMEs

Analista: Dr. Carlos, como é que atualmente recebe os dados financeiros da Belo Mercado?

Dr. Carlos: É um processo bastante penoso, devo dizer. No final de cada mês, o Sr. Manuel compila os dados que recebe dos gerentes — faturas, recibos, despesas — e envia-me tudo. Muitas vezes faltam documentos, ou os valores não batem certo com os talões. Perco muito tempo a reconciliar.

Analista: Que tipo de dados precisa especificamente?

Dr. Carlos: Preciso de todas as faturas e notas de crédito emitidas, organizadas por loja e por mês. Preciso dos dados de IVA — quanto foi cobrado em cada taxa (6%, 13%, 23%). Preciso dos dados de compras a fornecedores, com as respetivas faturas. E preciso de tudo isto em formato que eu possa importar para o meu software de contabilidade, idealmente CSV ou integração direta.

Analista: Há requisitos legais que devamos ter em conta?

Dr. Carlos: Sim, vários. As faturas devem cumprir os requisitos da Autoridade Tributária: número sequencial, NIF da empresa, data, descrição dos bens, valores com IVA discriminado. O software de faturação tem de ser certificado pela AT, se for usado para emitir faturas ao consumidor final. E os dados têm de ser guardados durante pelo menos 10 anos.

Analista: Algo mais que o sistema deveria oferecer-lhe?

Dr. Carlos: Um relatório mensal automático com o total de vendas por taxa de IVA, o total de compras a fornecedores e as margens por categoria. Se eu pudesse descarregar esse relatório em PDF no início de cada mês, poupava-me horas de trabalho.

Anexo B — Casos de Uso Detalhados (UC02–UC19)

UC02 — Anular Venda e Processar Devolução

ID / Nome	UC02 — Anular Venda e Processar Devolução
Ator Principal	Operador de Caixa / Gerente de Loja
Descrição	Anular uma venda finalizada ou processar devolução, atualizando *stock* e gerando nota de crédito.
Pré-condições	1) Utilizador autenticado (Operador/Gerente). 2) Venda com estado "Concluída". 3) Caixa aberta.
Pós-condições	1) Venda marcada "Anulada" ou devolução registada. 2) *Stock* incrementado. 3) Nota de crédito emitida. 4) Log de auditoria.
Fluxo Principal	1. Selecionar "Anulação/Devolução". 2. Pesquisar venda (nº fatura/data). 3. Selecionar venda. 4. Escolher: anulação total ou devolução parcial. 5. Preencher motivo. 6. Confirmar $\rightarrow$ transação atómica: registo devolução, atualização *stock*, nota de crédito, *log*.
Fluxos Alt.	**7a.** Venda $> 24\,\text{h} \rightarrow$ requer autorização do Gerente. **9a.** Reimprimir nota de crédito.
Exceções	E1: Venda não encontrada. E2: Qtd devolvida $>$ vendida. E3: Venda já anulada. E4: Falha BD (*rollback*).
Regras de Negócio	RN01: $> 24\,\text{h}$ requer Gerente. RN02: > 30 dias não permitido. RN03: Notas crédito sequenciais (AT).
RF Associados	RF02, RNF-06

Tabela 7.1: Caso de uso UC02 — Anular Venda e Processar Devolução.

UC03 — Aplicar Promoções e Descontos

ID / Nome	UC03 — Aplicar Promoções e Descontos
Ator Principal	Administrador / Gerente de Loja
Descrição	Criar, editar e gerir promoções (percentagem, valor fixo, leve-N-pague-M) aplicadas automaticamente no POS.
Pré-condições	1) Utilizador autenticado (Admin/Gerente). 2) Produtos ativos no catálogo.
Pós-condições	1) Promoção criada/atualizada na BD. 2) Associações produto-promoção registadas. 3) Desconto automático em vendas futuras.
Fluxo Principal	1. Aceder módulo "Promoções". 2. "Nova Promoção". 3. Preencher: descrição, tipo, valor, datas, abrangência (produto/categoria/global), lojas. 4. Guardar $\rightarrow$ validação e persistência.
Fluxos Alt.	**3a.** Editar promoção ativa/inativa. **3b.** Desativar promoção. **8a.** Conflito $\rightarrow$ aplica-se o maior desconto.
Exceções	E1: Data início no passado. E2: Data fim $<$ início. E3: Percentagem fora de intervalo. E4: Sem produto selecionado.
Regras de Negócio	RN01: Múltiplas promoções $\rightarrow$ maior desconto. RN02: Expiradas marcadas automaticamente. RN03: Gerente só na sua loja. RN04: Preço final ≥ 0.
RF Associados	RF03, RNF-01

Tabela 7.2: Caso de uso UC03 — Aplicar Promoções e Descontos.

UC04 — Abrir e Fechar Caixa

ID / Nome	UC04 — Abrir e Fechar Caixa
Ator Principal	Operador de Caixa
Descrição	Registar abertura e fecho de caixa diários, contagem de numerário e preparação para consolidação.
Pré-condições	1) Operador/Gerente autenticado. 2) (Abertura) Sem caixa aberta hoje. 3) (Fecho) Caixa aberta.
Pós-condições	1) (Abertura) Registo `CaixaDiaria` com fundo de caixa. 2) (Fecho) Valor contado, diferença, estado "Fechada", flag consolidação.
Fluxo Principal	**Abertura:** 1. "Abrir Caixa". 2. Introduzir fundo de caixa (€). 3. POS desbloqueado. **Fecho:** 1. "Fechar Caixa". 2. Sistema calcula valor esperado. 3. Resumo do dia. 4. Introduzir valor contado. 5. Diferença calculada (confere/excedente/défice). 6. Confirmar → POS bloqueado, dados prontos para consolidação.
Fluxos Alt.	**2a.** Caixa já aberta → impede. **2b.** Vendas pendentes → finalizar primeiro. **6a.** Diferença $> 10\,€$ → justificação obrigatória.
Exceções	E1: Valor inválido. E2: Falha de persistência.
Regras de Negócio	RN01: 1 caixa/loja/dia. RN02: Sem vendas com caixa fechada. RN03: Fecho aciona consolidação (UC12).
RF Associados	RF04, RF12, RNF-06

Tabela 7.3: Caso de uso UC04 — Abrir e Fechar Caixa.

UC05 — Gerir Catálogo de Produtos

ID / Nome	UC05 — Gerir Catálogo de Produtos
Ator Principal	Administrador da Cadeia
Descrição	Manter o catálogo centralizado: adição, edição e desativação de produtos disponíveis para todas as lojas.
Pré-condições	1) Administrador autenticado.
Pós-condições	1) Produto criado/atualizado/desativado. 2) Disponível em todas as lojas. 3) Alterações de preço registadas em `HistoricoPrecos`.
Fluxo Principal	1. Aceder "Catálogo de Produtos". 2. "Novo Produto". 3. Preencher: EAN-13, descrição, categoria/subcategoria, preço, IVA (6/13/23%), unidade de medida, imagem (opcional). 4. Guardar → validação (EAN único, dígito controlo) e persistência.
Fluxos Alt.	**3a.** Editar produto (histórico de preços automático). **3b.** Desativar (aviso se *stock* > 0). **3c.** Reativar.
Exceções	E1: EAN duplicado. E2: EAN inválido. E3: Preço inválido. E4: Imagem inválida.
Regras de Negócio	RN01: Catálogo centralizado (todas as lojas). RN02: Inativos não aparecem no POS. RN03: Histórico de preços obrigatório. RN04: *Soft delete* apenas.
RF Associados	RF05

Tabela 7.4: Caso de uso UC05 — Gerir Catálogo de Produtos.

UC06 — Consultar e Monitorizar *Stock*

ID / Nome	UC06 — Consultar e Monitorizar *Stock*
Ator Principal	Gerente de Loja
Descrição	Consultar níveis de *stock* por produto, alertas de *stock* mínimo e histórico de movimentos.
Pré-condições	1) Gerente autenticado. 2) Loja com produtos em `StockLoja`.
Pós-condições	1) Informação de *stock* apresentada. 2) Alertas marcados como "Visualizados".
Fluxo Principal	1. Módulo "Inventário". 2. Tabela paginada: código, descrição, categoria, qtd atual, mín., máx., estado (cores). 3. Filtros: categoria, estado, pesquisa. 4. Separador "Alertas": produtos abaixo do mínimo. 5. "Criar Encomenda" → UC10 pré-preenchido.
Fluxos Alt.	**7a.** Contagem manual → UC07. **7b.** Histórico de movimentos de um produto.
Exceções	E1: Sem produtos para os filtros.
Regras de Negócio	RN01: Mín./máx. definidos por produto/loja. RN02: Alertas em tempo real.
RF Associados	RF06, RNF-01, RNF-03

Tabela 7.5: Caso de uso UC06 — Consultar e Monitorizar *Stock*.

UC07 — Realizar Inventário Manual

ID / Nome	UC07 — Realizar Inventário Manual
Ator Principal	Gerente de Loja
Descrição	Contagem física de inventário, comparação com sistema, identificação de discrepâncias e ajuste de *stock*.
Pré-condições	1) Gerente autenticado. 2) Caixa fechada. 3) Produtos em `StockLoja`.
Pós-condições	1) Contagem registada. 2) *Stock* ajustado (se aceite). 3) Movimentos registados. 4) Discrepâncias $> 10\%$ geram alerta ao Admin.
Fluxo Principal	1. "Nova Contagem". 2. Verificar caixa fechada. 3. Lista de produtos com qtd sistema (readonly) e qtd contada (input). 4. "Finalizar Contagem". 5. Relatório de discrepâncias (excesso/quebra). 6. Aceitar ajuste ou recontar.
Fluxos Alt.	**5a.** Contagem parcial (guardar progresso). **6a.** Contagem por categoria.
Exceções	E1: Caixa aberta $\rightarrow$ impede. E2: Sessão expirada $\rightarrow$ retomar. E3: Valor inválido.
Regras de Negócio	RN01: Caixa deve estar fechada. RN02: Discrepâncias $> 10\%$ alertam Admin. RN03: Histórico de contagens mantido.
RF Associados	RF07, RNF-06

Tabela 7.6: Caso de uso UC07 — Realizar Inventário Manual.

UC08 — Gerar Sugestões de Reposição

ID / Nome	UC08 — Gerar Sugestões de Reposição
Ator Principal	Sistema (automático)
Descrição	Análise de *stock*, vendas e prazos de entrega para gerar sugestões automáticas de reposição.
Pré-condições	1) Produtos com mín./máx. definidos. 2) Histórico de vendas (≥ 7 dias). 3) Fornecedores ativos com produtos.
Pós-condições	1) Sugestões criadas em `SugestoesReposicao`. 2) Notificação ao Gerente.
Fluxo Principal	1. Execução diária (06h00) ou manual. 2. Para cada produto/loja: calcular velocidade de vendas (30 dias), dias de *stock*, ponto de encomenda. 3. Se qtd $\leq$ ponto de encomenda $\rightarrow$ gerar sugestão. 4. Prioridade: Alta (sem *stock*), Média (abaixo mín.), Baixa (outros). 5. Gerente aprova/rejeita/ajusta $\rightarrow$ cria encomenda (UC10).
Fluxos Alt.	**1a.** Geração manual pelo Gerente. **8a.** Aprovação em lote (agrupa por fornecedor).
Exceções	E1: Sem dados históricos suficientes. E2: Produto sem fornecedor.
Regras de Negócio	RN01: Margem segurança = 2 dias (configurável). RN02: Apenas produtos ativos com mín. definido. RN03: Pendentes $>$ 7 dias expiram.
RF Associados	RF08, RNF-01

Tabela 7.7: Caso de uso UC08 — Gerar Sugestões de Reposição.

UC09 — Gerir Fornecedores

ID / Nome	UC09 — Gerir Fornecedores
Ator Principal	Administrador da Cadeia
Descrição	Registar, editar e desativar fornecedores; associar produtos com preços de custo, descontos e prazos.
Pré-condições	1) Administrador autenticado.
Pós-condições	1) Fornecedor criado/atualizado/desativado. 2) Associações fornecedor-produto registadas.
Fluxo Principal	1. Módulo "Fornecedores". 2. "Novo Fornecedor". 3. Preencher: nome, NIF (mod 11), morada, contacto, email, ramo, condições de pagamento. 4. Guardar → validação. 5. Associar produtos: preço de custo, desconto (%), prazo de entrega (dias).
Fluxos Alt.	**3a.** Editar (NIF não editável). **3b.** Desativar (impede se encomendas pendentes). **9a.** Atualizar preço custo (histórico).
Exceções	E1: NIF duplicado. E2: NIF inválido. E3: Produto já associado.
Regras de Negócio	RN01: Inativos sem novas encomendas. RN02: Produto pode ter múltiplos fornecedores. RN03: *Soft delete*.
RF Associados	RF09

Tabela 7.8: Caso de uso UC09 — Gerir Fornecedores.

UC10 — Criar Encomenda a Fornecedor

ID / Nome	UC10 — Criar Encomenda a Fornecedor
Ator Principal	Gerente de Loja
Descrição	Criar encomenda de reposição a um fornecedor, com base nas necessidades de *stock*.
Pré-condições	1) Gerente/Administrador autenticado. 2) Fornecedor ativo com produtos associados.
Pós-condições	1) Encomenda registada (Rascunho ou Enviada). 2) Linhas com produto, qtd, preço, desconto. 3) Notificação ao fornecedor (se enviada).
Fluxo Principal	1. "Nova Encomenda". 2. Selecionar fornecedor. 3. Catálogo do fornecedor com *stock* atual e qtd sugerida. 4. Selecionar produtos e quantidades. 5. Total calculado. 6. "Guardar Rascunho" ou "Enviar Encomenda".
Fluxos Alt.	**1a.** Via alerta *stock* (UC06, pré-preenchido). **1b.** Via sugestão reposição (UC08). **5a.** Encomenda centralizada (Admin, múltiplas lojas). **8a.** Editar rascunho.
Exceções	E1: Qtd inválida. E2: Sem produto selecionado. E3: Fornecedor sem catálogo.
Regras de Negócio	RN01: Preços da tabela `FornecedorProduto`. RN02: $> 5\,000\,€$ requer aprovação Admin. RN03: Qtd sugerida = máx − atual.
RF Associados	RF06, RF08, RF09, RF10

Tabela 7.9: Caso de uso UC10 — Criar Encomenda a Fornecedor.

UC11 — Registar Receção de Mercadoria

ID / Nome	UC11 — Registar Receção de Mercadoria
Ator Principal	Gerente de Loja
Descrição	Registar a receção de produtos entregues por fornecedor, atualizando *stock* e estado da encomenda.
Pré-condições	1) Gerente autenticado. 2) Encomenda com estado "Enviada" ou "Em Trânsito".
Pós-condições	1) *Stock* incrementado. 2) Encomenda "Recebida" ou "Recebida Parcialmente". 3) Movimentos de *stock* registados. 4) Discrepâncias registadas.
Fluxo Principal	1. "Encomendas Pendentes". 2. Selecionar encomenda. 3. Detalhe: qtd encomendada (readonly) e qtd recebida (input). 4. Verificar fisicamente e ajustar. 5. "Confirmar Receção". 6. Sistema compara recebido/encomendado. 7. Atualiza *stock* e regista movimentos.
Fluxos Alt.	**5a.** Entrega parcial (qtd parciais, encomenda fica pendente). **5b.** Produto danificado (observações).
Exceções	E1: Qtd recebida inválida. E2: Falha de persistência (*rollback*).
Regras de Negócio	RN01: Atualiza custo médio se preço diferir. RN02: Discrepâncias $> 5\%$ notificam Admin.
RF Associados	RF06, RF08, RF10, RF11

Tabela 7.10: Caso de uso UC11 — Registar Receção de Mercadoria.

UC12 — Consolidar Dados Diários

ID / Nome	UC12 — Consolidar Dados Diários
Ator Principal	Sistema (automático)
Descrição	Calcular e materializar agregados diários (vendas, movimentos de *stock*, KPIs, *rankings* de produtos) em tabelas de leitura denormalizadas, suportando o *dashboard* de gestão e os relatórios analíticos. Reformulado na Etapa 2 para refletir a topologia centralizada (ver ADR-006).
Pré-condições	1) Fecho de caixa realizado de pelo menos uma loja, ou *cron* agendado das 23h30 ativado. 2) Dados operacionais do dia disponíveis na BD central.
Pós-condições	1) Tabelas `KPIDiario` e `RankingProdutosDiario` atualizadas para a(s) loja(s) abrangida(s). 2) Registo `ConsolidacaoDiaria` criado com estado Sucesso/Falha. 3) Evento registado no `AuditService`.
Fluxo Principal	1. *Trigger*: fecho de caixa (UC04, propagado pelo SD-UC04) ou *cron* 23h30. 2. `ConsolidationScheduler` identifica lojas com fecho concluído e agregação pendente. 3. `ConsolidationService` colabora com `SalesService`, `InventoryService` e `ReportingService` para calcular agregados (vendas por hora, *ticket* médio, *top* produtos, valor de *stock*, alertas). 4. Resultados persistidos numa transação atómica nas tabelas materializadas. 5. Estado da consolidação marcado como "Sucesso" e evento de auditoria registado.
Fluxos Alt.	**1a.** Disparo manual por Administrador IT (`POST /api/admin/consolidacoes/disparar`). **3a.** Re-consolidação de dia anterior (sobrescreve agregados existentes para a mesma loja/data).
Exceções	E1: Falha de cálculo (*timeout*, *deadlock*, recurso saturado) $\rightarrow$ retentativa automática a 30, 60 e 90 min (ADR-010). Após 3 falhas, estado marcado como "Falha" e alerta crítico para o Administrador IT. E2: Inconsistência detetada (e.g., caixa não fechada que aparenta estar) $\rightarrow$ registado em *log* e ignorado nesse ciclo.
Regras de Negócio	RN01: Uma consolidação por loja/dia (chave única `(loja_id, data)` em `ConsolidacaoDiaria`). RN02: Os dados operacionais nunca são modificados nem apagados pela consolidação. RN03: A consolidação não bloqueia a operação POS nem a abertura de novas caixas. RN04: Tempo total < 5 min (RNF-02).
RF Associados	RF12, RF13, RNF-02, RNF-06, RNF-07

Tabela 7.11: Caso de uso UC12 — Consolidar Dados Diários.

UC13 — Consultar *Dashboard* de KPIs

ID / Nome	UC13 — Consultar *Dashboard* de KPIs
Ator Principal	Administrador da Cadeia
Descrição	*Dashboard* interativo com KPIs: vendas, *ticket* médio, *top* produtos, *stock* em alerta, margem por categoria.
Pré-condições	1) Utilizador autenticado (Admin/Gerente). 2) Dados consolidados existentes.
Pós-condições	1) *Dashboard* apresentado com KPIs atualizados.
Fluxo Principal	1. "Relatórios & Dashboards". 2. KPIs: total vendas, nº vendas, *ticket* médio, *top* 5 produtos, *stock* em alerta, margem por categoria, gráfico 30 dias. 3. Filtro por período e loja. 4. *Refresh* automático (60 s). 5. *Drill-down* em cada KPI.
Fluxos Alt.	**4a.** Gerente: filtrado automaticamente pela sua loja. **6a.** Gerar relatório → UC14.
Exceções	E1: Sem dados consolidados. E2: Erro de consulta.
Regras de Negócio	RN01: Gerentes veem só a sua loja. RN02: Admin vê todas. RN03: Dados da última consolidação bem-sucedida.
RF Associados	RF13, RNF-03

Tabela 7.12: Caso de uso UC13 — Consultar *Dashboard* de KPIs.

UC14 — Gerar e Exportar Relatórios

ID / Nome	UC14 — Gerar e Exportar Relatórios
Ator Principal	Administrador da Cadeia
Descrição	Gerar relatórios analíticos (vendas, *stock*, compras, financeiros) filtrados por loja/período/categoria, exportar em PDF/CSV.
Pré-condições	1) Utilizador autenticado. 2) Dados consolidados.
Pós-condições	1) Relatório gerado/exportado. 2) Acesso registado no *log*.
Fluxo Principal	1. Módulo "Relatórios". 2. Selecionar tipo (vendas/*stock*/compras/financeiro). 3. Filtros: loja, período (≤ 1 ano), categoria. 4. "Gerar Relatório" → dados agregados apresentados. 5. "Exportar" → PDF ou CSV.
Fluxos Alt.	**6a.** Sem dados para filtros. **2a.** Relatório personalizado (colunas à escolha).
Exceções	E1: Datas inválidas. E2: Intervalo > 365 dias. E3: Erro na exportação.
Regras de Negócio	RN01: Gerentes só a sua loja. RN02: Financeiros só para Admin e Contabilista. RN03: *Timestamp* e utilizador no relatório.
RF Associados	RF14, RNF-03

Tabela 7.13: Caso de uso UC14 — Gerar e Exportar Relatórios.

UC16 — Gerir Utilizadores e Perfis

ID / Nome	UC16 — Gerir Utilizadores e Perfis
Ator Principal	Administrador / Administrador IT
Descrição	Criar, editar e desativar contas, atribuir perfis (Admin, Gerente, Operador, IT) e associar a lojas.
Pré-condições	1) Administrador/IT autenticado.
Pós-condições	1) Utilizador criado/editado/desativado. 2) Associações loja registadas. 3) Credenciais enviadas por email. 4) *Log* de auditoria.
Fluxo Principal	1. "Gestão de Utilizadores". 2. "Novo Utilizador". 3. Preencher: nome, email (único), perfil, lojas. 4. Sistema gera *password* temporária (12 caracteres, bcrypt). 5. Registo criado com `password_temporaria=true`. 6. Email com credenciais enviado.
Fluxos Alt.	**3a.** Editar utilizador. **3b.** Desativar (*soft delete*). **3c.** Reativar. **10a.** *Reset* de *password*.
Exceções	E1: Email duplicado. E2: Email inválido. E3: Falha no envio de email (utilizador criado, alerta manual).
Regras de Negócio	RN01: *Soft delete* apenas. RN02: Máx. 1 Gerente ativo por loja. RN03: Admin tem acesso global. RN04: *Password* temporária obrigatória no 1.º *login*.
RF Associados	RF15, RF16, RF18, RNF-08, RNF-10

Tabela 7.14: Caso de uso UC16 — Gerir Utilizadores e Perfis.

UC17 — Editar Perfil Próprio

ID / Nome	UC17 — Editar Perfil Próprio
Ator Principal	Utilizador (qualquer perfil)
Descrição	Editar dados pessoais (nome, contacto, foto) e alterar *password*.
Pré-condições	1) Utilizador autenticado.
Pós-condições	1) Dados atualizados em `Utilizadores`. 2) Se *password* alterada: novo *hash*, `password_temporaria=false`. 3) *Log* de auditoria.
Fluxo Principal	1. "O Meu Perfil". 2. Formulário: nome (editável), email/perfil/lojas (readonly), contacto, foto. 3. Alterar e "Guardar".
Fluxos Alt.	**4a.** Alterar *password*: atual + nova + confirmação (complexidade: 8 car., maiúscula, minúscula, dígito, símbolo). **4b.** 1.º *login*: alteração obrigatória.
Exceções	E1: *Password* atual incorreta. E2: Complexidade insuficiente. E3: Confirmação não coincide. E4: Imagem inválida.
Regras de Negócio	RN01: Não pode alterar próprio email/perfil. RN02: *Password* temporária obrigatória. RN03: Nova $\neq$ últimas 3 *passwords*.
RF Associados	RF17, RNF-08, RNF-10

Tabela 7.15: Caso de uso UC17 — Editar Perfil Próprio.

UC18 — Consultar Logs de Auditoria

ID / Nome	UC18 — Consultar Logs de Auditoria
Ator Principal	Administrador / Administrador IT
Descrição	Consultar registos de auditoria (logins, CRUD, alterações) para segurança, rastreabilidade e conformidade.
Pré-condições	1) Administrador/IT autenticado. 2) Registos em `LogsAuditoria`.
Pós-condições	1) *Logs* apresentados conforme filtros. 2) Consulta registada no *log* (rastreabilidade total).
Fluxo Principal	1. "Auditoria & Segurança" $\rightarrow$ "Logs". 2. Lista paginada (50/página, desc): Data/Hora, Utilizador, Ação, Detalhes, Loja, IP, Resultado. 3. Filtros: período, utilizador, tipo, loja, resultado. 4. Clicar $\rightarrow$ detalhe completo (JSON antes/depois). 5. Exportar CSV/PDF.
Fluxos Alt.	**3a.** Pesquisa por texto livre. **6a.** Exportação em PDF.
Exceções	E1: Sem registos para filtros. E2: Erro na exportação.
Regras de Negócio	RN01: *Logs* imutáveis. RN02: Retenção mínima 5 anos. RN03: Acesso restrito a Admin. RN04: Consulta auto-registada.
RF Associados	RF18, RNF-06, RNF-09

Tabela 7.16: Caso de uso UC18 — Consultar Logs de Auditoria.

UC19 — Gerir Lojas da Cadeia

ID / Nome	UC19 — Gerir Lojas da Cadeia
Ator Principal	Administrador da Cadeia
Descrição	Registar, editar e desativar lojas; configurações operacionais (horários, *stocks* por defeito, associação de utilizadores).
Pré-condições	1) Administrador autenticado.
Pós-condições	1) Loja criada/atualizada/desativada. 2) `ConfiguracoesLoja` registadas. 3) `StockLoja` inicializado (qtd=0). 4) *Log* de auditoria.
Fluxo Principal	1. "Gestão de Lojas". 2. "Nova Loja". 3. Preencher: designação, morada, código postal, contacto, email, horário, *stock* mín./máx. por defeito. 4. Guardar $\rightarrow$ validação (designação única, máx. $>$ mín.). 5. `StockLoja` inicializado para todos os produtos ativos. 6. Associar utilizadores via UC16.
Fluxos Alt.	**3a.** Editar loja. **3b.** Desativar (impede se caixa aberta). **3c.** Reativar.
Exceções	E1: Designação duplicada. E2: Máx. $\leq$ mín. E3: Falha de persistência.
Regras de Negócio	RN01: Inativas mantêm dados históricos. RN02: *Soft delete*. RN03: Mín. 1 Gerente antes de operar.
RF Associados	RF19, RNF-01

Tabela 7.17: Caso de uso UC19 — Gerir Lojas da Cadeia.

Anexo C — Cronograma e Diagrama de Gantt

O cronograma completo do projeto, incluindo dependências e marcos de validação entre etapas, encontra-se representado no diagrama de Gantt produzido pela equipa. A atividade de preparação documental foi considerada desde o primeiro dia e decorreu em paralelo com as restantes atividades técnicas.

Anexo D — Diagrama UML Geral de Casos de Uso

Neste anexo apresenta-se o diagrama UML geral de casos de uso do sistema Belo Mercado Manager, com os principais atores e respetivas interações com elevado nível de detalhe. Os casos de uso em formato expandido mantêm-se no Anexo B.